

摘要

边缘应用的低时延需求对任务调度提出了不同要求。模型预部署的多深度神经网络（DNN）推理需在动态负载和并发干扰下协调批处理效率与请求截止期限；缓存受限的有向无环图（DAG）应用需结合任务依赖，协调服务副本布局与任务执行位置。本文围绕异构边缘环境开展上述两类研究，具体工作如下：

（1）针对多 DNN 在线推理中历史执行反馈利用不足、批处理等待与截止期限风险难以协调的问题，本文建立事件驱动的调度模型，并提出历史增强与风险感知调度方法 HERA。该方法利用门控循环单元（GRU）编码近期已完成批次的执行反馈，结合队列状态与请求紧迫程度过滤高风险候选动作，在离散软演员—评论家（SAC）框架下联合选择任务队列、批大小、执行设备与等待动作（WAIT），兼顾尾部时延、服务等级协议（SLA）违约率（超期请求占比）与执行能耗。基于实测性能剖面的仿真实验表明，对八个静态负载水平下的结果等权平均后，相较先来先服务（FCFS）方法，HERA 的平均 SLA 违约率和平均端到端第 99 百分位（P99）时延分别降低 14.24% 和 26.77%；在随机切换负载场景下，两项指标分别降低 8.79% 和 20.12%。

（2）针对缓存受限 DAG 应用的服务布局与任务卸载协调问题，本文建立联合优化模型，提出双时间尺度协调方法 TCOP。在慢时间尺度，中央协调器运行需求驱动的缓存协调机制（DCC），依据历史服务请求、执行反馈和既有副本分布，在各服务器的缓存容量约束下更新缓存配置；在快时间尺度，各入口服务器采用成对 Dueling Double 深度 Q 网络（Pairwise PD3QN）为就绪任务选择执行位置。该网络将任务与服务器特征配对，通过共享评分估计动作价值。基于五类科学工作流的仿真实验表明，在主实验配置下的冻结评估中，TCOP 的平均应用完成时间为 0.2668 s，较 DAOC 和采用相同 DCC 缓存机制的离散 SAC 分别降低 65.51% 和 11.09%。交叉组合与机制分析表明，TCOP 的整体性能改善主要体现在服务加载时延降低，并伴随服务覆盖扩大和本地缓存利用提高。

关键词：边缘计算；多 DNN 推理；任务卸载；服务缓存；深度强化学习

ABSTRACT

Low-Latency Task Scheduling Methods for Heterogeneous Edge Environments

Low-latency edge applications place different demands on task scheduling. Online inference with multiple predeployed deep neural networks (DNNs) must balance batching efficiency against request deadlines under dynamic workloads and execution interference. Cache-constrained directed acyclic graph (DAG) applications must coordinate service replica placement with task execution locations while satisfying task dependencies. This thesis studies these two scenarios in heterogeneous edge environments. The main contributions are as follows:

(1) To address underused historical execution feedback and the difficulty of balancing batching delays against deadline risks in online multi-DNN inference, this thesis develops an event-driven scheduling model and proposes HERA, a history-enhanced risk-aware scheduling method. HERA uses a gated recurrent unit (GRU) to encode execution feedback from recently completed batches and filters high-risk candidate actions using queue states and deadline urgency. Within a discrete soft actor-critic (SAC) framework, it jointly selects the task queue, batch size, execution device, and waiting action, accounting for tail latency, service-level agreement (SLA) violations, and execution energy. Simulations driven by measured performance profiles show that, with equal weighting across eight static load levels, HERA reduced the average SLA violation rate and average end-to-end 99th-percentile (P99) latency by 14.24% and 26.77%, respectively, relative to first-come, first-served (FCFS). Under randomly switching workloads, the corresponding reductions were 8.79% and 20.12%.

(2) To coordinate service placement and task offloading for cache-constrained DAG applications, this thesis develops a joint optimization model and proposes TCOP, a two-timescale coordination method. On the slow timescale, a central coordinator runs demand-driven coordinated caching (DCC), updating cache configurations under per-server ca-

capacity constraints using historical service requests, execution feedback, and the existing replica distribution. On the fast timescale, each ingress server uses a pairwise dueling double deep Q-network (Pairwise PD3QN) to select execution locations for ready tasks. The network pairs task and server features and estimates action values through a shared scoring function. In simulations using five types of scientific workflows, TCOP achieved an average application completion time of 0.2668 s in frozen evaluation under the main experimental configuration, reducing this time by 65.51% relative to DAOC and by 11.09% relative to discrete SAC with the same DCC caching mechanism. Cross-combination experiments and mechanism analyses indicate that TCOP's overall performance improvement is mainly reflected in lower service loading latency, alongside broader service coverage and greater local cache utilization.

Keywords: Edge computing; multi-DNN inference; task offloading; service caching; deep reinforcement learning

目 录

第一章 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	2
1.2.1 边缘计算任务卸载与资源调度研究	3
1.2.2 深度强化学习调度方法研究	4
1.2.3 面向多 DNN 推理的边缘资源管理研究	5
1.2.4 面向服务缓存与依赖任务的调度研究	7
1.2.5 小结与研究问题定位	9
1.2.6 研究挑战与问题定义	9
1.3 本文研究内容与创新点	10
1.4 论文组织结构	11
第二章 相关技术基础	13
2.1 边缘计算基础	13
2.1.1 异构边缘计算架构	13
2.1.2 任务调度与资源约束	13
2.2 多 DNN 推理基础	14
2.2.1 请求排队与批处理	14
2.2.2 执行干扰与时延指标	15
2.3 DAG 任务与服务缓存	16
2.3.1 任务依赖与完成时间	16
2.3.2 服务缓存与任务卸载	17
2.4 强化学习与历史编码	17
2.4.1 马尔可夫决策过程	17
2.4.2 GRU 历史编码	18
2.4.3 离散 SAC 算法	19
2.4.4 深度 Q 网络及其改进	20
2.5 本章小结	21

第三章 基于历史增强与风险感知的多 DNN 推理调度方法	22
3.1 多 DNN 推理调度问题引入	22
3.2 系统模型与问题建模	23
3.2.1 系统模型	24
3.2.2 推理任务与队列模型	24
3.2.3 批处理、时延与能耗模型	25
3.2.4 优化目标与问题定义	27
3.3 HERA 算法设计	29
3.3.1 在线调度过程建模	29
3.3.2 历史编码与策略网络	30
3.3.3 风险估计与 Gate 机制	31
3.3.4 在线决策流程与训练更新	33
3.3.5 复杂度分析	36
3.4 HERA 实验结果与分析	37
3.4.1 实验环境及参数设置	37
3.4.2 对比算法	40
3.4.3 静态负载下的性能分析	41
3.4.4 动态负载下的性能分析	41
3.4.5 不同 DNN 模型的性能分析	43
3.4.6 历史编码与风险门控的机制及消融分析	44
3.4.7 截止期限差异与请求构成变化分析	45
3.5 本章小结	47
第四章 面向异构缓存约束的服务协调与 DAG 任务卸载方法	48
4.1 DAG 任务卸载问题引入	48
4.2 系统模型与问题建模	49
4.2.1 系统模型	50
4.2.2 DAG 表示与通信模型	51
4.2.3 任务执行时间模型	52
4.2.4 服务缓存与服务获取模型	53
4.2.5 优化目标与问题定义	54

4.3	TCOP 算法设计	55
4.3.1	DCC 服务缓存协调机制	56
4.3.2	任务卸载决策过程建模	59
4.3.3	任务—服务器匹配评分网络	61
4.3.4	缓存更新、卸载决策与训练流程	62
4.3.5	复杂度分析	64
4.4	TCOP 实验结果与分析	65
4.4.1	实验环境及参数设置	65
4.4.2	对比算法	67
4.4.3	训练过程与验证表现分析	68
4.4.4	总体性能分析	69
4.4.5	不同 workflow 类型下的性能分析	70
4.4.6	计算与缓存资源共同扩展分析	71
4.4.7	缓存效果与时延分解分析	72
4.4.8	缓存策略与卸载策略的组合消融分析	73
4.5	本章小结	75
第五章	总结与展望	76
5.1	全文工作总结	76
5.2	未来工作展望	77
参考文献		78

表 目 录

表 3.1	本章主要符号说明	23
表 3.2	HERA 训练与调度参数	39
表 3.3	不同设备上的 DNN 实测性能	39
表 4.1	本章主要符号说明	49
表 4.2	网络与仿真参数	66
表 4.3	TCOP 主要超参数	67

图 目 录

图 1.1	论文研究内容与章节安排	12
图 3.1	HERA 在线调度与闭环反馈框架	29
图 3.2	静态负载下的总体性能	42
图 3.3	动态负载下的总体性能	42
图 3.4	不同 DNN 模型的服务性能	43
图 3.5	动态负载下总队列长度与风险门控激活率的变化	44
图 3.6	历史编码与风险门控的消融结果	44
图 3.7	截止期限差异的影响	46
图 3.8	请求构成变化的影响	46
图 4.1	DAG 任务卸载与服务缓存系统示意图	50
图 4.2	不同方法的训练验证表现	69
图 4.3	主环境下不同方法的平均与 P95 应用完成时间	70
图 4.4	不同工作流类型下各方法的平均应用完成时间	71
图 4.5	计算与缓存资源共同扩展时的平均应用完成时间	72
图 4.6	不同方法的服务覆盖与缓存利用	73
图 4.7	不同方法的应用完成时间与主要关键路径时延分量	74
图 4.8	不同缓存机制与卸载策略组合下的平均应用完成时间	74

第一章 绪论

1.1 研究背景与意义

实时视频分析、移动机器人和增强现实等应用需要在有限时间内完成数据处理与反馈，对计算系统的响应时延提出了较高要求。终端设备受计算能力、存储空间和能耗限制，难以满足部分计算密集型任务的实时处理要求；将任务交由远端云执行，又会引入数据上传和长距离网络传输开销。边缘计算通过在靠近用户和数据源的位置部署计算与存储资源，为时延敏感应用提供就近处理条件^[1, 2]。在此基础上，如何根据应用需求组织任务执行，是影响边缘服务质量的重要问题。

边缘环境中的资源异构性与运行状态变化，使低时延服务面临不同于单一计算节点的调度问题。不同设备的计算能力、并发执行能力和能耗特征存在差异，同一任务在不同设备上的执行代价并不相同。同时，请求到达强度与任务构成随时间变化，资源竞争也会改变设备的实际处理速度。调度决策因此需要结合任务需求和运行状态，在适当时刻将任务分配至合适的执行设备，避免局部资源拥塞或额外等待抵消就近计算带来的收益。

多深度神经网络（Deep Neural Network, DNN）在线推理是其中一类典型场景。边缘平台同时处理多种模型的推理请求，不同请求具有不同的计算需求和截止期限。即使模型已经预先部署，调度器仍需协调请求队列、批大小、执行设备与派发时机。批处理可以提高并行执行效率，但等待请求组成较大批次会消耗时延余量；较小批次能够减少组批等待，却可能降低计算资源的利用效率^[3]。因此，批处理规模与执行位置需要结合当前负载共同确定。

设备并发干扰和请求构成变化进一步增加了推理调度的不确定性。静态性能估计或单一时刻的队列状态，难以充分反映近期执行速度的变化，已完成批次的执行反馈因而具有利用价值。同时，较低的平均响应时延并不意味着所有请求均能及时完成，少量严重延迟的请求仍可能影响服务体验^[4]。面向这类场景，需要在控制执行能耗的同时，兼顾截止期限满足情况与尾部时延，研究能够利用历史反馈并识别高风险调度动作的在线决策方法。

另一类边缘应用由多个具有先后依赖关系的任务组成，可用有向无环图（Directed Acyclic Graph, DAG）描述其执行结构。后继任务需要等待前驱任务完成并获得相应输出，执行位置的选择会改变任务间的数据传输开销。各任务还可能依赖特定程序、容器镜像或模型组件，服务是否已在目标服务器缓存也会影响任务执行条件^[5, 6]。当服务器无法同时保存全部服务时，任务可能需要在执行前远程获取所需服务。因此，应用完成时间需要综合考虑依赖数据传输、服务加载和计算执行，而不能仅依据单个任务的计算耗时判断。

在缓存容量和计算能力均存在差异的多服务器环境中，服务配置与任务卸载相互影响。缓存副本的分布决定任务可以从何处获取服务，卸载决策则决定已有副本能否得到有效利用。过多保存相同服务可能压缩其他服务的覆盖范围，而单纯扩大覆盖又未必能够满足高频服务的就近访问需求。此外，缓存配置的周期性调整与逐任务执行位置选择具有不同的决策频率。如何协调跨服务器的服务布局，并使卸载决策同时考虑任务依赖与服务获取代价，是降低 DAG 应用完成时间需要解决的问题。

本文围绕异构边缘环境中的低时延任务调度，分别研究模型预部署条件下的多 DNN 独立推理请求调度与缓存受限条件下的 DAG 任务卸载。前者侧重动态负载和执行性能变化下的请求级时延控制，利用近期执行反馈协调批处理等待与请求紧迫度；后者侧重任务依赖与服务配置共同作用下的应用级完成效率，结合服务副本分布与前驱位置评估执行代价。两项研究从运行反馈与服务可用性两个角度分析调度决策的影响，有助于识别不同场景下的时延瓶颈，明确相应的调度决策重点。

1.2 国内外研究现状

国内外学者围绕边缘计算中的任务卸载与资源调度开展了研究，形成了数学优化、启发式搜索和深度强化学习等求解思路。随着多模型推理和依赖型应用部署到边缘，研究内容进一步涉及批处理、执行干扰、服务缓存与任务间数据传输。本节按任务卸载与资源调度、深度强化学习调度、多 DNN 推理资源管理、服务缓存与依赖任务调度四个方面梳理相关工作，并据此确定本文的研究问题。

1.2.1 边缘计算任务卸载与资源调度研究

任务卸载研究主要回答任务在何处执行，以及如何为执行过程分配通信和计算资源。Mao 等^[1]从通信视角梳理了移动边缘计算中的计算卸载、无线资源管理和能耗优化问题，说明传输与计算开销需要结合考虑。Islam 等^[7]进一步总结了多接入边缘计算的系统架构与卸载策略。两类综述反映出，卸载方案的效果与任务划分方式、可用执行节点、通信条件和优化目标有关，在不同模型下得到的结论需要结合具体假设理解。

对于由多个子任务组成的应用，调度粒度会影响资源协同方式。谢满德等^[8]研究云边端协同下的多用户细粒度任务卸载，先划分应用并确定子任务优先级，再利用改进的模拟退火粒子群算法搜索卸载方案，综合考虑时延、能耗和服务器租用成本。

端到端时延优化还需要覆盖任务执行前后的通信过程。李松等^[9]面向数字孪生边缘网络，将任务卸载、计算和结果反馈纳入全过程模型，联合优化设备关联、卸载比例、发射功率、传输带宽及数字孪生估计处理速率，并结合内部凸近似和匈牙利算法进行交替求解。与仅优化计算阶段的处理时间相比，这类工作扩展了时延的核算范围，使执行位置、链路资源与设备关联能够在同一目标下协调。

面对持续变化的请求和资源状态，在线优化研究进一步考虑当前决策对后续系统负载的影响。Sun 等^[10]在边云协同架构中联合优化通信资源分配与任务卸载，以用户设备能耗为目标，同时考虑任务截止期限和长期排队时延约束。其方法利用 Lyapunov 优化将时间耦合问题转化为实时决策问题，再通过分解和资源匹配求解。这表明，低时延调度不仅涉及当前任务的执行代价，也需要处理任务积压带来的长期影响。

上述研究分别从资源协同、任务粒度和在线决策等角度提高卸载效率。数学优化方法能够明确表达目标与约束，启发式搜索则为复杂组合决策提供了求解途径。对于多 DNN 在线推理，还需要描述组批等待、模型间的截止期限差异及设备执行干扰；对于 DAG 应用，还需要考虑前驱输出到齐与所需服务可用的条件。这些运行特征决定了通用卸载模型需要面向具体任务作进一步细化。

1.2.2 深度强化学习调度方法研究

深度强化学习通过与环境交互学习状态到动作的映射，为动态边缘环境中的连续决策提供了另一类方法。与每次请求到达后重新求解完整优化问题的方式不同，训练后的策略可根据观测直接选择动作，但其决策效果依赖状态表示、动作空间和反馈目标是否与实际调度问题一致。相关研究已从单节点选择扩展到历史信息利用、跨节点协作及分布式策略训练。

Tang 和 Wong^[11]针对不可分割、具有时延要求的任务，研究边缘负载不确定时的分布式卸载。该方法结合长短期记忆网络、Dueling DQN 与 Double DQN，在不掌握其他设备任务模型和卸载决策的条件下学习长期代价。历史负载信息由循环网络参与编码，使策略能够利用当前观测之外的时序信息。该方法的决策对象是任务是否卸载及目标边缘节点，而非多模型请求的组批与派发。

当终端数量增加时，智能体间的相互影响和联合动作空间规模成为重要问题。卢先领和李德康^[12]将大规模多接入边缘计算中的卸载建模为部分可观测决策过程，结合 LSTM 和平均场多智能体学习，用历史观测补充状态信息，并通过平均场近似降低联合动作表示的复杂度。该工作侧重设备数量增长下的分布式协作。

代钰等^[13]从跨服务器通信角度处理部分可观测问题，将边缘系统表示为图，通过图注意力机制交换节点消息，并利用双注意力机制筛选对策略优化更有用的信息。与仅利用单个智能体观测的方式相比，这类方法通过邻居消息补充空间信息。结合前述历史编码研究可以看出，补充观测既可以利用时间维度上的执行记录，也可以依靠节点之间的信息交换，其适用方式取决于系统允许获得的信息及通信开销。

面向包含依赖任务的物联网应用，Wang 等^[14]提出 TF-DDRL，将 Transformer 引入分布式 Actor-Critic 框架，并结合优先经验回放和离策略校正，处理异构边云资源上的应用调度。该研究将长期关联建模与分布式训练结合起来，使学习方法能够适应具有任务依赖的调度过程。

现有学习式调度已利用循环网络、图结构和注意力机制改善状态表示，并将历史信息纳入决策过程。本文进一步关注这些信息如何服务于两类具体决策：在

多 DNN 推理中，近期批次执行反馈需要与请求紧迫度、组批行为及候选动作风险相结合；在缓存受限的任务卸载中，状态需要反映任务需求、前驱位置与服务器服务配置，动作表示也需要支持不同候选位置之间的比较。

1.2.3 面向多 DNN 推理的边缘资源管理研究

面向 DNN 的边缘资源管理既包括模型计算在端、边、云之间的分配，也包括推理请求进入系统后的排队与执行。张智涵等^[15]从判别式和生成式模型的特点出发，梳理协同推理的时延与推理质量需求。Zhang 等^[16]提出 DVFO，通过深度强化学习联合调整边缘设备的 CPU、GPU 和内存频率，以及需要卸载到云端的特征计算，以协调时延和能耗。这类研究将模型执行方式与计算资源配置联系起来，其决策重点与完整模型预部署后的请求调度有所区别。

协同推理的通信过程同样需要调度。丁男等^[2]面向移动机器人的端边协同推理，在 ROS 2 中设计混合数据流动态调度方法，根据 DNN 中间数据的传输时间要求，在优先级优先与时间优先策略之间切换，并在数据分发服务流控制器中调整输出队列。其研究表明，模型分区后的计算收益还需要相应的通信调度支持；这类数据流调度与边缘服务器内部的推理请求组批属于不同层面的资源管理问题。

多模型共享执行设备时，资源竞争会影响单次推理的执行时间。Zhao 等^[17]提出 Miriam，通过弹性内核生成与运行时内核协调，支持边缘 GPU 上不同实时性要求的多 DNN 并发推理。Gujarati 等^[18]提出 Clockwork，从单次推理执行时间的可预测性出发组织模型服务和资源调度，以获得可预测的端到端性能。前者关注共享 GPU 上的细粒度竞争控制，后者强调执行过程的可预测性，二者都说明设备运行机制不能简单等同于固定的计算速率。

动态批处理是协调推理吞吐量与响应时延的重要方式。BCEdge^[3]利用强化学习联合调整批大小和并发执行配置，在多 DNN 服务中权衡吞吐量与服务等级目标（Service Level Objective, SLO）满足情况。该方法将批处理与设备并发相结合，避免孤立配置批大小。对于在线请求，批处理决策还会改变队列中请求的等待时间，因此需要同时关注计算效率和请求剩余时延余量，而不能仅以单批执行速度评价调度结果。

Zhang 等^[19]提出 Coinf, 将时延预测、任务紧迫性判断与优势演员—评论家 (A2C) 调度结合, 根据初步批大小和历史系统负载的 98% 分位统计预测执行时间, 再依据队列、运行模型及资源状态确定模型和批大小。Coinf 使用历史负载的汇总统计辅助预测; HERA 则将近期已完成批次的执行反馈按时间顺序输入 GRU, 形成供策略与价值网络使用的历史增强表示, 并依据在线时延估计、队列压力和剩余时限筛选候选动作, 再由离散 SAC 在筛选后的动作集合中联合选择请求队列、执行设备、批大小与等待动作。

针对不同应用具有不同服务等级目标的情况, Chen 等^[20]提出 HarmonyBatch, 将共享同一 DNN 模型、但具有不同 SLO 的应用请求分组, 并联合配置批大小与异构 CPU/GPU 无服务器函数资源, 以降低满足性能要求的资源成本。Nguyen 等^[21]提出 OctopInf, 面向边缘视频分析结合细粒度资源分配、自适应批处理和边缘设备与服务器之间的负载均衡, 进一步通过时空调度安排 GPU 上的推理任务共置。两项工作分别从异构资源配置和动态边缘执行环境拓展了批处理调度。

国内研究也关注多任务推理中的参数共享和负载均衡。魏学超等^[22]提出 PetS, 将参数高效 Transformer 任务表示为共享算子与任务专属算子的组合, 在输入长度、任务类型和负载条件下设计协同批处理, 并利用任务参数迁移改善多 GPU 负载分布。该方法利用了不同任务之间的参数共享关系, 与多个完整 DNN 模型分别预部署的场景不同。

模型服务的资源需求还可能受到数据变化与模型更新影响。吴舒霞等^[23]针对边缘实时视频分析设计 CL4VA, 利用精度下降预测、混合样本池和强化学习控制器选择样本并确定模型重训练时机。这类工作关注模型质量随数据漂移变化时的资源安排, 研究目标包含推理精度和更新开销; 本文的在线推理研究则保留给定模型结构, 将重点放在请求的执行调度上。

从这些研究可以看出, 多 DNN 推理的优化变量已覆盖计算频率、模型分区、内核执行、组批及资源配置。Dean 和 Barroso^[4]对大规模在线服务的研究指出, 短暂的高时延可能在系统规模和利用率提高时产生明显影响, 因此平均时延之外还需要考察时延分布尾部。针对动态多 DNN 请求, 本文进一步关注队列选择、执行

设备、批大小与等待动作的联合作用，利用近期执行反馈辅助决策，并同时评价截止期限违约率、尾部时延和执行能耗。

1.2.4 面向服务缓存与依赖任务的调度研究

服务缓存与任务卸载的联合研究将任务执行所需的软件环境纳入资源管理。Xu 等^[24]提出 OREO，基于 Lyapunov 优化和 Gibbs 采样联合处理动态服务缓存与任务卸载，考虑服务异构、空间需求耦合和在线决策。对于依赖任务，Zhao 等^[5]在预先给定、容量受限的服务缓存条件下研究卸载，以应用完成时间为目标设计求解方法。二者分别体现了动态服务配置和缓存条件下依赖任务执行的研究思路，说明服务可用性会影响卸载方案的代价与可行性。

围绕任务依赖本身，徐建等^[25]研究车联网中多个 DAG 应用的部分卸载，先根据执行优先级将任务图转化为序列，再采用基于循环神经网络的序列到序列策略网络学习卸载方案，优化时延与能耗的综合效用。与独立任务选择执行节点相比，这类研究在策略输入与决策顺序中体现了任务之间的先后关系，有助于协调局部卸载收益和应用整体完成过程。

在同时考虑依赖与服务需求方面，李云等^[6]建立 DAG 任务卸载、服务缓存与资源优化模型，采用多智能体深度确定性策略梯度方法，在应用最大可容忍时延等约束下优化系统能耗。该研究将动态计算请求与不完整系统信息纳入决策，并分析缓存命中情况。虽然同样涉及 DAG 任务与服务缓存，其优化重点是满足时延约束下的能耗，而非直接降低应用完成时间，两类目标对应的调度取舍有所不同。

Liang 等^[26]研究时延约束下的依赖任务卸载与服务缓存，综合考虑服务、传输和任务处理成本。对于服务预配置的子问题，其方法利用动态规划确定卸载方案，并在一般情形中采用 Gibbs 采样调整服务缓存。与在线学习策略相比，这类方法更强调给定问题结构下的优化求解及性能分析。

服务需求的时间和空间差异是缓存配置的重要依据。梁吉申等^[27]面向多用户、多边缘服务器系统，结合图注意力网络、LSTM 与多智能体强化学习，联合选择任务卸载、资源分配及服务缓存策略，优化平均时延与能耗成本。唐朝刚等^[28]面向车载边缘计算，将服务缓存、任务卸载和计算资源分配问题分解为缓存与卸载

联合决策以及计算资源分配两个子问题，并采用深度强化学习求解前一子问题。这些工作利用需求信息和问题分解减少联合决策的求解困难。

其他国内研究进一步将数字孪生和车辆移动性纳入联合优化。郑家瑜等^[29]构建数字孪生云边网络，将服务缓存与计算卸载问题解耦，并结合改进的深度强化学习和凸优化求解，以降低长期服务时延。左琳立等^[30]考虑车辆与路侧单元、基站交互的不确定性，通过多智能体学习联合优化服务缓存与任务迁移。两类方法分别关注系统状态映射与移动过程中的服务连续性，反映出缓存决策需要与任务所处的网络条件共同设计。

缓存配置和任务执行具有不同更新频率，双时间尺度方法因此成为一种联合决策思路。Yu 等^[31]提出 2Ts-DRL，将服务缓存配置与快速卸载、资源分配安排在不同时间尺度，并采用联邦学习进行分布式训练。对于批量提交的工作流，Chen 等^[32]提出 PAWSSP，先根据任务并行性确定服务布局，再安排任务次序、执行服务器和处理器核，以协调工作流调度与服务部署。前者强调更新频率的区分，后者强调批量工作流并行执行对服务布局的要求。

面向分布式 DAG 卸载，Golzari Oskoui 和 Sansò^[33]提出 DAOC，各云小站依据本地服务需求与加载代价独立、周期性地更新缓存，并广播缓存变化；入口云小站将全网缓存状态、前驱位置以及当前和后继任务的服务需求纳入 DQN 状态，结合动作引导选择执行服务器。该方法已在卸载决策中利用 DAG 依赖与跨节点缓存信息。本文 TCOP 面向缓存容量异构场景，在慢时间尺度由中央协调器运行 DCC，依据历史需求、执行反馈和既有副本分布估计加载节省，在各服务器的缓存容量约束下构造候选布局并进行覆盖修复。在快时间尺度，Pairwise PD3QN 将当前任务与各候选服务器特征配对，通过共享编码与评分结构估计动作价值，由入口服务器选择执行位置。两者的主要差异在于缓存配置的协调范围，以及 TCOP 对任务—服务器匹配关系的显式表示。

已有工作已经从预配置缓存扩展到动态缓存、双时间尺度决策和依赖任务联合优化。对于本文研究的缓存容量异构场景，需要在各服务器的缓存容量约束下协调服务副本分布，并使任务执行位置的选择反映前驱数据位置和服务获取代价。

缓存布局决定可用服务副本的分布，卸载策略决定任务如何利用这些副本，两者共同影响应用完成时间。

1.2.5 小结与研究问题定位

综上，通用任务卸载研究为通信、计算与能耗之间的协调提供了建模和求解方法，学习式调度进一步利用历史观测、节点协作和结构化状态应对环境变化。面向多 DNN 推理，现有方法在可调资源、模型执行方式及运行反馈利用上各有侧重，动态负载下近期执行反馈与请求风险的协调利用仍需研究。

服务缓存与依赖任务研究已采用联合优化和双时间尺度控制，将服务副本布局与任务执行位置共同纳入决策。在缓存容量异构的多服务器环境中，如何兼顾跨服务器服务覆盖与已有副本利用，仍是影响应用完成效率的问题。由此，本文聚焦动态执行状态感知与服务资源协调两方面的调度问题。

1.2.6 研究挑战与问题定义

(1) 动态执行状态的有效表征。多 DNN 请求的到达强度和构成不断变化，同一设备的实际执行时间还受到并发干扰影响。当前队列长度可以反映积压程度，却未必充分表达近期服务能力的变化。如何以有限的决策开销利用历史执行反馈补充当前观测，是在线推理调度需要解决的问题。

(2) 批处理收益与请求时延风险的协调。较大批次可能提高执行效率，但等待组批或派发到不合适的设备，会消耗请求的截止期限余量。队列、执行设备、批大小与派发时机共同影响调度结果，难点在于协调批处理收益与请求级时延风险，在控制执行能耗的同时降低截止期限违约率和尾部时延。

(3) 有限缓存中的服务覆盖与副本利用。DAG 任务既要满足前驱完成与数据到齐条件，也要获得执行所需的服务。服务器计算能力和缓存容量不同，使同一服务放置在不同位置具有不同收益。全网服务覆盖、热点副本利用和任务执行位置共同影响服务加载与数据传输，局部缓存命中率难以单独反映这些因素对应用完成时间的影响。

(4) 缓存更新与逐任务决策的协调。周期性服务配置面向一段时间内的需求，卸载决策则响应当前就绪任务及候选服务器状态。二者的决策频率与信息需求不同，且缓存配置影响任务的执行位置，任务需求和执行反馈又影响后续配置。如何协调这种相互作用，是降低平均应用完成时间的另一难点。

1.3 本文研究内容与创新点

针对上述问题，本文分别提出历史增强与风险感知的推理调度方法，以及服务缓存协调与任务卸载方法。具体研究内容与创新点如下。

(1) 基于历史增强与风险感知的多 DNN 推理调度。建立事件驱动的联合调度模型，将请求队列、执行设备、批大小和等待动作纳入决策。提出历史增强与风险感知调度方法 HERA，将近期运行反馈的时序编码与动作风险筛选结合：利用门控循环单元编码近期批次执行反馈，并结合在线时延估计、队列压力和请求紧迫度，对候选动作进行分阶段风险筛选。在筛选后的动作集合中采用离散软演员—评论家算法选择动作，根据各模型的截止期限违约率与目标值的偏差调整训练权重，并结合尾部时延风险代理和能耗惩罚引导策略学习。通过静态与动态负载实验、机制消融及敏感性分析，从截止期限违约率、端到端 P99 时延和执行能耗等方面评价所提方法。

(2) 面向异构缓存约束的服务协调与 DAG 任务卸载。建立考虑任务依赖、服务获取与异构缓存容量的联合优化模型，以平均应用完成时间为目标，提出双时间尺度服务缓存协调与成对 Q 学习卸载方法 TCOP，将跨服务器服务覆盖协调与任务—服务器成对价值估计结合。在慢时间尺度，设计需求驱动的跨服务器缓存协调机制，依据历史服务请求、预期服务加载时间节省、执行反馈和既有副本分布，在各服务器的缓存容量约束下构造候选布局，经覆盖修复与更新判定确定下一周期的缓存配置。在快时间尺度，各入口服务器将当前任务与候选服务器的特征配对，通过共享编码与评分结构估计动作价值，并利用反映应用完成时间增量的奖励学习卸载策略。基于五类科学 workflows 开展性能评估，并通过时延分解、缓存效果与缓存—卸载交叉组合实验，分析两类决策对应用完成时间的影响。

两项研究均以异构边缘资源中的低时延处理为目标，分别面向模型预部署的独立推理请求和缓存受限的 DAG 应用。前者侧重动态运行状态下的请求级时延控制，后者侧重任务依赖与服务可用性约束下的应用完成效率。两项研究分别采用与场景相应的运行模型和评价指标，从运行状态感知与服务资源协调两个角度形成互补的研究内容。

1.4 论文组织结构

本文共分为五章，研究内容与章节安排如图 1.1 所示。

第一章为绪论，介绍研究背景与现状，明确研究内容与创新点。

第二章介绍相关技术基础，依次阐述边缘计算、多 DNN 推理、DAG 任务与服务缓存，以及强化学习与历史编码，为后续两章的方法设计提供基础。

第三章面向多 DNN 推理，研究动态负载下的违约与尾部时延控制，介绍历史增强与风险感知调度方法，并分析实验结果。

第四章面向缓存受限 DAG 应用，研究服务缓存与任务卸载的联合优化，介绍双时间尺度决策方法，并分析实验结果。

第五章总结全文工作，分析研究不足，并展望后续研究方向。

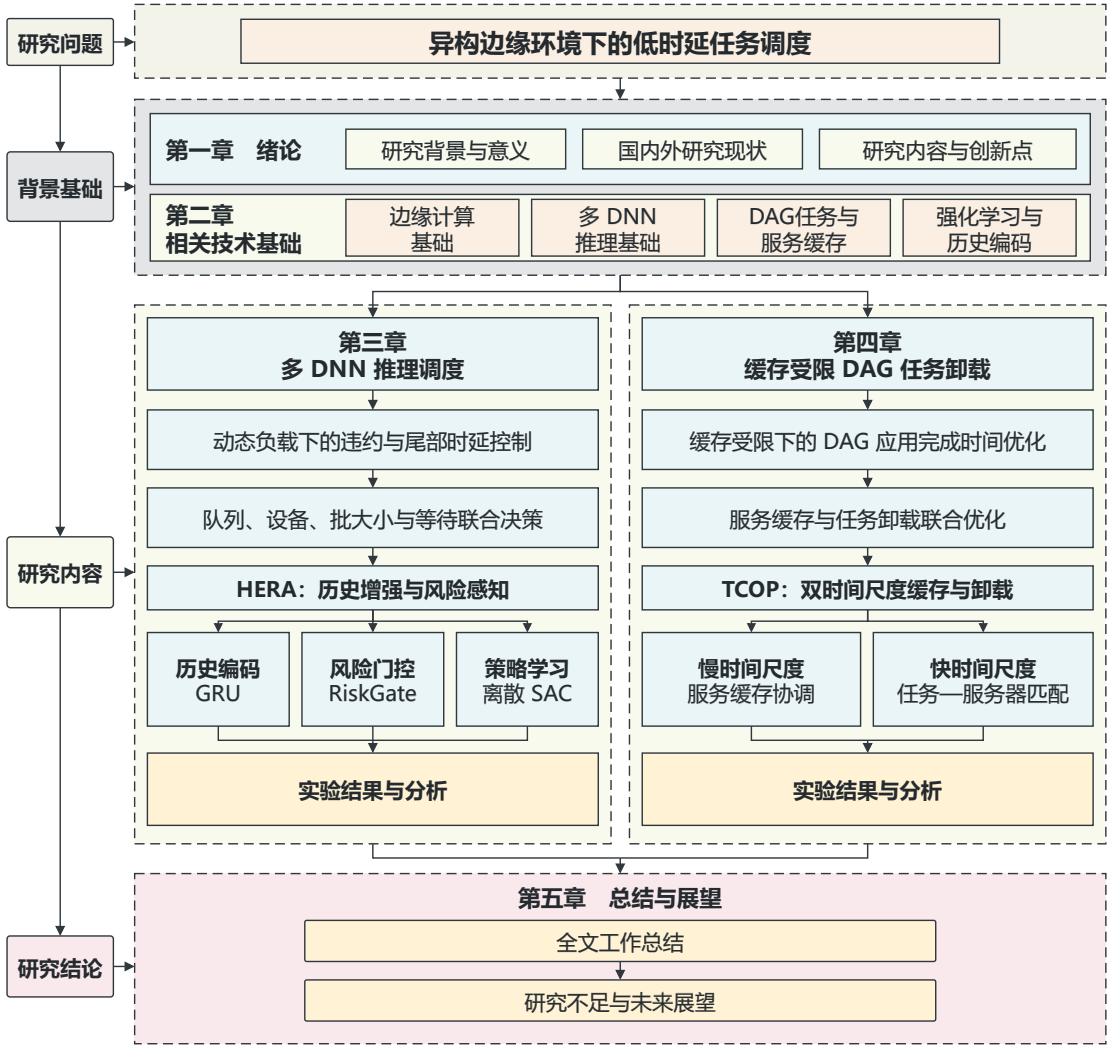


图 1.1 论文研究内容与章节安排

第二章 相关技术基础

本章介绍异构边缘环境下低时延任务调度的技术基础，依次说明边缘资源约束、多 DNN 推理、DAG 任务与服务缓存，以及强化学习和历史编码方法，为后续系统建模与算法设计提供基础。

2.1 边缘计算基础

2.1.1 异构边缘计算架构

边缘计算将计算与存储能力部署在靠近终端和数据源的位置。典型的云—边—端架构中，终端层生成数据并提交请求，边缘层就近处理任务，云端层提供集中式计算、存储与服务资源。边缘侧处理能够缩短数据传输路径，但实际响应时间仍受网络条件和节点负载影响^[1]。

异构性主要体现为节点资源配置及其运行能力的差异。不同边缘节点的处理器的类型、计算速度、可用内存、缓存容量和并发能力并不相同，同一任务在不同节点上的执行时间也可能存在差异。对于深度神经网络推理，CPU、GPU 等执行设备对不同模型和批大小的处理效率不同；多个模型同时运行时，计算单元和内存带宽的竞争还会改变实际执行速度^[3]。因此，节点的静态硬件配置与当前运行状态需要结合起来描述。

多节点部署允许调度器依据任务需求与资源状态选择执行位置。本文的多 DNN 推理场景中，模型已部署在所有候选设备上，调度重点是请求的执行安排；DAG 任务卸载场景则由入口服务器选择本地或其他可达边缘服务器，并考虑依赖数据传输与服务获取。两类场景的服务准备条件与调度范围有所不同。

2.1.2 任务调度与资源约束

任务调度确定执行位置、顺序、派发时机及资源使用方式，任务卸载侧重执行位置选择。持续到达的推理请求涉及队列选择、组批与派发；依赖任务则须满足前驱完成、数据到齐及服务可用等条件^[5]。

计算资源约束限定节点可承接的工作量。处理器速度影响计算耗时，并发容量限制同时执行的任务或批次数，共享资源竞争则可能延长实际执行时间^[3]。批处理可提高计算效率，但等待组批可能增加时延。不同设备的执行能耗也有差异，调度需兼顾时延与能耗要求。

通信约束影响输入上传、跨节点转发与依赖数据传输。计算能力更强的节点可能缩短计算时间，却增加传输开销。对于 DAG 应用，后继任务须获得所需前驱输出，因此执行位置还会影响其最早开始时间^[5]。

存储容量限制节点可长期保留的程序、运行库或模型等服务内容^[24]。缓存未命中时，任务可能需要远程获取服务后才能执行。缓存配置决定副本分布，卸载位置影响副本利用，两类决策共同影响服务获取代价。

2.2 多 DNN 推理基础

多 DNN 推理服务在同一边缘平台处理多个模型的请求，各类请求的到达频率、计算需求和时延要求不同。以下以模型已部署、请求相互独立的场景介绍请求组织与性能评价。

2.2.1 请求排队与批处理

在线推理请求包含待处理的输入数据、所需模型类型及相应的时延要求。请求经上行传输到达边缘入口后，可按模型类型进入不同队列，等待执行资源。

批处理将同一模型的多个请求组织为一个批次，由执行设备统一处理。一个批次包含的请求数量称为批大小。适当增大批大小有助于利用设备的并行计算能力，并分摊执行开销，提高单位时间内完成的请求数量。批大小与并发批次数是不同概念，前者描述单次执行包含多少请求，后者描述设备同时承接多少个批次，二者都会影响吞吐量和请求时延^[3]。

较大批次可能提高计算效率，但等待组批会消耗请求的可用时限；较小批次减少等待，却可能降低设备利用率。动态批处理依据队列、设备状态和请求紧迫程度调整批大小与派发时机。同批请求虽同时完成，生成时间不同，其端到端时延与期限满足情况也可能不同。

多模型共享设备时，组批需与队列选择、设备分配协调。队列长度仅反映积压数量，不能直接表示计算工作量或请求紧迫程度；是否立即派发还取决于设备可用性。因此，调度需共同考虑队列、设备、批大小与派发时机。

2.2.2 执行干扰与时延指标

给定模型、输入形式与硬件配置，性能测量可提供推理耗时的参考^[18]。共享设备上，多个批次可能竞争计算单元、内存带宽与硬件缓存，干扰程度随模型组合、批大小和并发状态变化^[3]。实际执行时间因而可能偏离离线测量值，需要结合当前占用与近期反馈估计。

端到端时延统计请求从终端生成至边缘完成推理的时间。记请求 i 的生成时刻为 g_i 、完成时刻为 t_i^{fin} ，所属批次为 $j(i)$ ，则端到端时延 L_i 为

$$L_i = t_i^{fin} - g_i = T_i^{tran} + T_i^{wait} + T_{j(i)}^{cmp} \quad (2.1)$$

其中， T_i^{tran} 为上行传输时延， T_i^{wait} 为请求到达边缘队列后至派发执行前的等待时延， $T_{j(i)}^{cmp}$ 为所属批次的实际执行时延。等待时延包含等待执行资源和等待组批的过程。每个请求均经历所在批次的完整执行时长。模型已预部署，因此该分解不包含模型加载与迁移开销。

相对截止期限规定请求从生成起可使用的时间，上行传输也计入其中。端到端时延超过所属模型的相对截止期限即为违约，违约请求数占评价请求总数的比例称为截止期限违约率，在第三章的多 DNN 推理实验中亦称服务等级协议（SLA）违约率。不同模型的时限可不同，总体违约率可结合分模型结果分析。

平均时延反映总体响应水平，尾部时延则描述少量较慢请求的情况，通常采用高百分位数衡量^[4]。P99 时延是观测端到端时延的第 99 百分位数，表示约 99% 的观测请求不超过该水平。

违约率反映期限满足情况，不区分超时程度；P99 反映高百分位响应水平，二者相互补充。比较时须统一负载与统计范围，并计入评价集合中全部请求的完成结果。第三章结合这两项指标和执行能耗评价调度效果。

2.3 DAG 任务与服务缓存

DAG 应用由相互依赖的任务组成，任务间传递中间结果并调用所需服务。与独立推理请求相比，此类应用需同时描述任务依赖与服务可用性，并考察整个应用的完成时间。

2.3.1 任务依赖与完成时间

有向无环图用有向边表示任务之间的依赖，且不存在沿有向边返回自身的闭合路径。记应用 m 的任务图为 $G_m = (V_m, E_m)$ ，其中 V_m 为任务集合， E_m 为依赖边集合。若 $(j, i) \in E_m$ ，任务 i 需要前驱任务 j 的输出，必须等待前驱完成并获得相应数据后才能开始执行^[5]。

沿有向边直接指向某任务的节点为其直接前驱，该任务直接指向的节点为其直接后继。无前驱和无后继的任务分别为入口与出口任务。无依赖的任务在输入和资源具备时可并行，汇合任务则须等待各前驱分支的数据到齐。

拓扑序保证每个前驱任务排在其后继之前，但不要求所有任务按这一顺序串行执行。任务的实际执行仍须满足数据就绪与资源可用条件。

前驱与后继位于不同服务器时，其输出需跨节点传输。多个前驱输出均为必需输入时，数据到齐时刻取各输出到达的最晚时刻，而非传输时延之和。因此，前驱的执行位置也影响后继启动时间。

应用在全出口任务完成后结束。记应用 m 的到达时刻为 A_m ，任务 i 的完成时刻为 $F_{m,i}$ ，出口集合为 V_m^{out} ，则应用完成时间 T_m 为

$$T_m = \max_{i \in V_m^{out}} (F_{m,i} - A_m). \quad (2.2)$$

单出口时，式 (2.2) 取该出口相对应用到达时刻的完成时间；多出口时，由最晚完成的出口确定。不同分支可以并行执行。第四章以平均应用完成时间评价卸载效果。

2.3.2 服务缓存与任务卸载

服务缓存保存任务执行所需的程序、运行库或模型等组件^[24]。服务镜像提供执行环境，任务输入与前驱输出则是处理数据，任务执行需同时满足服务可用和数据就绪条件。

服务缓存命中是指执行服务器已保存任务所需的服务。未命中时，可从其他服务器或云端加载所需服务，加载开销取决于镜像大小与链路条件。本地命中则不产生远程服务加载时延。

不同服务器的缓存容量可以不同。同一服务可在多节点保存副本，但均须满足容量约束。热点服务副本便于就近访问，也挤占其他服务空间，需要按需求分布协调副本数量与服务种类覆盖。

卸载决策需权衡计算、数据传输和服务获取代价。已缓存服务的节点可减少加载开销，但计算速度或后台负载未必有利；计算较快的节点也可能增加传输与加载耗时。因此，缓存命中率不能单独代表应用完成效率，还需结合前驱位置与节点能力判断。

服务缓存与任务卸载可按不同频率更新，前者面向一段时间的需求，后者随当前任务变化^[31]。第四章采用双时间尺度，协调周期性服务布局与逐任务卸载。

2.4 强化学习与历史编码

调度动作影响队列、资源占用及后续任务执行。强化学习评价动作的长期效果，历史编码补充当前观测未反映的运行信息。本节介绍相应的决策过程与算法基础。

2.4.1 马尔可夫决策过程

马尔可夫决策过程（Markov Decision Process, MDP）用于描述智能体与环境的序贯交互，可表示为 $M = (S, A, P, R, \gamma)$ 。其中， S 为状态空间， A 为动作空间， P 为状态转移概率， R 为奖励函数， γ 为折扣因子。在第 k 个决策步，智能体根据状态 s_k 选择动作 a_k ，环境返回奖励 r_k 并转移至下一状态。马尔可夫性要求，在给定当前状态与动作后，下一状态的分布不再依赖更早的交互历史^[34]。

策略 $\pi(a|s)$ 给出状态 s 下选择动作 a 的概率。对于包含 K 个决策步的有限轨迹，策略的期望累计回报可写为

$$J(\pi) = E_{\pi} \left[\sum_{k=0}^{K-1} \gamma^k r_k \right] \quad (2.3)$$

其中， $0 \leq \gamma \leq 1$ ；折扣因子越小，未来奖励权重衰减越快，取 1 时为无折扣回报。状态价值 $V^{\pi}(s)$ 表示从当前状态执行策略的期望回报，动作价值 $Q^{\pi}(s, a)$ 进一步指定当前动作。

决策步并不必然对应等长的物理时间间隔。事件驱动调度中，请求到达、执行完成等事件触发状态更新，相邻决策的持续时间可以不同，因而可采用半马尔可夫决策过程（Semi-Markov Decision Process, SMDP）描述^[34]。若按物理时间折扣，可令单次转移的折扣系数为 $\gamma_k = \exp(-\beta \Delta t_k)$ ，其中 β 为时间折扣率， Δt_k 为转移持续时间；同一时刻连续决策时，该系数为 1。第三章采用事件驱动的交互方式，第四章的决策步则表示任务分配次序，不能将两者的步数直接理解为相同长度的运行时间。

当前观测未必包含完整运行状态，近期执行反馈的编码可补充设备变化信息。

2.4.2 GRU 历史编码

门控循环单元（Gated Recurrent Unit, GRU）通过隐藏状态递归处理有序输入，并利用更新门和重置门控制历史信息的保留与更新^[35]。设序列第 ℓ 项输入为 $\mathbf{x}_{\ell}$ ，上一项隐藏状态为 $\mathbf{h}_{\ell-1}$ ，采用更新门控制历史保留比例的记法，其基本计算过程为

$$\begin{aligned} \mathbf{z}_{\ell} &= \sigma(\mathbf{W}_z \mathbf{x}_{\ell} + \mathbf{U}_z \mathbf{h}_{\ell-1} + \mathbf{b}_z) \\ \mathbf{r}_{\ell} &= \sigma(\mathbf{W}_r \mathbf{x}_{\ell} + \mathbf{U}_r \mathbf{h}_{\ell-1} + \mathbf{b}_r) \\ \tilde{\mathbf{h}}_{\ell} &= \tanh(\mathbf{W}_h \mathbf{x}_{\ell} + \mathbf{U}_h (\mathbf{r}_{\ell} \odot \mathbf{h}_{\ell-1}) + \mathbf{b}_h) \\ \mathbf{h}_{\ell} &= \mathbf{z}_{\ell} \odot \mathbf{h}_{\ell-1} + (1 - \mathbf{z}_{\ell}) \odot \tilde{\mathbf{h}}_{\ell} \end{aligned} \quad (2.4)$$

其中, $\mathbf{z}_\ell$ 和 $\mathbf{r}_\ell$ 分别为更新门与重置门, $\tilde{\mathbf{h}}_\ell$ 为候选隐藏状态; $\mathbf{W}$ 、 $\mathbf{U}$ 表示相应的权重矩阵, $\mathbf{b}$ 表示偏置, 均通过训练获得。 σ 为 Sigmoid 函数, 门值处于 0 与 1 之间; $\tanh$ 为双曲正切函数, $\odot$ 表示逐元素乘法。式 (2.4) 中的下标 ℓ 表示历史序列位置, 与环境决策步 k 区分。

重置门控制候选状态使用的旧信息, 更新门控制旧状态与候选状态的混合。按式 (2.4), 更新门越接近 1, 保留历史越多; 越接近 0, 则更多采用候选状态。

用于边缘推理调度时, 可将近期完成批次的模型类型、执行设备、批大小、执行时延与能耗组织为反馈序列, 再将编码结果与当前队列及设备观测结合。历史序列仅包含已完成批次的反馈, 并按实际获得反馈的顺序构建。第三章由 GRU 提取近期执行反馈, 以补充静态参考性能与当前观测; 该编码器不直接决定调度动作, 最终决策仍由策略网络完成。

2.4.3 离散 SAC 算法

软演员—评论家算法 (Soft Actor-Critic, SAC) 采用最大熵框架, 在提高累计奖励的同时保留策略随机性^[36]。演员输出动作策略, 评论家估计动作价值。离散 SAC 用有限动作集上的概率分布表示策略, 通过求和计算动作期望^[37]。

记 $A(s)$ 为状态 s 下的可用动作集合, 策略熵 $H(\pi(\cdot|s))$ 为动作负对数概率的期望。最大熵目标为

$$J_{ME}(\pi) = E_\pi \left[\sum_{k=0}^{K-1} \gamma^k [r_k + \alpha H(\pi(\cdot|s_k))] \right] \quad (2.5)$$

其中, 熵温度系数 α 控制奖励与策略熵的相对权重。采用双评论家时, 软状态价值为

$$V(s) = \sum_{a \in A(s)} \pi_\theta(a|s) \left[\min_{j=1,2} Q_{\psi_j}(s, a) - \alpha \ln \pi_\theta(a|s) \right] \quad (2.6)$$

其中, θ 为策略参数, ψ_j 为第 j 个评论家的参数。取两个价值估计的较小值以缓解高估。将式 (2.6) 中的评论家替换为目标评论家, 所得软价值记为 $\bar{V}(s)$, 一步训练目标为

$$y_k = r_k + \gamma(1 - d_k) \bar{V}(s_{k+1}) \quad (2.7)$$

其中, $d_k = 1$ 表示转移后轨迹终止, 此时不再使用后续价值。评论家通过减小预测值与目标值的误差更新; 目标网络以较慢速度跟随在线网络, 为估计提供相对稳定的参照。对给定状态, 演员最小化以下损失:

$$\ell_\pi(s) = \sum_{a \in A(s)} \pi_\theta(a|s) \left[\alpha \ln \pi_\theta(a|s) - \min_{j=1,2} Q_{\psi_j}(s, a) \right] \quad (2.8)$$

演员据此提高高价值动作的概率, 并受熵项约束。离散 SAC 通过经验回放复用样本, 属于离策略学习。式 (2.5)–(2.8) 为固定步折扣的基础形式; 第三章采用历史增强观测, 按实际时间累计奖励并构造多步目标。

2.4.4 深度 Q 网络及其改进

深度 Q 网络 (Deep Q-Network, DQN) 以神经网络近似动作价值函数, 直接比较离散动作的长期回报^[38]。经验回放从已存储的交互样本中抽样, 减弱连续样本的相关性; 目标网络延迟更新, 避免训练目标随在线参数每次变化而同步改变。记在线网络参数为 θ , 目标网络参数为 $\bar{\theta}$, 一步目标为

$$y_k^{DQN} = r_k + \gamma(1 - d_k) \max_{a' \in A(s_{k+1})} Q_{\bar{\theta}}(s_{k+1}, a') \quad (2.9)$$

训练减小 $Q_\theta(s_k, a_k)$ 与目标值的差距。 ϵ -greedy 策略以一定概率探索, 其余选择估计价值最大的动作, 评估时可关闭探索。最大值操作同时用于动作选择与评价, 估计噪声可能引起价值高估。

Double DQN 用在线网络选择下一动作, 再由目标网络评价, 以减轻价值高估^[39]。其目标为

$$\begin{aligned} a^* &= \arg \max_{a' \in A(s_{k+1})} Q_\theta(s_{k+1}, a') \\ y_k^{DDQN} &= r_k + \gamma(1 - d_k) Q_{\bar{\theta}}(s_{k+1}, a^*) \end{aligned} \quad (2.10)$$

这一处理不同于 SAC 对双评论家取较小值的方式。终止样本只保留实际奖励，不再估计后续价值。

Dueling 网络将动作价值分为状态价值分支和动作优势分支，分别描述当前状态的整体价值与不同动作的相对差异^[40]。采用优势均值归一化时，动作价值表示为

$$Q_{\theta}(s, a) = V_{\theta}(s) + A_{\theta}(s, a) - \frac{1}{|A(s)|} \sum_{a' \in A(s)} A_{\theta}(s, a') \quad (2.11)$$

其中， $A_{\theta}(s, a)$ 、 $V_{\theta}(s)$ 分别为优势与状态价值， $|A(s)|$ 为可用动作数。减去优势均值后，状态价值对应动作价值的平均水平，优势项保留动作间差异，有助于效果相近的动作共享状态学习信息。

Double DQN 与 Dueling 分别改进目标值和网络表示，可结合使用。多步回报累计若干步奖励，并在非终止端点引入后续价值。第四章据此构建任务与候选服务器的成对编码和价值估计。

2.5 本章小结

本章介绍了异构边缘环境下低时延任务调度的相关技术，说明了计算、通信与存储资源对任务执行的约束，梳理了多 DNN 推理的排队、批处理、执行干扰与时延指标，以及 DAG 任务依赖与完成时间计算、服务缓存与任务卸载的联系。随后介绍了马尔可夫决策过程、GRU 历史编码、离散 SAC、深度 Q 网络及其改进的基本原理，为后续建模与算法设计提供基础。

第三章面向模型预部署的独立推理请求，研究动态环境下的在线调度；第四章面向具有任务依赖和缓存容量约束的 DAG 应用，研究服务缓存协调和任务卸载。两类研究共享异构边缘资源背景，但分别关注请求级时延与应用整体完成时间，所需的状态描述和决策方式也有所不同。

第三章 基于历史增强与风险感知的 多 DNN 推理调度方法

3.1 多 DNN 推理调度问题引入

本章研究异构边缘平台上的多 DNN 在线推理调度问题。不同模型的请求持续到达，其计算需求、到达强度与截止时间存在差异。各模型已预部署在所有候选设备上，每个请求由一个设备完成完整推理，请求之间不存在任务依赖。在上述条件下，调度器需要根据队列积压和设备运行状态安排请求执行，使有限资源能够及时响应不同模型的服务需求。

批处理有助于提高执行效率，但组批等待会消耗请求的剩余时限^[3]。较大的批次可能提高吞吐量，却未必适合紧迫请求；选择较小批次立即执行，也可能降低资源利用率。同时，不同设备对模型与批大小的处理能力不同，多个批次并发运行还会产生资源竞争。队列选择、设备分配、批大小和派发时机相互影响，单独调整其中一项难以充分协调执行效率与时延风险。

动态环境使上述权衡进一步复杂化。请求强度与模型构成的变化会改变队列压力，并发干扰则使实际执行耗时偏离离线参考值。离线性能测量可提供执行时间参考^[18]，但仅凭该参考值或当前队列快照，难以及时反映设备性能的变化。近期已完成批次的时延与能耗记录能够补充设备运行信息，结合请求剩余时限，可为判断候选动作是否加剧积压和超时提供依据。因此，调度需要同时感知近期执行变化与当前请求紧迫程度。

针对上述问题，本章提出基于历史增强与风险感知的多 DNN 推理调度方法。通过编码近期执行反馈补充状态表示，并结合在线时延估计、队列压力和请求紧迫程度筛选候选动作，在此基础上学习联合调度策略。本章以降低截止时间违约率与端到端尾部时延为目标，同时考虑执行能耗。后续依次给出系统模型与问题定义、调度算法设计及实验结果分析。

3.2 系统模型与问题建模

本节依次定义多 DNN 在线推理调度中的设备资源、请求队列、执行开销和优化目标。以 k 表示调度决策索引，以 t 表示物理时间，相邻决策的物理时间间隔不固定。本章主要符号见表 3.1。

表 3.1 本章主要符号说明

符号	含义	单位或取值
$\mathcal{M}, \mathcal{D}$	DNN 服务集合与执行设备集合	-
$C_d, n_d(t_k)$	设备并发上限与当前运行批次	批次
$\mathcal{B}_{m,d}, b$	允许的批大小集合与所选批大小	$b \in \mathcal{B}_{m,d}$
k	调度决策索引	非负整数
$t, t_k, \Delta t_k$	物理时间、决策时刻与转移时长	s
$Q_m(t), q_m(t)$	模型请求队列及其长度	队列；请求数
g_i, t_i^{arr}	请求生成时刻与边缘到达时刻	s
$\delta_m, \tau_i^{ddl}, s_i(t)$	相对截止期限、绝对截止期限与剩余时限	s
$\lambda_m^{gen}(t), \hat{\lambda}_{m,k}$	终端请求生成强度与边缘到达率估计	请求/s
$\bar{T}_{m,d,b}, \hat{T}_k(a)$	参考与预测批次执行时延	s
$\bar{\mu}_{m,d,b}, \hat{\mu}_k(a)$	参考与预测服务率	请求/s
$\Gamma_j(t)$	批次运行减速因子	$[1, \Gamma_{\max}]$
$\bar{E}_{m,d,b}^{base}, \bar{E}^{req}(K)$	参考与平均单请求执行能耗	J/请求
$T_i^{tran}, T_i^{wait}, T_j^{cmp}$	上传、排队组批与批次计算时延	s
$L_i, P99(\mathcal{R})$	请求端到端时延与评价集合的 P99	s
a_k	队列、设备、批大小联合派发或等待动作	(m, d, b) 或 WAIT
$\mathcal{A}_k^{phy}, \mathcal{A}_k^{gate}$	物理可行动作集合与门控动作集合	-
$\mathbf{s}_k, \mathcal{H}_k, \mathbf{x}_k$	当前观测、历史记录与网络输入	-
H	历史窗口保留的已完成批次	批次
$u_{m,k}, p_{m,k}$	归一化队首剩余时限与队列压力	无量纲
$v_m(\mathcal{R}), \varepsilon_m$	模型级违约率及其目标	$[0, 1]$
E_{ref}, B_E^J	能耗归一化参考与单请求能耗预算	J/请求
$\tilde{E}_k, \tilde{B}_E, c_k^E$	归一化转移能耗、单请求预算与超额成本	归一化量
$R_k^{tail}, C_k^{tail}, \tilde{C}_k^{tail}$	瞬时风险、原始与回写后的尾部成本	无量纲
$\lambda_{m,e}^{SLA}$	模型在当前训练回合的 SLA 惩罚权重	$[\lambda_{\min}, \lambda_{\max}]$

3.2.1 系统模型

系统由请求生成终端、边缘调度器和异构执行设备组成。记 DNN 服务集合为 $\mathcal{M}$ ，执行设备集合为 $\mathcal{D}$ 。调度器为每种模型维护一个先进先出（FIFO）队列，并集中决定队列选择、执行设备、批大小及派发时机。每个请求仅调用一种 DNN 服务，由所选设备完成完整推理；同一批次只包含同一模型请求。

设备 d 最多同时运行 C_d 个批次，模型 m 在该设备上允许采用的批大小集合为 $\mathcal{B}_{m,d}$ 。执行槽位共同表征内存与并发容量约束。所有模型均已部署在全部设备上，因此本章不单独计入模型加载、迁移和切换开销，也不设置模型缓存更新决策。不同设备对同一模型和批大小具有不同的执行性能，其差异由参考时延与能耗参数描述。

3.2.2 推理任务与队列模型

请求 i 表示为 $\mathcal{J}_i = (g_i, m_i, \delta_{m_i})$ ，其中 g_i 为终端生成时刻， m_i 为模型类型， δ_{m_i} 为该类型的相对截止期限。请求经过时长为 T_i^{tran} 的上行传输后进入边缘队列。其边缘到达时刻、绝对截止期限和时刻 t 的剩余时限分别为

$$t_i^{arr} = g_i + T_i^{tran}, \tau_i^{ddl} = g_i + \delta_{m_i}, s_i(t) = \tau_i^{ddl} - t \quad (3.1)$$

截止期限从终端生成请求时起算，因此上传过程也会消耗剩余时限。模型 m 的请求在终端侧按照强度为 $\lambda_m^{gen}(t)$ 的泊松过程生成，该强度随时间分段变化；边缘实际到达还受到上行时延变化的影响。记模型队列为 $Q_m(t)$ 、队列长度为 $q_m(t)$ ，请求按到达边缘的先后顺序排列。

在决策时刻 t_k ，调度器根据已观测的请求到达、批次完成或达到等待上限的事件更新系统状态。派发动作 $a_k = (m, d, b)$ 将队列 m 中最前面的 b 个请求送至设备 d ；等待动作（WAIT）则使系统推进到下一可观测事件。记设备当前运行的批次数为 $n_d(t_k)$ ，物理可行动作集合为

$$\mathcal{A}_k^{phy} = \{\text{WAIT}\} \cup \left\{ (m, d, b) \mid \begin{array}{l} m \in \mathcal{M}, d \in \mathcal{D}, b \in \mathcal{B}_{m,d} \\ q_m(t_k) \geq b, n_d(t_k) < C_d \end{array} \right\} \quad (3.2)$$

式 (3.2) 要求派发批大小属于设备支持范围、队列中有足够请求，且设备尚有空闲执行槽位。派发后立即更新队列和设备占用，允许在同一物理时刻继续作出调度决策。令 $A_m(t_k, t_{k+1})$ 表示区间 $(t_k, t_{k+1}]$ 内完成上传并进入模型 m 队列的请求数，则第 k 次决策的派发量及队列更新关系为

$$\begin{aligned} D_{m,k} &= \sum_{d \in \mathcal{D}} \sum_{b \in \mathcal{B}_{m,d}} b \mathbb{1}[a_k = (m, d, b)] \\ q_m(t_{k+1}) &= q_m(t_k) + A_m(t_k, t_{k+1}) - D_{m,k} \end{aligned} \quad (3.3)$$

其中， $\mathbb{1}[\cdot]$ 为示性函数。执行 WAIT 时，各队列的派发量为零，请求继续等待，剩余时限随物理时间推进而减少。式 (3.3) 中的到达量统计边缘入队请求，不能直接用同一区间的终端生成数量替代。

3.2.3 批处理、时延与能耗模型

对于模型、设备和批大小组合 (m, d, b) ，离线测量给出参考批次执行时延 $\bar{T}_{m,d,b}$ 和参考单请求执行能耗 $\bar{E}_{m,d,b}^{base}$ 。相应的服务特征及参考服务率为

$$\mathcal{P}_{m,d,b} = \left(\bar{T}_{m,d,b}, \bar{E}_{m,d,b}^{base}, \bar{\mu}_{m,d,b} \right), \bar{\mu}_{m,d,b} = \frac{b}{\bar{T}_{m,d,b}} \quad (3.4)$$

参考时延描述该组合的基准性能，实际执行进度还受到并发干扰影响。设批次 j 包含模型 m_j 的 b_j 个请求，在设备 d_j 上于 σ_j 开始执行，于 t_j^{fin} 完成。记 $C_j(t)$ 为同设备上其他并发批次的集合， $\chi_{jt}(t) \geq 0$ 为批次间的有效干扰强度，则减速因子和批次完成条件为

$$\Gamma_j(t) = \min \left\{ \Gamma_{\max}, 1 + \sum_{\ell \in C_j(t)} \chi_{j\ell}(t) \right\} \quad (3.5)$$

$$\int_{\sigma_j}^{t_j^{fin}} \frac{1}{\Gamma_j(t) \bar{T}_{m_j, d_j, b_j}} dt = 1$$

其中, $\Gamma_{\max}$ 为环境设定的减速上限, 满足 $1 \leq \Gamma_j(t) \leq \Gamma_{\max}$ 。积分项累计批次已完成的归一化工作量; 干扰增强会减慢执行进度, 并推迟完成时刻。实际计算时延为 $T_j^{cmp} = t_j^{fin} - \sigma_j$ 。干扰强度不直接提供给调度策略, 策略利用可观测状态及已完成批次的反馈感知执行性能变化。

批次执行能耗按下式估计:

$$\widehat{E}_j = b_j \bar{E}_{m_j, d_j, b_j}^{base} \left[1 + \eta_E \left(\Gamma_j(\sigma_j) - 1 \right) \right] \quad (3.6)$$

其中, $\eta_E \geq 0$ 控制能耗对干扰的敏感程度, $\Gamma_j(\sigma_j)$ 为派发时刻的减速因子。批次执行能耗按式 (3.6) 在派发时估计一次, 由参考单请求能耗、批次请求数和派发时刻的减速因子确定。后续干扰变化会影响式 (3.5) 中的执行进度, 但不重新估计该批次能耗。该值是用于调度的近似估计, 而非设备总功率的连续积分; 若同一时刻的多个派发动作逐次结算, 估计值还可能受事件处理顺序影响。

对输入数据量为 z_i 的请求, 采用香农容量形式描述上行传输速率^[1], 有

$$R_i = B_i \log_2 \left(1 + \frac{P_i h_i}{N_0 B_i + I_i} \right), T_i^{tran} = \frac{z_i}{R_i} \quad (3.7)$$

其中, B_i 、 P_i 、 h_i 、 N_0 和 I_i 分别为可用带宽、终端发射功率、信道功率增益、噪声功率谱密度及链路聚合干扰功率。分母 $N_0 B_i + I_i$ 表示噪声与干扰总功率。

记 $j(i)$ 为请求 i 所属批次。请求在边缘的排队与组批等待时延, 以及从终端生成至边缘完成推理的端到端时延分别为

$$\begin{aligned}
 T_i^{wait} &= \sigma_{j(i)} - t_i^{arr} \\
 L_i &= t_i^{fin} - g_i = T_i^{tran} + T_i^{wait} + T_{j(i)}^{cmp}
 \end{aligned} \tag{3.8}$$

同一批次内的请求在该批次完成时结束推理，但由于生成及到达时刻不同，其等待时延和端到端时延可以不同。等待项从进入边缘队列时开始计算，不重复包含上传时延。

3.2.4 优化目标与问题定义

本章从截止期限违约、尾部时延和执行能耗三个方面评价调度效果。设评价请求集合为 $\mathcal{R}$ ，其中模型 m 的请求构成子集 $\mathcal{R}_m = \{i \in \mathcal{R} : m_i = m\}$ 。当该子集非空时，模型级违约率及策略 π 下的总体违约率定义为

$$\begin{aligned}
 \nu_m(\mathcal{R}) &= \frac{\sum_{i \in \mathcal{R}_m} \mathbb{1}[L_i > \delta_{m_i}]}{|\mathcal{R}_m|} \\
 J_{SLA}^\pi(\mathcal{R}) &= \frac{\sum_{i \in \mathcal{R}} \mathbb{1}[L_i^\pi > \delta_{m_i}]}{|\mathcal{R}|}
 \end{aligned} \tag{3.9}$$

上标 π 表示策略产生的完成结果，总体违约率按模型请求数加权。评价窗口结束后，系统继续运行至 $\mathcal{R}$ 中所有请求完成，避免遗漏未完成请求； $P99(\mathcal{R})$ 为该集合实际端到端时延的第 99 百分位数。

在前 K 次决策中，令 $\widehat{E}_k$ 为第 k 次派发对应的批次执行能耗估计， n_k 为派发请求数。WAIT 的两项取值均为零。当累计派发请求数大于零时，平均单请求执行能耗为

$$\overline{E}^{req}(K) = \frac{\sum_{k=0}^{K-1} \widehat{E}_k}{\sum_{k=0}^{K-1} n_k} \tag{3.10}$$

这一指标只包含批次执行能耗，不包含终端通信、设备空闲与待机以及基础设施运行能耗。训练中进一步以公共能耗参考值 E_{ref} 归一化，令 $\widetilde{E}_k = \widehat{E}_k / E_{ref}$ 、

$\widetilde{B}_E = B_E^J / E_{ref} > 0$, 其中 B_E^J 为单请求能耗预算, 单位为 J/request。单次转移与累计超额能耗成本为

$$\begin{aligned} c_k^E &= [\widetilde{E}_k - \widetilde{B}_E n_k]_+ \\ J_E^+(\pi) &= \mathbb{E}_\pi \left[\sum_{k=0}^{K-1} c_k^E \right] \end{aligned} \quad (3.11)$$

其中, $[x]_+ = \max(x, 0)$ 。超额项只惩罚本次派发能耗超过“单请求预算乘以请求数”的部分, 因此累计超额成本与式 (3.10) 的平均执行能耗是不同的量。

由于 P99 是完整请求集合上的统计量, 在线训练采用由当前队列、剩余时限和转移持续时间构造的尾部风险成本 C_k^{tail} 作为代理, 其具体计算在 3.3 节给出。相应累计尾部风险与模型级期望违约率为

$$\begin{aligned} J_{tail}(\pi) &= \mathbb{E}_\pi \left[\sum_{k=0}^{K-1} C_k^{tail} \right] \\ V_m(\pi) &= \mathbb{E}_\pi [v_m^\pi(\mathcal{R})] \end{aligned} \quad (3.12)$$

记 Π 为可接受调度策略集合, ε_m 为模型 m 的目标违约率。联合调度问题写为

$$\begin{aligned} \min_{\pi \in \Pi} & J_{tail}(\pi) + J_E^+(\pi) \\ s.t. & V_m(\pi) \leq \varepsilon_m, \forall m \in \mathcal{M} \\ & a_k \in \mathcal{A}_k^{phy}, k = 0, \dots, K-1 \end{aligned} \quad (3.13)$$

式 (3.13) 在物理可行性和模型级 SLA 目标下, 降低累计尾部风险代理与超额执行能耗, 并非直接最小化实测 P99。若外部负载使目标违约率不可达, ε_m 仅作为风险调节参考。下一节将该部分可观测、事件驱动调度过程建模为半马尔可夫决策过程, 并介绍学习策略。

3.3 HERA 算法设计

HERA 将历史执行反馈、风险门控与离散 SAC 结合，其在线调度与反馈流程如图 3.1 所示。历史编码补充近期执行信息，Gate 根据时限与队列压力筛选动作，策略网络联合选择队列、设备和批大小，或继续等待。批次完成后，执行反馈用于更新历史窗口与时延估计，供后续调度使用。

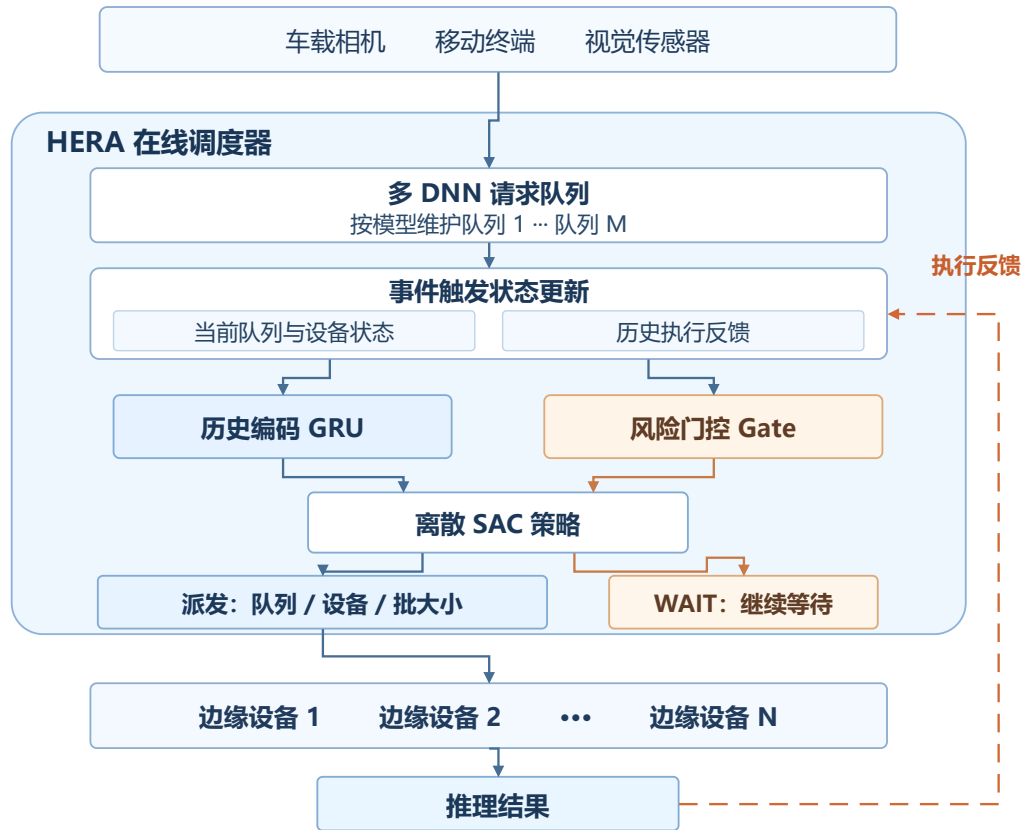


图 3.1 HERA 在线调度与闭环反馈框架

3.3.1 在线调度过程建模

调度决策由请求到达、批次完成及等待上限事件触发，相邻决策的物理时间间隔不固定。因此，本章采用半马尔可夫决策过程描述在线交互^[34]。第 k 次决策的观测 $\mathbf{s}_k$ 包括当前队列与设备运行信息；有效干扰强度等环境内部量不可直接观测，仅靠当前快照难以完整描述执行状态。记近期完成批次的有序记录为 $\mathcal{H}_k$ ，后文统一以 $\mathbf{x}_k = (\mathbf{s}_k, \mathcal{H}_k)$ 表示网络输入，不假设该有限历史能够恢复完整环境状态。

队首紧迫程度、动作局部状态与 Gate 均采用第 3.2.2 节定义的请求剩余时限。

策略从门控集合 $\mathcal{A}_k^{gate}$ 中选择动作，该集合是式 (3.2) 物理可行集合的子集。派发后立即改变队列与设备占用，WAIT 则推进到下一可观测事件。令 $\Delta t_k = t_{k+1} - t_k$ ，连续派发时该间隔可以为零。训练按实际经过的物理时间折扣回报，并以尾部风险、模型级 SLA 偏差及超额执行能耗构造奖励，避免将不同持续时间的转移视为等长时间步。

3.3.2 历史编码与策略网络

历史窗口保留最近 $H = 16$ 个已完成批次的模型类型、执行设备、批大小、执行时延和执行能耗，并按反馈顺序输入 GRU^[35]。演员网络与两个在线评论家分别配置独立的历史编码器，生成各自的历史增强表示：

$$\mathbf{h}_k^\ell = GRU_{\omega_\ell}(\mathcal{H}_k), \mathbf{c}_k^\ell = [\mathbf{s}_k, \mathbf{h}_k^\ell] \quad (3.14)$$

其中， $\ell \in \{\pi, Q_1, Q_2\}$ 标识演员或评论家， ω_ℓ 为相应编码器参数。历史编码与当前观测拼接后送入对应网络，使策略利用近期执行反馈，同时保留当前队列和设备状态。各网络采用相同的编码结构，但不共享历史编码器参数。

对派发动作，动作特征 $\varphi(a)$ 由 WAIT 标志、模型与设备的独热编码、批大小的对数及线性特征、参考服务特征组成，连续特征在编码前归一化。派发动作的 WAIT 标志为零；WAIT 动作仅该标志为一，其余特征为零。演员用两个 MLP 分别编码历史增强上下文和动作特征，由上下文分支给出基础分数，再叠加动作条件分支的修正。将最终分数记为 $z_\theta(\mathbf{x}_k, a)$ ，掩码后的策略为

$$\pi_\theta(a|\mathbf{x}_k) = \frac{\exp(z_\theta(\mathbf{x}_k, a))}{\sum_{a' \in \mathcal{A}_k^{gate}} \exp(z_\theta(\mathbf{x}_k, a'))}, a \in \mathcal{A}_k^{gate} \quad (3.15)$$

门控集合外动作的概率为零。训练阶段按该分布采样，评价与部署阶段选择概率最大的动作。两个评论家除输入各自的历史增强表示和动作特征外，还输入动作局部状态，包括对应模型的队列状态、队首剩余时限及目标设备占用与并发

状态，分别输出标量 $Q_{\psi_j}(\mathbf{x}_k, a)$ 。两个价值网络结构相同、参数独立，训练中采用较小的价值估计构造目标，与 SAC 的双评论家设计一致^[36]。

3.3.3 风险估计与 Gate 机制

Gate 所需的到达率来自边缘入口的实际入队记录。设第 ℓ 个观测窗口内，模型 m 的入队请求数为 $A_{m,\ell}^{obs}$ ，窗口时长为 w_ℓ ，则采用指数加权移动平均更新到达率：

$$\widehat{\lambda}_m^{(\ell)} = \alpha_\lambda \frac{A_{m,\ell}^{obs}}{w_\ell} + (1 - \alpha_\lambda) \widehat{\lambda}_m^{(\ell-1)} \quad (3.16)$$

其中， $0 < \alpha_\lambda \leq 1$ 为平滑系数。决策时使用最近一次窗口更新所得的 $\widehat{\lambda}_{m,k}$ ，不以终端生成强度替代边缘实际到达率。对于动作 $a = (m, d, b)$ ，以模型—设备对的在线修正因子 $\widehat{\rho}_{m,d,k}$ 校准参考批次时延：

$$\widehat{T}_k(a) = \overline{T}_{m,d,b} \widehat{\rho}_{m,d,k} \quad (3.17)$$

修正因子利用“实测时延与对应参考时延之比”的指数加权移动平均及历史波动形成保守估计。缺少反馈时采用固定保守初值，每个新完成批次仅用于更新一次在线修正因子。门控计算前，依据当前修正因子计算各候选动作的预测执行时延。

令 $\mathcal{M}_k^+$ 为当前非空队列集合， i_m 为模型 m 的队首请求， $\mathcal{A}_{k,m}^{phy}$ 为该模型的物理可行派发集合。归一化剩余时限、预测服务率及队列压力定义为

$$\begin{aligned} u_{m,k} &= \frac{s_{i_m}(t_k)}{\delta_m}, \widehat{\mu}_k(a) = \frac{b}{\widehat{T}_k(a)} \\ \widehat{\mu}_{m,k} &= \max_{a \in \mathcal{A}_{k,m}^{phy}} \widehat{\mu}_k(a) \\ p_{m,k} &= \frac{q_m(t_k) + \widehat{\lambda}_{m,k} \delta_m}{\max \{\widehat{\mu}_{m,k} \delta_m, 1\}} \end{aligned} \quad (3.18)$$

无可行派发动作时, $\widehat{\mu}_{m,k}$ 取零。 $u_{m,k}$ 越小, 队首请求越紧迫; 队列压力 $p_{m,k}$ 以当前积压请求数与一个截止期限内预计到达请求数之和为分子, 以同期最佳可行处理量与 1 中的较大值为分母, 衡量短期请求需求相对于处理能力的大小。此外, Gate 分别比较候选批次的预测完成时刻与各请求的截止期限, 以预计按时完成的请求数除以预测执行时延, 得到预测按时服务率。

门控首先处理紧迫队列。当最小归一化剩余时限不大于 0.35 时, 选择该值最小的模型 m^* , 优先保留满足下式的派发动作:

$$a \in \mathcal{A}_{k,m^*}^{phy}, \widehat{T}_k(a) \leq s_{i_{m^*}}(t_k) - 0.05\delta_{m^*} \quad (3.19)$$

该条件为队首请求预留相对截止期限 5% 的裕量。若筛选结果为空, 但该模型最佳预测按时服务率大于零, 则保留按时服务率不低于最佳值 90% 的动作; 若预测均不能按时完成, 则保留预测服务率不低于该模型最佳值 90% 的动作, 以维持积压处理能力。

若无紧迫队列且最大压力不小于 0.50, Gate 选择压力最大的模型, 保留预测服务率不低于其最佳值 80% 的动作。任一分支生效时, 仅保留所选模型的派发动作并排除 WAIT; 均未触发时保留原物理集合。若所选模型无可行派发, 或全部队列为空, 则返回原集合。门控依赖在线估计, 不构成硬性截止期限保证。

风险门控的完整筛选流程见算法 3.1。

算法 3.1 风险门控动作筛选

输入：当前观测、边缘到达率估计、候选执行时延预测及物理可行动作集合 $\mathcal{A}_k^{phy}$ 。

输出：门控后的动作集合 $\mathcal{A}_k^{gate}$ 。

1. 求非空队列集合 $\mathcal{M}_k^+$ ；若其为空，返回 $\mathcal{A}_k^{phy}$ 。
 2. 对每个 $m \in \mathcal{M}_k^+$ ，按式 (3.18) 计算紧迫程度、预测服务率和队列压力，并计算预测按时服务率；无可行派发时，两种最佳服务率均取零。
 3. 若 $\min_{m \in \mathcal{M}_k^+} u_{m,k} \leq 0.35$ ，取 $m^* = \arg \min_{m \in \mathcal{M}_k^+} u_{m,k}$ 。
 4. 构造 $\mathcal{A}^{safe} = \{a \in \mathcal{A}_{k,m^*}^{phy} : \widehat{T}_k(a) \leq s_{i_{m^*}}(t_k) - 0.05\delta_{m^*}\}$ 。
 5. 若 $\mathcal{A}^{safe} \neq \emptyset$ ，令 $\mathcal{A}_k^{gate} = \mathcal{A}^{safe}$ 。
 6. 否则若该模型最佳预测按时服务率大于零，保留 $\mathcal{A}_{k,m^*}^{phy}$ 中预测按时服务率不低于该最佳值 90% 的动作。
 7. 否则保留 $a \in \mathcal{A}_{k,m^*}^{phy}$ 中满足 $\widehat{\mu}_k(a) \geq 0.90\widehat{\mu}_{m^*,k}$ 的动作；结束内层判断。
 8. 否则若 $\max_{m \in \mathcal{M}_k^+} p_{m,k} \geq 0.50$ ，取 $m^* = \arg \max_{m \in \mathcal{M}_k^+} p_{m,k}$ ，保留 $a \in \mathcal{A}_{k,m^*}^{phy}$ 中满足 $\widehat{\mu}_k(a) \geq 0.80\widehat{\mu}_{m^*,k}$ 的动作。
 9. 否则令 $\mathcal{A}_k^{gate} = \mathcal{A}_k^{phy}$ ；结束外层判断。
 10. 若所选模型无可行派发动作，恢复 $\mathcal{A}_k^{gate} = \mathcal{A}_k^{phy}$ 。
 11. 返回 $\mathcal{A}_k^{gate}$ 。
-

3.3.4 在线决策流程与训练更新

在线执行依次更新观测和到达率估计、构建物理可行集合、预测执行时延、应用 Gate，再由演员选择动作。动作执行后，调度器记录下一观测、持续时间及奖励分量；新完成批次用于更新历史窗口与时延估计器。训练采用离散 SAC^[36, 37]，通过回放上述交互学习策略。

长期排队与资源阻塞会增加尾部时延风险^[4]。为给每次转移提供反馈，HERA 将队列压力与归一化剩余时限结合，定义瞬时风险及原始尾部成本：

$$\begin{aligned}
 R_k^{tail} &= \min \left\{ R_{\max}, \sum_{m \in \mathcal{M}_k^+} p_{m,k} [\zeta - u_{m,k}]_+^\xi \right\} \\
 C_k^{tail} &= \frac{\Delta t_k}{2\delta_{ref}} (R_k^{tail} + R_{k+1}^{tail})
 \end{aligned} \tag{3.20}$$

其中, ζ 为风险激活阈值, ξ 控制风险随时限缩短的增长程度, $R_{\max}$ 限制风险上界, δ_{ref} 为时间归一化参考, 取值为 41.61 ms。前后两个状态分别确定其非空队列, 空队列不贡献风险。将两端风险均值乘以转移时长, 可惩罚风险持续积累; 零时长转移的原始尾部成本为零。该量是训练代理, P99 则由实际端到端时延统计得到。

为将持续阻塞的影响关联到先前派发, 原始尾部成本的一半保留在当前转移, 另一半平均分给该转移期间处于执行状态的各个不同批次, 并追溯至各自派发转移; 没有活跃批次时, 全部成本留在当前转移。回合结束后统一回写, 形成用于后续回放的 $\tilde{C}_k^{tail}$ 。

本章基于式 (3.13) 构造受拉格朗日惩罚思想启发的训练代理, 以请求计数刻画 SLA 偏差, 并结合尾部成本追溯分配、物理时间折扣和最大熵策略更新。该训练目标与原约束问题不严格等价, 不提供约束可行性或全局最优保证。第 e 个回合的训练奖励为

$$r_k = -\kappa \left[\tilde{C}_k^{tail} + \sum_{m \in \mathcal{M}} \lambda_{m,e}^{SLA} (v_{m,k} - \varepsilon_m n_{m,k}) + c_k^E \right] \tag{3.21}$$

其中, κ 为奖励缩放系数, $v_{m,k}$ 统计本次新检测到的模型违约请求数, $n_{m,k}$ 为本次派发请求数, c_k^E 沿用式 (3.11)。能耗在派发时计入, 违约在检测到超时时记录, 因此二者可发生于不同转移。各模型 SLA 权重在回合内固定, 在回合结束后按观测违约率 $V_{m,e}$ 调整:

$$\lambda_{m,e+1}^{SLA} \leftarrow \min \left\{ \lambda_{\max}, \max \left[\lambda_{\min}, \lambda_{m,e}^{SLA} + \eta_\lambda (V_{m,e} - \varepsilon_m) \right] \right\} \tag{3.22}$$

其中, η_λ 为更新步长, $\lambda_{\min}$ 与 $\lambda_{\max}$ 限定权重范围。违约率高于目标时提高相应惩罚权重, 低于目标时降低权重。该更新用于调节不同模型的服务风险。

经验聚合在累计时长达到 W 、决策数达到 $N_{\max}$ 或回合终止时停止。设从第 k 次转移起聚合 N_k 步, 累计时长为 $\Delta T_k^{(N_k)} = \sum_{i=0}^{N_k-1} \Delta t_{k+i}$, 则多步回报及终点折扣为

$$\begin{aligned} R_k^{(N_k)} &= \sum_{i=0}^{N_k-1} \exp\left(-\beta \sum_{h=0}^{i-1} \Delta t_{k+h}\right) r_{k+i} \\ \bar{\gamma}_k^{(N_k)} &= \exp\left(-\beta \Delta T_k^{(N_k)}\right) (1 - d_{k+N_k-1}) \end{aligned} \quad (3.23)$$

其中, β 为连续时间折扣率, d_{k+N_k-1} 为终止标志。内层空和取零; 同一物理时刻的连续派发保留在经验序列中, 但不会额外产生时间折扣。记聚合终点为 $k' = k + N_k$, 由目标评论家构造软状态价值与回归目标:

$$\begin{aligned} \bar{V}_{k'} &= \sum_{a' \in \mathcal{A}_{k'}^{\text{gate}}} \pi_\theta(a' | \mathbf{x}_{k'}) \left[\min_{j \in \{1,2\}} Q_{\bar{\psi}_j}(\mathbf{x}_{k'}, a') - \alpha \ln \pi_\theta(a' | \mathbf{x}_{k'}) \right] \\ y_k &= R_k^{(N_k)} + \bar{\gamma}_k^{(N_k)} \bar{V}_{k'} \end{aligned} \quad (3.24)$$

目标网络参数记为 $\bar{\psi}_j$, α 为熵温度。评论家对回放样本最小化预测值 $Q_{\psi_j}(\mathbf{x}_k, a_k)$ 与 y_k 的均方误差, 并在两个评论家间取平均, 目标值不参与反向传播。终止转移的终点折扣为零, 不再引入后续价值。演员的损失函数为

$$\mathcal{L}_\pi(\mathbf{x}_k) = \sum_{a \in \mathcal{A}_k^{\text{gate}}} \pi_\theta(a | \mathbf{x}_k) \left[\alpha \ln \pi_\theta(a | \mathbf{x}_k) - \min_{j \in \{1,2\}} Q_{\psi_j}(\mathbf{x}_k, a) \right] \quad (3.25)$$

训练更新中, 评论家小批量在不同场景间均衡采样, 并按队列压力与尾部风险分层; 演员使用独立的场景均衡小批量。回放保留原始历史窗口, 由当前网络重新编码, 并以当前 SLA 权重重构奖励和多步目标。每次更新还调整熵温度, 并用 $\bar{\psi}_j \leftarrow (1 - \tau) \bar{\psi}_j + \tau \psi_j$ 软更新目标评论家及其历史编码器, 其中 $0 < \tau < 1$ 。回合结束时补存剩余转移、回写追溯成本并更新 SLA 权重。部署仅保留演员及其历史编码器, 结合在线估计与 Gate 执行决策。

HERA 的完整训练流程见算法 3.2。

算法 3.2 HERA 训练流程

输入：物理可行动作集合 $\mathcal{A}_k^{phy}$ 、算法 3.1 的门控规则、归一化能耗预算 $\widetilde{B}_E$ 、各模型 SLA 目标 $\{\varepsilon_m\}_{m \in \mathcal{M}}$ 、聚合时窗 W 、最大聚合步数 $N_{\max}$ 和连续时间折扣率 β 。

输出：训练后的策略 π_θ 及演员历史编码器。

1. 初始化演员、双评论家及目标评论家、各自独立的历史编码器、熵温度 α 、经验回放池 $\mathcal{B}_{rep}$ 和 SLA 权重。
 2. 对每个训练回合 e ，执行步骤 3~17。
 3. 重置环境、历史窗口 $\mathcal{H}_k$ 、在线估计器、N 步缓存及尾部成本记录。
 4. 回合未终止时，重复步骤 5~15。
 5. 获取观测 $\mathbf{s}_k$ ，更新边缘到达率估计及各网络的历史表示。
 6. 构建 $\mathcal{A}_k^{phy}$ ，预测候选执行时延，通过算法 3.1 得到 $\mathcal{A}_k^{gate}$ 。
 7. 在门控集合内采样 $a_k \sim \pi_\theta(\cdot | \mathbf{x}_k)$ 并执行，记录 $\mathbf{s}_{k+1}$ 、 Δt_k 、终止标志 d_k 及奖励分量。
 8. 仅利用新完成批次更新历史窗口与时延估计器。
 9. 记录尾部成本及其追溯分配，将原始转移加入 N 步缓存，保留零时长转移。
 10. 若累计时长达到 W 、步数达到 $N_{\max}$ 或 $d_k = 1$ ，将聚合经验存入 $\mathcal{B}_{rep}$ 。
 11. 达到训练更新时机时，执行步骤 12~15。
 12. 从 $\mathcal{B}_{rep}$ 按场景均衡、队列压力与尾部风险分层采样评论家小批量 $\mathcal{D}_Q$ 。
 13. 用当前 SLA 权重按式 (3.21) 重构奖励，按式 (3.23) 聚合多步回报，并以式 (3.24) 的 y_k 为目标更新双评论家。
 14. 从 $\mathcal{B}_{rep}$ 独立采样场景均衡的演员小批量 $\mathcal{D}_\pi$ 。
 15. 最小化式 (3.25)，更新熵温度 α ，软更新目标评论家及其历史编码器。
 16. 回合结束后，存入剩余 N 步转移，并将追溯尾部成本回写至相应派发记录。
 17. 按式 (3.22) 更新 SLA 权重。
 18. 返回策略 π_θ 及演员历史编码器。
-

3.3.5 复杂度分析

以下按显式枚举候选动作、逐请求检查预测截止期限的实现方式分析运算量。设完整动作数 $A = 1 + \sum_{m \in \mathcal{M}} \sum_{d \in \mathcal{D}} |\mathcal{B}_{m,d}|$ ，最大批大小为 $b_{\max}$ ，历史特征维度与 GRU 隐藏维度分别为 f 和 h 。对长度为 H 的窗口进行编码，运算量为 $O(Hh(f+h))$ 。

给定队列摘要与设备状态后，门控遍历候选动作及其批内请求，开销上界为 $O(Ab_{\max})$ 。

令 C_{ctx} 为上下文 MLP 的运算量， C_a 为单动作特征编码与评分的运算量，将掩码和概率归一化的线性开销并入动作项，则单次在线决策的时间复杂度上界为

$$T_{online} = O(Hh(f + h) + C_{ctx} + A(C_a + b_{\max})) \quad (3.26)$$

其余维度与单动作评分开销固定时，上界关于动作数为线性，历史缓存占用为 $O(Hf)$ 。训练另需双评论家、目标网络的前向计算、在线网络的反向更新和多步奖励重构。设小批量大小为 B ，单样本网络更新开销为 F_{train} ，则单次更新的复杂度为 $O(B(F_{train} + N_{\max}))$ 。实际耗时还取决于网络配置、实现与硬件条件。

3.4 HERA 实验结果与分析

本节采用实测性能数据驱动的仿真环境，评估 HERA 在异构设备上的多 DNN 推理调度性能。实验先比较静态与动态负载下的总体表现，再分析各模型的服务差异，通过调度行为与消融实验考察历史编码和风险门控的作用，最后测试截止期限差异及请求构成变化的影响。

3.4.1 实验环境及参数设置

实验以 RTX 3060 GPU 和 Apple M3 为异构执行设备，利用实测时延、能耗数据与参数化并发干扰模型构建仿真环境，两类设备的并发上限分别为 2 个和 1 个批次。推理任务选用 MobileNetV3-Small^[41]、EfficientNet-B0^[42] 和 ResNet50^[43]，均采用 ImageNet^[44] 预训练权重，输入尺寸为 $3 \times 224 \times 224$ 。三个模型均预部署于两类设备，不计模型加载、迁移和切换开销。实验考察请求调度，不涉及模型压缩或推理精度调整。

主实验中，三类请求各占三分之一，终端按泊松过程生成请求。负载水平 ρ 为平均请求生成速率与参考服务速率之比，后者依据模型请求比例及各模型在不同设备上的峰值吞吐量计算。因此， ρ 表示基于性能测量的相对负载，实际资源压

力还会受到并发干扰影响。每个请求携带 50~100 KB 压缩图像，上行带宽为 20 MHz，等效频谱效率在 2~4 bit/s/Hz 内均匀采样，对应约 5~10 MB/s 的传输速率和 5~20 ms 的上传时延；边缘侧解码并预处理图像，分类结果较小，其回传时延忽略不计。

截止期限在正式测试前独立标定。在 $\rho = 0.5$ 下，采用 5 个环境随机种子，各运行 10 s；参考调度器按设备吞吐量比例分配请求，最大批大小为 32，最大等待时间为 20 ms。将三类模型共 32 885 个已完成请求合并，得到统一的端到端时延分布，取其 P80、P90 和 P95，分别设置 MobileNetV3-Small、EfficientNet-B0 和 ResNet50 的截止期限为 35.72、41.61 和 47.77 ms。该过程不针对各对比算法分别标定，所得期限在主实验的不同算法与负载条件下保持不变。

HERA 从随机初始化开始，在轻载、重载及负载突变条件下训练，测试时冻结网络参数。演员与评论家、GRU 历史编码器、动作编码器的隐藏层维度分别为 192、64 和 96；训练共 400 回合，经验回放容量为 90 000，小批量大小为 256。其余参数列于表 3.2。其中，目标 SLA 违约比例设为 0.02（2%），用于训练中的风险调节，不表示所有测试负载下均能满足该目标。

设备性能测量覆盖批大小 1、2、4、8、16 和 32，使用 PyTorch eager 模式与 FP32 精度，RTX 3060 和 Apple M3 分别采用 CUDA 与 MPS 后端。每种“模型—设备—批大小”组合先预热 50 次，再完成不少于 500 次同步时延测量，测量时长在两类设备上分别不少于 8 s 和 15 s。能耗按 5 个测量块汇总，使用 nvidia-smi 与 powermetrics 每 100 ms 采样一次功率，并扣除空闲功率中位数，测量结果见表 3.3。两平台的功率采集范围不同，统一除以能耗参考值 E_{ref} 仅调整数值尺度。跨设备与策略间的能耗比较均限于所定义的净执行能耗代理，不代表统一测量边界下的整机绝对能耗或节能幅度。

并发减速因子依据 RTX 3060 的共置测量设置，在批次派发或完成时，根据模型组合、批大小及资源竞争情况更新。主实验的训练和测试采用相同干扰参数：全局强度为 1.0，跨模型系数为 1.15，模型对变异系数为 0.25，每回合隐藏缩放因子在 [0.75, 1.25] 内均匀采样，最大减速倍数为 3.0。仿真中的执行能耗由实测性能数据与干扰模型共同估计。

表 3.2 HERA 训练与调度参数

参数	取值
演员/评论家隐藏层维度	192
GRU / 动作编码器隐藏层维度	64 / 96
训练回合数 / 回放容量 / 小批量大小	400 / 90 000 / 256
历史窗口长度	16
演员与评论家学习率	3×10^{-4}
连续时间折扣率	0.01 s^{-1}
到达率 EWMA 平滑系数	0.2
尾部风险激活阈值 / 风险指数	0.5 / 2.0
时间归一化参考 δ_{ref}	41.61 ms
N 步时间窗口 / 最大决策步数	0.05 s / 64
目标 SLA 违约比例	0.02
SLA 权重初值 / 取值范围	5.0 / [1, 30]

表 3.3 不同设备上的 DNN 实测性能

设备与指标	MobileNetV3- Small	EfficientNet-B0	ResNet50
RTX 3060: 批次时延 (ms)	4.13~5.89	6.16~27.23	4.93~59.83
净执行能耗 (J/请求)	0.0136~0.0357	0.0600~0.1242	0.1439~0.3505
峰值吞吐量 (请求/s)	5429 (32)	1175 (32)	535 (32)
Apple M3: 批次时延 (ms)	8.37~17.76	12.43~108.34	10.34~220.13
净执行能耗 (J/请求)	0.0055~0.0430	0.0276~0.0783	0.0703~0.1202
峰值吞吐量 (请求/s)	1802 (32)	343 (16)	145 (32)

注：时延与净执行能耗范围对应不同批大小；峰值吞吐量后的括号给出对应批大小。

评价指标为端到端 SLA 违约率、P99 时延和单请求执行能耗。P99 先在每条 60 s 轨迹内计算，再对 5 个配对环境随机种子的结果取平均，不将不同种子的请求混合后计算分位数。能耗归一化参考值为 0.078 J/请求，即批大小为 1 时各模型一设备组合的单请求执行能耗中位数；归一化预算为 1.4，对应约 0.11 J/请求的执行能耗阈值。该阈值用于训练与评价尺度对齐，不是整机功率上限。

3.4.2 对比算法

在相同仿真环境中，将 HERA 与一种学习方法和五种启发式方法进行比较。各算法采用一致的任务、设备与物理动作约束，启发式参数在独立验证轨迹上选择后冻结。配对随机种子下的初始状态相同，但队列与设备状态随各算法的调度决策独立演化。

(1) Coinf^[19]：结合时延预测、紧迫性判断与优势演员—评论家 (A2C) 进行推理调度。根据初步批大小及历史资源负载日志的 P98 统计区分任务紧迫程度，由 A2C 综合模型队列、运行任务和资源状态选择模型与批大小。实验将其适配为多设备调度，并采用与 HERA 相同的可观测信息范围。

(2) 固定超时批处理 (FTS)：借鉴 TensorFlow Serving^[45] 的固定超时机制，在达到目标批大小或触发超时刷新条件时派发批次。

(3) 自适应等待调度 (AQS)：借鉴 Triton Dynamic Batchers^[46] 的优选批大小和有界排队机制，依据队列压力与请求剩余时限调整组批等待。

(4) 并发批次调度 (CPS)：借鉴 Ebird^[47] 的并发多批次执行机制，在多个执行槽位中维护运行批次，并根据预测违约情况和服务能力排序候选动作。

(5) 最早截止时间优先 (EDF)：比较各模型队首请求的绝对截止时间，优先服务截止时间最早的队列。

(6) 先来先服务 (FCFS)：依据全局请求到达顺序选择队列，当设备可用时立即派发物理约束允许的最大批次。

其中，FTS、AQS 和 CPS 是针对本章异构设备、模型队列及动作约束实现的机制级适配，并非对相应推理服务系统的完整复现。后续比较反映这些调度机制在统一实验环境中的表现。

3.4.3 静态负载下的性能分析

静态实验设置 8 个负载水平：0.40、0.50、0.60、0.70、0.75、0.80、0.85 和 0.90。每条轨迹内负载水平固定，请求按相应速率的泊松过程到达，实验结果如图 3.2 所示。其中，P99 时延采用对数坐标，以展示高负载下不同方法的尾部差异。

对 8 个负载水平等权平均后，HERA 的 SLA 违约率为 33.75%，FCFS 为 39.36%，相对降低 14.24%；HERA 的平均 P99 时延为 92.68 ms，较 FCFS 的 126.55 ms 降低 26.77%。从负载变化趋势看，中高负载下两项指标的改善更明显。

负载增加时，排队、组批等待及并发干扰共同消耗请求的剩余时限。在 $\rho = 0.9$ 时，各算法违约率均明显上升，HERA 也达到 85.92%，但其 P99 时延仍为该负载下所比较方法中的最低值。由于负载按设备峰值吞吐量归一化，而截止期限在 $\rho = 0.5$ 标定后保持固定，高负载下的实际服务能力不足以保证大多数请求按时完成。上述结果表明，HERA 能缓解尾部时延增长，但不能消除资源不足造成的违约。

HERA 在 8 个负载水平上的平均执行能耗低于设定阈值，较能耗最低的 Coinf 高 1.82%。因此，静态实验中的性能收益主要体现为违约率与尾部时延下降。

3.4.4 动态负载下的性能分析

动态实验采用 Random-switch 场景：每隔 0.25 s 在上述 8 个负载水平间随机切换，每条 60 s 轨迹的时间平均负载为 0.75，各时间段内仍按对应速率生成泊松请求。切换后，上一时间段积累的队列和未完成批次继续影响当前调度，因而需要同时处理新到达请求与已有执行状态。

如图 3.3 所示，HERA 的总体 SLA 违约率为 31.32%，较 FCFS 的 34.33% 降低 8.79%；P99 时延由 FCFS 的 119.99 ms 降至 95.84 ms，降低 20.12%。HERA 利用在线到达率估计及近期批次执行反馈作出决策，在该负载切换场景下取得了最低的总体违约率与 P99 时延。HERA 的单请求执行能耗低于设定阈值，但比 Coinf 高 1.58%。

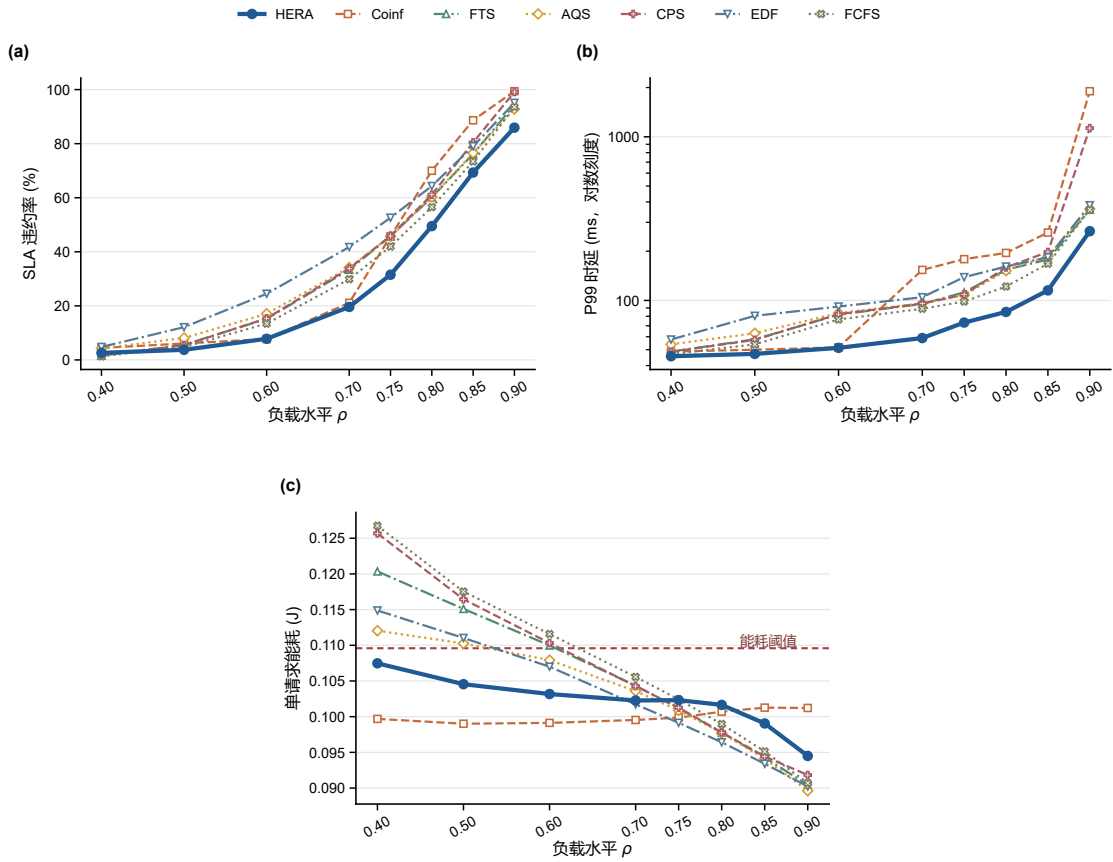


图 3.2 静态负载下的总体性能：（a）SLA 违约率；（b）P99 时延；（c）单请求执行能耗

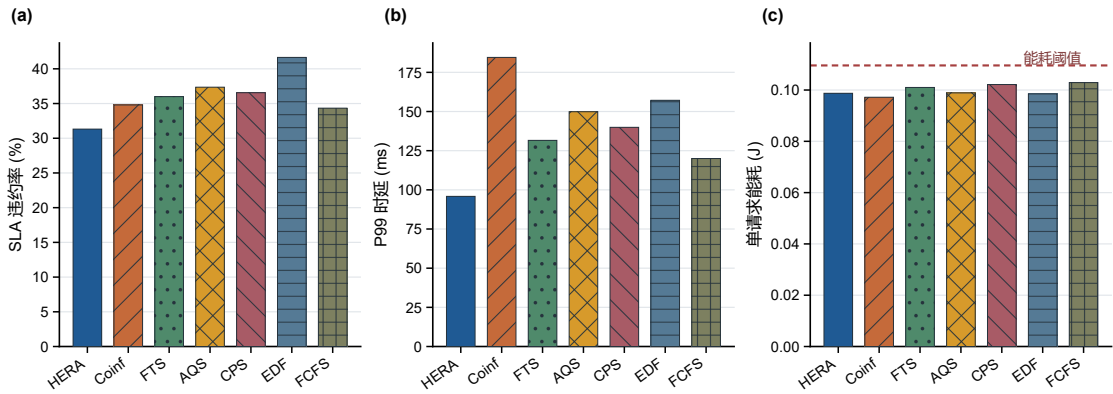


图 3.3 动态负载下的总体性能：（a）SLA 违约率；（b）P99 时延；（c）单请求执行能耗

3.4.5 不同 DNN 模型的性能分析

总体指标可能掩盖不同模型之间的服务差异，因此分别统计三类 DNN 请求的违约率和 P99 时延，如图 3.4 所示。HERA 在静态与 Random-switch 场景中，对每类模型均取得了最低的 P99 时延。对于计算需求较高的 ResNet50，静态违约率由 FCFS 的 49.74% 降至 36.67%，动态违约率由 46.41% 降至 35.13%，分别相对降低 26.27% 和 24.32%。

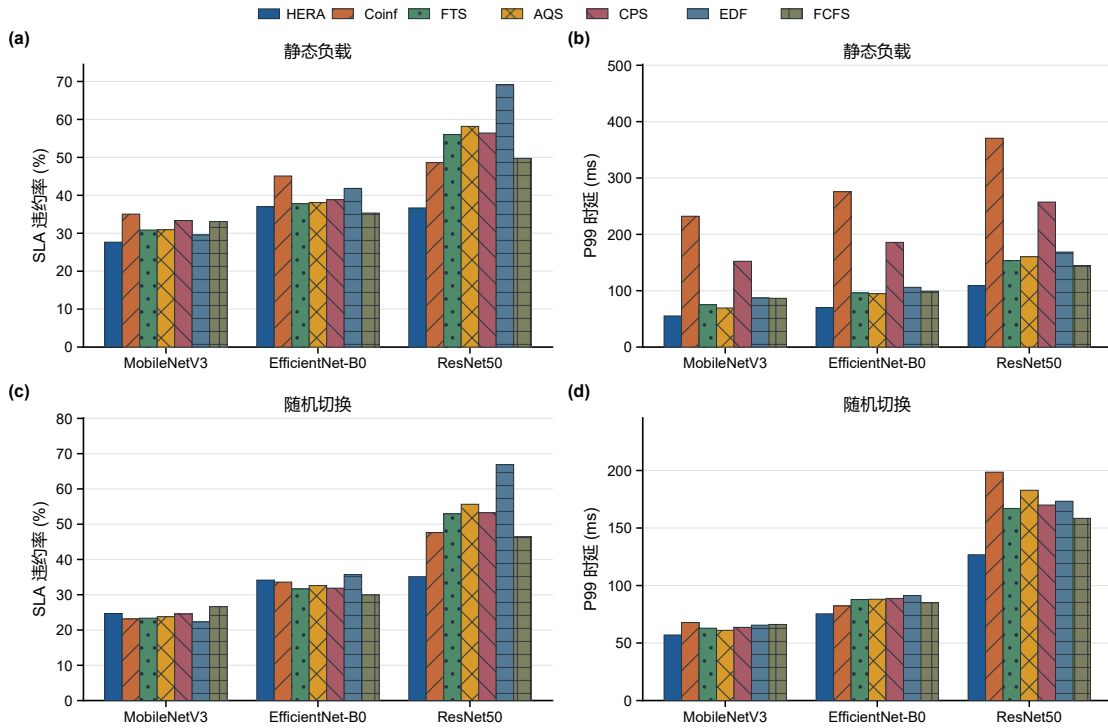


图 3.4 不同 DNN 模型的服务性能：(a) (b) 静态违约率与 P99 时延；(c) (d) 动态违约率与 P99 时延。MobileNetV3 指 MobileNetV3-Small

各模型的违约率改善并不一致。MobileNetV3-Small 在静态场景下取得最低违约率，但其动态违约率为 24.68%，高于 EDF 的 22.29%。EfficientNet-B0 在静态与动态场景下的违约率分别为 36.99% 和 34.14%，也高于 FCFS 的 35.33% 和 29.94%。尾部时延与截止期限违约率衡量的侧重点不同：前者反映时延分布尾端，后者统计越过指定期限的请求比例。HERA 在有限资源下改善了总体表现和各模型的尾部时延，但各模型的违约率并未同时达到最低。

3.4.6 历史编码与风险门控的机制及消融分析

首先考察风险门控随队列压力变化的响应。在一条 60 s Random-switch 轨迹上，以 0.75 s 为统计窗口记录总队列长度和 Gate 激活率。一次决策中，只要 Gate 移除至少一个物理可行动作，就记为一次激活；窗口激活率为该类决策占窗口内全部决策的比例。

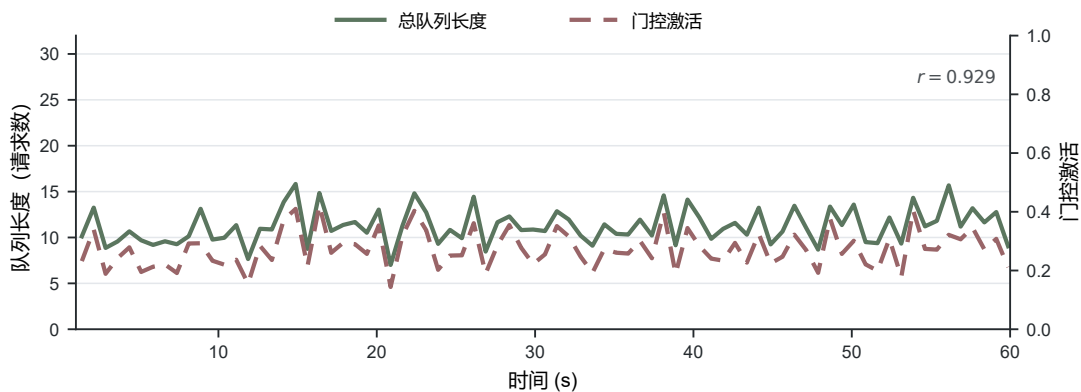


图 3.5 动态负载下总队列长度与风险门控激活率的变化

图 3.5 中，总队列长度与 Gate 激活率的 Pearson 相关系数为 0.929，反映门控在该轨迹中随积压变化的响应。为比较两个模块的作用，分别移除历史编码、风险门控及二者，并保持其余模块和实验设置不变。

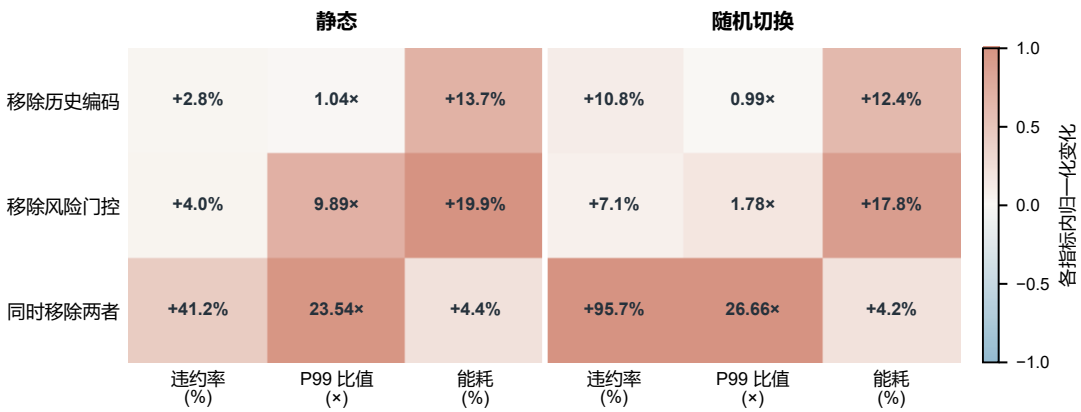


图 3.6 历史编码与风险门控的消融结果

注：违约率与能耗数值为相对完整 HERA 的变化百分比，P99 为相对倍数；颜色按各指标内部归一化，不能直接跨指标比较颜色深浅。

消融结果见图 3.6。移除 History-GRU 后，静态与动态违约率分别相对增加 2.8% 和 10.8%，执行能耗分别增加 13.7% 和 12.4%，而 P99 时延分别为完整方法的 1.04 倍和 0.99 倍，变化较小。在所评估条件下，历史编码主要改善动态负载下的违约率和执行能耗，对 P99 时延的影响较小。

移除风险门控后，演员直接从完整物理可行动作集合中选择动作，静态与动态 P99 时延分别增至完整方法的 9.89 倍和 1.78 倍，违约率和执行能耗也随之上升。同时移除两个模块时，P99 时延分别增至 23.54 倍和 26.66 倍，违约率分别相对增加 41.2% 和 95.7%。与单独移除模块相比，联合移除造成了更大的尾部退化，表明历史信息与风险筛选在该调度策略中具有互补作用，其中风险门控对尾部时延控制的影响更直接。

3.4.7 截止期限差异与请求构成变化分析

(1) 截止期限差异

为考察不同模型时限要求的影响，在保持平均截止期限不变的条件下调整模型间差异。记模型 m 的基准截止期限为 δ_m^0 ，三类模型的平均截止期限为 $\bar{\delta}$ ，用差异系数 γ 调整各模型相对均值的偏离程度：

$$\delta_m(\gamma) = \bar{\delta} + \gamma(\delta_m^0 - \bar{\delta}) \quad (3.27)$$

实验分别取 $\gamma \in \{0, 0.5, 1.0, 1.5\}$ ，采用 60 s Random-switch 轨迹。 $\gamma = 0$ 时三类模型使用相同截止期限， $\gamma = 1$ 恢复主实验设置， $\gamma = 1.5$ 则扩大模型间差异；各设置的平均截止期限均约为 41.70 ms。

如图 3.7 所示，当差异系数由 0 增至 1.5 时，HERA 的违约率由 30.90% 升至 33.54%，P99 时延由 79.06 ms 增至 103.10 ms。在四种设置中，HERA 的平均违约率与 P99 时延均为最低值；相对各设置、各指标对应的最优基线，二者分别降低 5.4%~12.8% 和 1.9%~21.6%。其中，违约率的最优基线依次为 FTS、Coinf、FCFS 和 Coinf，P99 时延的最优基线依次为 EDF、FCFS、FCFS 和 FCFS。当差异

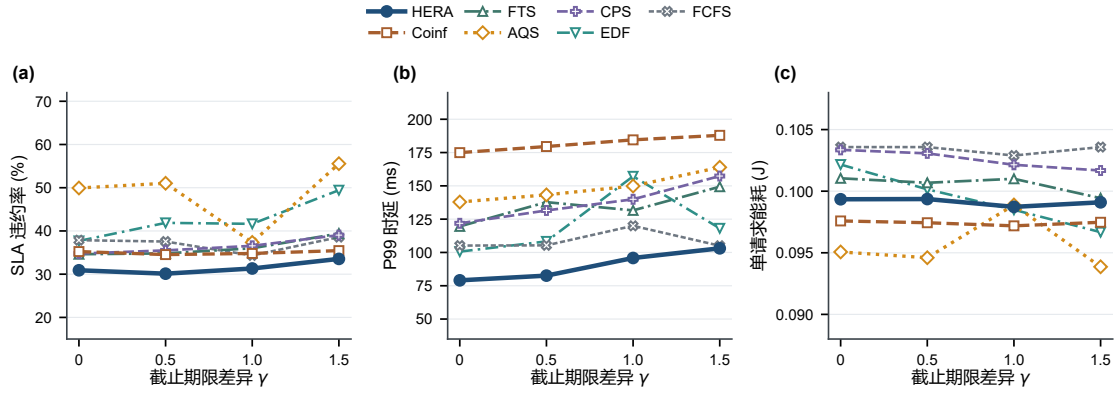


图 3.7 截止期限差异的影响：(a) SLA 违约率；(b) P99 时延；(c) 单请求执行能耗

系数为 1.5 时，HERA 相对 FCFS 的 P99 时延降低 1.9%，两者在该配置下的结果较为接近。各设置下 HERA 的执行能耗约为 0.099 J/请求。

(2) 请求构成变化

在 Random-switch 负载切换机制下，进一步改变 MobileNetV3-Small、EfficientNet-B0 和 ResNet50 的请求比例。在轻量模型请求占比较高的场景（Lightweight-heavy）中，请求比例为 60%:30%:10%；在计算密集型模型请求占比较高的场景（Compute-heavy）中，请求比例为 10%:30%:60%；构成漂移（DNN-mix drift）场景则在 60 s 内由前一比例逐渐变化为后一比例。实验结果如图 3.8 所示。

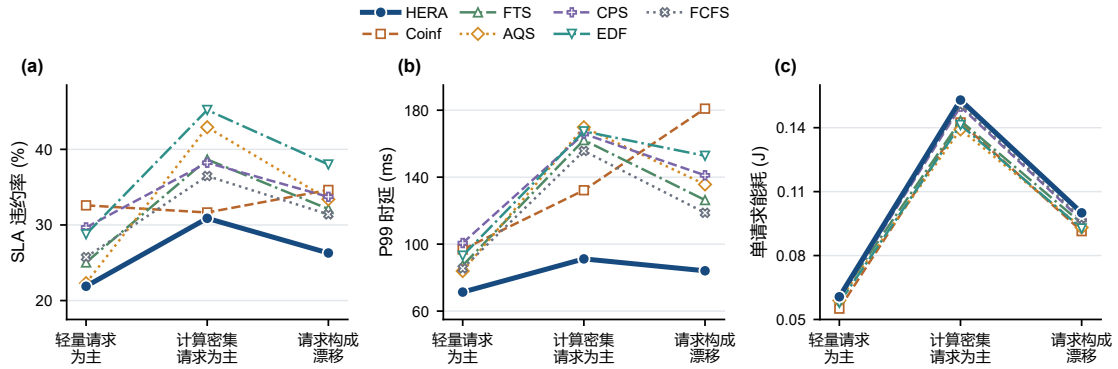


图 3.8 请求构成变化的影响：(a) SLA 违约率；(b) P99 时延；(c) 单请求执行能耗

轻量模型请求占比较高时，HERA 的违约率和 P99 时延分别为 21.89% 和 71.35 ms，AQS 分别为 22.30% 和 83.94 ms，P99 时延降低 15.0%。计算密集型模型请求占比较高时，HERA 的两项指标分别为 30.88% 和 91.18 ms，Coinf 分别为

31.65% 和 132.18 ms, P99 时延降低 31.0%。在构成漂移场景中, HERA 的违约率为 26.29%, P99 时延为 84.08 ms, 较 FCFS 的 31.40% 和 118.70 ms 分别降低 16.3% 和 29.2%。

轻量模型执行时间较短, 调度方法之间的差异相对有限; 计算密集请求增多时, 更长的执行过程可能放大排队与组批等待的影响, HERA 在计算密集型模型请求占比较高的场景及构成漂移场景中表现出更大的 P99 改善。同时, 计算密集型模型请求占比较高时, 各方法的执行能耗均超过阈值, HERA 的时延收益也伴随较高的执行能耗, 约高于最低能耗基线 10%。上述结果表明, 请求构成同时影响调度时延与执行能耗。

3.5 本章小结

本章面向模型已预部署、请求相互独立的异构边缘推理场景, 提出历史增强与风险感知调度方法 HERA。该方法将近期已完成批次的反馈与当前观测结合, 通过风险门控与调度策略协调服务队列、执行设备、批大小及派发时机, 综合考虑截止期限违约、尾部时延风险与执行能耗。

基于实测硬件性能标定的仿真实验表明, 在八种静态负载的等权汇总结果中, HERA 相对 FCFS 的总体违约率和 P99 时延分别降低 14.24% 和 26.77%; 在 Random-switch 动态负载下, 两项指标分别降低 8.79% 和 20.12%。按上述执行能耗代理计算, 两类主实验中的平均单请求执行能耗均低于设定阈值。消融结果显示, 历史编码有助于降低动态负载下的违约率与执行能耗, 风险门控对抑制尾部时延增长的作用更直接。高负载下仍存在较高违约率, 说明该方法的时延改善受可用服务能力约束。

本章以模型可直接执行为前提, 未涉及服务缓存配置与任务间依赖。下一章转向服务缓存受限的 DAG 应用, 研究服务布局与任务执行位置的协调, 以应用完成时间评价调度效果。

第四章 面向异构缓存约束的服务协调与 DAG 任务卸载方法

4.1 DAG 任务卸载问题引入

第三章在模型已预部署、请求相互独立的条件下，围绕队列、设备与批处理决策降低在线推理时延。本章研究具有服务需求和前后依赖关系的 DAG 应用，重点研究异构缓存约束下的服务协调与任务卸载。任务的执行位置不仅决定计算耗时，还影响所需服务的获取和前驱数据的传输，因此需要从应用整体完成时间考察调度效果。

在多边缘服务器环境中，入口服务器接收 DAG 应用，并为当前可分配任务选择执行位置。服务器的计算能力与缓存容量存在差异，有限缓存无法在每台服务器上保存全部服务。若目标服务器未缓存所需服务，任务需从其他边缘服务器或云端加载；若前驱与后继分配到不同服务器，还会产生依赖数据传输开销^[5]。因此，算力较强的服务器未必对应更短的应用完成时间。

服务缓存和任务卸载相互影响：缓存配置决定可直接利用的服务副本及远程加载代价，卸载决策改变各服务器的服务需求。仅按本地需求独立更新缓存^[33]，可能在有限槽位中保留较多重复副本，难以兼顾全网服务覆盖；逐任务同时调整全网缓存，则需反复处理较大的组合决策空间。卸载策略还需结合当前任务需求、前驱位置与服务器特征，比较不同执行位置的适配程度。

针对上述问题，本章提出双时间尺度服务缓存协调与任务卸载方法 TCOP。在慢时间尺度，根据历史需求与执行反馈协调跨服务器服务缓存；在快时间尺度，将当前任务与候选服务器特征配对，通过共享评分结构选择执行位置，以降低平均应用完成时间。后续依次介绍系统模型与问题定义、算法设计及实验结果分析。

4.2 系统模型与问题建模

本节依次描述异构边缘系统、DAG 任务及其通信与执行开销，并给出服务缓存与任务卸载的联合优化问题。本章以 k 表示缓存周期编号，以 t 表示任务分配步，两者均不同于任务在统一时间轴上的完成时刻。本章主要符号见表 4.1。

表 4.1 本章主要符号说明

符号	含义	单位或取值
M, S, Q	应用实例数、边缘服务器数与服务种类数	正整数
k, t	缓存周期编号与任务分配步	非负整数
G_m, V_m, E_m	应用 DAG、任务集合与依赖边集合	-
$V_m^{out}, \Omega_{m,t}$	出口任务集合与当前已分配任务集合	-
$A_m, F_{m,i}$	应用到达与任务完成的统一时间轴时刻	s
T_m	相对应用到达时刻的应用完成时间	s
$c_{m,i}, W_s$	任务计算量与服务器外生后台工作量	CPU 周期
f_s	服务器 CPU 频率	CPU 周期/s
$d_{m,i}^{in}, d_{m,j,i}$	初始输入数据量与前驱依赖数据量	bit
$q_{m,i}$	任务所需服务编号	$\{1, \dots, Q\}$
K_s, B	服务器缓存容量与系统总缓存槽位数	槽位
L_q	服务镜像大小	bit
$\ell_{s',s}, \ell_s^c$	服务器间及云端的单位数据传输时间	s/bit
R_{m,g_m}^u	用户至入口服务器的上行速率	bit/s
$g_m, s_{m,i}$	入口服务器与任务执行服务器编号	$\{1, \dots, S\}$
$X^{(k)}$	缓存周期内的服务缓存配置矩阵	$\{0, 1\}^{S \times Q}$
$x_{s,q}^{(k)}, a_{m,i,s}$	服务缓存变量与任务分配变量	$\{0, 1\}$
$R_{m,i,s}^{pre}$	前驱数据到齐时刻	s
$\tau_{m,i,s}^{usr}, \tau_{m,i,s}^{svc}$	初始输入传输与服务加载时延	s
$\tau_s^{wait}, \tau_{m,i,s}^{cmp}$	后台等待与任务计算时延	s
$\Gamma_{k-1}, z_{m,i}$	周期开始前的历史信息与当前卸载状态	-
π_C, π_O	服务缓存策略与逐任务卸载策略	-
$\lambda_{s,q}^{(k)}, \omega_{s,k}$	历史服务需求权重与服务器执行质量权重	无量纲
$J_k(X), C_{upd}$	历史加权加载节省与配置更新代价	s
$h_s^{(k)}, H_{\min}$	配置年龄与最小保持周期	缓存周期数

4.2.1 系统模型

系统由移动用户、互联边缘服务器与远端云组成，如图 4.1 所示。用户将 DAG 应用提交至最近的边缘服务器，该服务器作为入口，为应用中的各任务选择执行位置。任务可在入口服务器或其他可达边缘服务器上执行，服务器之间传输任务数据及服务镜像；云端只提供服务镜像，不参与任务执行^[5, 33]。

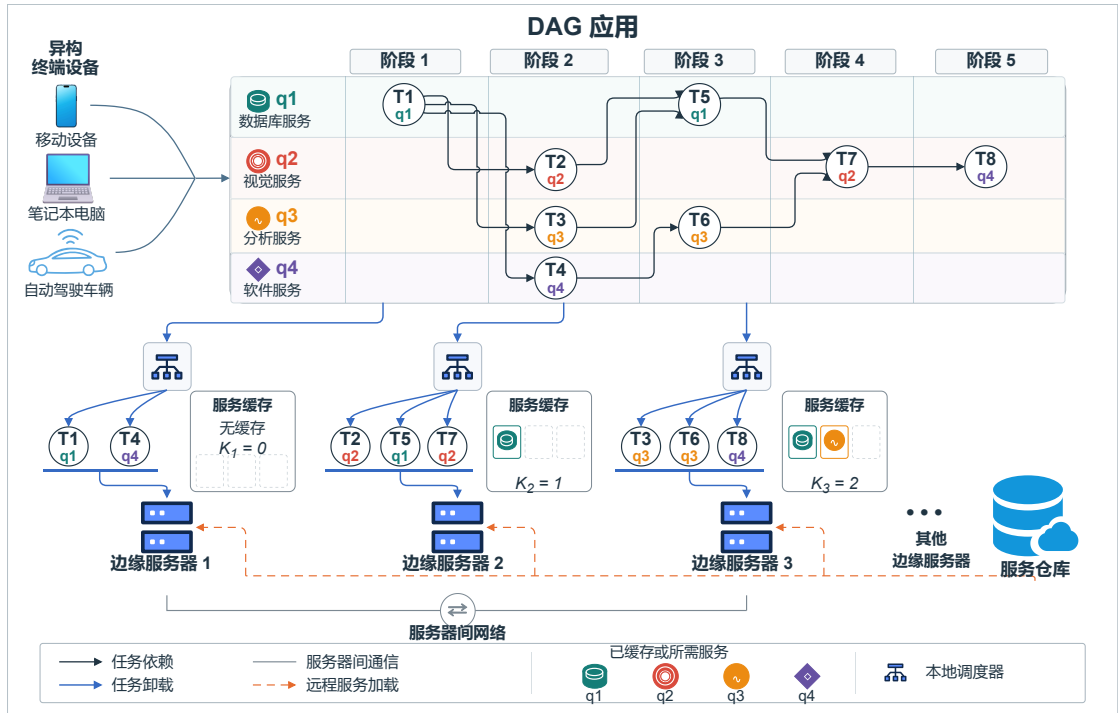


图 4.1 DAG 任务卸载与服务缓存系统示意图

图中以八个任务和三台服务器为例，任务节点标注所需服务，服务器内列出分配任务与缓存副本。未命中服务可经边缘互联链路或从云端按需加载，零缓存服务器仍可执行任务。

考虑 M 个应用实例、 S 台边缘服务器和 Q 种服务，分别以 $m \in \{1, \dots, M\}$ 、 $s \in \{1, \dots, S\}$ 和 $q \in \{1, \dots, Q\}$ 编号。服务器 s 的 CPU 频率为 f_s ，缓存容量为 K_s 。服务器间的计算能力与缓存容量可以不同。每种服务占用一个缓存槽位，记容量向量为 $K = (K_1, \dots, K_S)$ 、系统总缓存槽位数为 B ，则有

$$K_s \in \{0, 1, \dots, Q\}, \quad s = 1, \dots, S, \quad B = \sum_{s=1}^S K_s \quad (4.1)$$

服务缓存配置在周期开始时更新，并在周期内保持不变；入口服务器则逐任务决定卸载位置。两类决策分别确定服务副本分布和任务执行资源，通过服务获取开销相互联系。

4.2.2 DAG 表示与通信模型

应用 m 的任务依赖关系表示为

$$G_m = (V_m, E_m), \quad E_m \subseteq V_m \times V_m \quad (4.2)$$

其中， V_m 为真实任务集合， E_m 为依赖边集合。若 $(j, i) \in E_m$ ，任务 i 需等待前驱 j 完成且其输出到达执行服务器。任务 i 所需 CPU 周期数、初始输入量和服务类型分别为 $c_{m,i}$ 、 $d_{m,i}^{in}$ 和 $q_{m,i}$ ；其直接前驱与直接后继集合分别记为 $Pred_{m,i}$ 和 $Succ_{m,i}$ ，前驱 j 向该任务传输的数据量为 $d_{m,j,i}$ 。无直接后继的任务构成出口任务集合：

$$V_m^{out} = \{i \in V_m : Succ_{m,i} = \emptyset\} \quad (4.3)$$

应用接纳时，其完整 DAG 拓扑与任务静态属性已知。环境按依赖关系和既定拓扑次序给出当前可分配任务，入口服务器只为该任务选择执行服务器，不同时优化整个 DAG 的任务分配次序。记第 t 次卸载决策后的已分配任务集合为 $\Omega_{m,t}$ 、任务数为 N_m ，则

$$\Omega_{m,0} = \emptyset, \quad \Omega_{m,N_m} = V_m, \quad N_m = |V_m| \quad (4.4)$$

每次分配后，环境依据前驱的执行位置与完成时刻、当前缓存配置及服务器参数递推任务完成时刻，再推进后续分配。任务分配步仅表示决策次序，不直接表示物理时间的流逝。

记 g_m 为应用 m 的入口服务器， R_{m,g_m}^u 为用户至入口服务器的上行速率， $\ell_{s',s}$ 为源服务器 s' 至目标服务器 s 的单位数据传输时间，且 $\ell_{s,s} = 0$ 。该参数由直连链路或既定多跳路径确定。任务初始输入先上传到入口服务器，必要时再转发至执行服务器，输入传输时延为

$$\tau_{m,i,s}^{usr} = d_{m,i}^{\text{in}} \left(\frac{1}{R_{m,g_m}^u} + \ell_{g_m,s} \right) \quad (4.5)$$

当任务在入口服务器执行时，式 (4.5) 的服务器间转发项为零。除初始输入外，任务还需接收全部直接前驱的输出。记应用到达时刻为 A_m ，前驱 j 的完成时刻与执行位置分别为 $F_{m,j}$ 、 $s_{m,j}$ ，则任务 i 在服务器 s 上的前驱数据到齐时刻为

$$R_{m,i,s}^{\text{pre}} = \begin{cases} A_m, & \text{Pred}_{m,i} = \emptyset, \\ \max_{j \in \text{Pred}_{m,i}} (F_{m,j} + d_{m,j,i} \ell_{s_{m,j},s}), & \text{Pred}_{m,i} \neq \emptyset \end{cases} \quad (4.6)$$

式 (4.6) 取各前驱输出到达目标服务器的最晚时刻；无前驱任务以应用到达时刻为起点。前驱与当前任务位于同一服务器时，相应的依赖数据传输项为零。

4.2.3 任务执行时间模型

服务器后台等待以外生工作量 W_s 表征，其单位为 CPU 周期数。任务 i 在服务器 s 上的计算时延与后台等待时延分别为

$$\tau_{m,i,s}^{\text{cmp}} = \frac{c_{m,i}}{f_s}, \quad \tau_s^{\text{wait}} = \frac{W_s}{f_s} \quad (4.7)$$

本章采用多核并行的简化模型，以外生后台工作量刻画等待，不显式建模用户任务之间的排队竞争，也不设置批处理决策。服务器频率同时影响计算与后台等待时延，缓存布局则通过服务获取开销影响任务完成时刻。

记 $s_{m,i}$ 为任务 i 的执行服务器， $\tau_{m,i,s}^{svc}$ 为服务加载时延，其定义见 4.2.4 节。以前驱数据到齐为起点，将初始输入传输、服务加载、后台等待和计算作为顺序阶段，任务完成时刻为

$$F_{m,i} = R_{m,i,s}^{pre} + \tau_{m,i,s}^{usr} + \tau_{m,i,s}^{svc} + \tau_s^{wait} + \tau_{m,i,s}^{cmp}, \quad s = s_{m,i} \quad (4.8)$$

式 (4.8) 保留上述阶段的顺序累加关系，不计各阶段之间的重叠执行。 $F_{m,i}$ 是统一时间轴上的完成时刻，而 $F_{m,i} - A_m$ 是相对应用到达时刻的任务完成时间。应用需等待全部出口任务完成，其完成时间定义为

$$T_m = \max_{i \in V_m^{out}} (F_{m,i} - A_m) \quad (4.9)$$

因此，应用完成时间由最晚完成的出口任务决定，而不是所有任务耗时之和。单出口时，式 (4.9) 退化为该出口相对应用到达时刻的完成时间。卸载位置改变局部开销后，还会通过前驱递推关系影响后续任务。

4.2.4 服务缓存与服务获取模型

记第 k 个缓存周期的配置为 $X^{(k)} = [x_{s,q}^{(k)}]$ 。当服务器 s 缓存服务 q 时， $x_{s,q}^{(k)} = 1$ ，否则取零。同一服务允许在多台服务器上保留副本，但各服务器均须满足自身容量约束：

$$\sum_{q=1}^Q x_{s,q}^{(k)} \leq K_s, \quad x_{s,q}^{(k)} \in \{0, 1\}, \quad s = 1, \dots, S \quad (4.10)$$

服务的缓存占用按槽位计量，镜像大小则用于计算加载时延。记服务 q 的镜像大小为 L_q ，服务器 s 从云端获取单位服务数据的时间为 ℓ_s^c 。目标服务器未缓存所需服务时，从云端或已有边缘副本中选择加载时延最小的来源，对应的单位服务获取时间为

$$\phi_{s,q}(X^{(k)}) = \min \left\{ \ell_s^c, \min_{s': x_{s',q}^{(k)}=1} \ell_{s',s} \right\} \quad (4.11)$$

若服务在边缘侧没有副本，式 (4.11) 的内层最小值取 $+\infty$ ，由云端提供服务。根据任务需求与目标服务器的缓存状态，服务加载时延为

$$\tau_{m,i,s}^{svc} = L_{q_{m,i}} \phi_{s,q_{m,i}}(X^{(k)}) (1 - x_{s,q_{m,i}}^{(k)}) \quad (4.12)$$

目标服务器已缓存服务时，式 (4.12) 取零；未命中时，加载时延由镜像大小及可用来源共同决定。所有可达边缘服务器均可通过缓存或按需加载获得服务，因此 $K_s = 0$ 的服务器仍可执行任务。

按需加载不改变持久缓存矩阵 $X^{(k)}$ ，也不形成供后续任务复用的缓存副本。若目标服务器仍未缓存某项服务，后续同类任务仍需计入该服务的加载时延。周期性缓存更新与任务执行前的临时服务获取是两类不同操作。

4.2.5 优化目标与问题定义

服务缓存策略 π_C 决定各周期的配置，卸载策略 π_O 为当前任务选择执行服务器。记 $a_{m,i,s}$ 为二元任务分配变量，任务 i 分配至服务器 s 时取一，否则取零。两类策略均使用决策时可获得的信息。

缓存策略使用周期开始前的历史记录。记 Γ_{k-1} 为第 k 个周期开始前已观测的服务请求、任务执行反馈与既有缓存配置； K 、 L 和 ℓ 分别汇总缓存容量、镜像大小以及边缘互连与云端加载链路参数，则

$$X^{(k)} = \pi_C(\Gamma_{k-1}, K, L, \ell) \quad (4.13)$$

卸载状态 $z_{m,i}$ 汇总当前任务属性、前驱位置、缓存配置与历史执行质量权重。其状态空间 $\mathcal{Z}^{cau}$ 由已知 DAG 静态属性及决策时可获得的信息确定，不包含尚未观测到的未来执行反馈。相应决策写为

$$\begin{aligned} s_{m,i} &= \pi_O(z_{m,i}), \quad z_{m,i} \in \mathcal{Z}^{cau}, \\ a_{m,i,s} &= \mathbb{I}\{s = s_{m,i}\}, \quad s = 1, \dots, S \end{aligned} \quad (4.14)$$

其中， $\mathbb{I}\{\cdot\}$ 为示性函数。考虑包含 M 个应用实例的评价时域，以平均应用完成时间的期望为目标，联合优化问题表示为

$$\begin{aligned} P1: \quad & \min_{\pi_C, \pi_O} \quad \mathbb{E} \left[\frac{1}{M} \sum_{m=1}^M T_m \right] \\ s.t. \quad & \sum_{s=1}^S a_{m,i,s} = 1, \quad m = 1, \dots, M, i \in V_m, \\ & \sum_{q=1}^Q x_{s,q}^{(k)} \leq K_s, \quad s = 1, \dots, S, \forall k, \\ & a_{m,i,s} \in \{0, 1\}, \quad x_{s,q}^{(k)} \in \{0, 1\} \end{aligned} \quad (4.15)$$

式 (4.15) 中的完成时间由通信、服务加载及任务完成时刻递推关系确定，期望针对工作流、网络与服务器状态的随机性。约束分别要求每个任务唯一分配、各服务器满足缓存容量限制，以及决策变量取二元值；DAG 依赖由式 (4.6) 与式 (4.8) 保证，策略还须满足式 (4.13) 和式 (4.14) 的信息条件。

缓存配置改变服务获取代价，卸载位置又决定各服务器观察到的服务需求，两类离散决策相互耦合。下一节利用两类决策更新频率的差异，在慢时间尺度协调服务缓存，在快时间尺度选择任务执行位置。

4.3 TCOP 算法设计

TCOP 将服务缓存协调与任务卸载划分为两个时间尺度。DCC 在缓存周期结束后汇总历史需求与执行反馈，确定下一周期的服务副本布局；卸载网络在周期

内固定的缓存配置下，为当前任务选择执行服务器。本节依次介绍缓存协调机制、卸载决策建模、任务—服务器匹配评分网络及训练流程。

4.3.1 DCC 服务缓存协调机制

DCC 以所有服务均从云端加载为比较基准，评价候选缓存配置可节省的服务获取时间。记 $\lambda_{s,q}^{(k)}$ 为截至周期 k 的历史需求权重，即分配至服务器 s 且需要服务 q 的请求占全网服务请求的平滑比例。候选配置 X 的历史加权加载节省为

$$J_k(X) = \sum_{s=1}^S \sum_{q=1}^Q \lambda_{s,q}^{(k)} L_q [\ell_s^c - \phi_{s,q}(X)] \quad (4.16)$$

其中， $\phi_{s,q}(X)$ 按式 (4.11) 计算。单位获取时间差乘以镜像大小 L_q ，得到一次请求相对云端加载的时延节省，再按历史需求加权。 $J_k(X)$ 衡量历史需求加权的服务器加载时延节省，用于评价候选缓存配置。记周期内已分配任务总数为 n_k 、服务器—服务请求数为 $Y_{s,q}^{(k)}$ ，采用指数移动平均更新需求权重：

$$\lambda_{s,q}^{(k)} = \begin{cases} (1 - \alpha_d) \lambda_{s,q}^{(k-1)} + \alpha_d \frac{Y_{s,q}^{(k)}}{n_k}, & n_k > 0, \\ \lambda_{s,q}^{(k-1)}, & n_k = 0 \end{cases} \quad (4.17)$$

其中， α_d 为需求平滑系数。无请求时保留原权重。为区分服务器的历史执行质量，DCC 对分配至服务器 s 的任务统计计算时延与后台等待时延之和的均值，记为 $\tau_{s,k}^{obs}$ ；服务加载与前驱通信不计入该观测。若本周期该服务器的任务数 $n_{s,k}$ 大于零，则

$$\mu_{s,k} = (1 - \alpha_h) \mu_{s,k-1} + \alpha_h \tau_{s,k}^{obs}, \quad n_{s,k} > 0 \quad (4.18)$$

其中， α_h 为执行反馈平滑系数。首次观测时以 $\tau_{s,k}^{obs}$ 初始化 $\mu_{s,k}$ ，无本地新观测时保持原值。评分所用时延估计 $\mu_{s,k}^{fb}$ 优先取本地历史值；缺少本地历史时，取

全网任务平均执行时延的移动平均。另以相同方式平滑本周期平均 CPU 周期数，得到 c_k^{ema} ，将 $\frac{c_k^{ema}}{c_{\max}}$ 限制在区间 $[0, 1]$ 内得到 χ_k 。服务器执行质量权重为

$$\omega_{s,k} = \left(\frac{\min_{s'=1,\dots,S} \mu_{s',k}^{fb}}{\max \{ \mu_{s,k}^{fb}, \varepsilon_{num} \}} \right)^{w_c \chi_k} \quad (4.19)$$

其中， $c_{\max}$ 为计算量参考上限， w_c 控制权重调整强度， ε_{num} 为防止除零的数值下限。历史执行时延较短的服务器获得较高权重； χ_k 越大，对较慢服务器的评分折减越明显。权重初始取一， $\chi_k = 0$ 时也统一取一。周期结束后得到的 ω_k 在下一周期启用，因此周期 k 内卸载决策使用 ω_{k-1} 。

向服务器 s 添加服务 q 时，将候选配置中的对应元素由零改为一，得到 X' 。以 $v_q(X)$ 表示当前配置中该服务的副本数，边际加载节省与添加评分分别为

$$\begin{aligned} \Delta_{s,q}^{(k)}(X) &= J_k(X') - J_k(X) \\ \psi_{s,q}^{(k)}(X) &= \frac{\omega_{s,k} \Delta_{s,q}^{(k)}(X)}{1 + v_q(X)} \end{aligned} \quad (4.20)$$

评分同时考虑新增副本的加载节省、缓存位置的执行质量和已有同类副本数。只有配置年龄 $h_s^{(k)}$ 达到最小保持周期 $H_{\min}$ 的服务器允许调整，其余保持当前配置。候选布局的贪心构造步骤见算法 4.1。

贪心添加后，DCC 通过服务替换修复全网需求服务覆盖。替换仅作用于允许调整的服务器，以尚无边缘副本的需求服务替换已有服务；若被替换服务仍有历史需求，须在其他服务器上保留其副本。在服务器 s 上以服务 q 替换服务 r ，得到新候选配置 X' ，替换评分为

$$g_{s,r,q}^{(k)}(X) = \omega_{s,k} [J_k(X') - J_k(X)] \quad (4.21)$$

覆盖修复优先增加需求服务覆盖，不要求替换评分为正，因此可能接受加载节省下降但覆盖增加的替换。候选配置 $X_c^{(k)}$ 仍须经过下述周期更新判定。

配置更新代价按更新前的副本布局计算，仅累加新增缓存服务的加载时延：

$$C_{upd}(X^{(k)}, X_c^{(k)}) = \sum_{s=1}^S \sum_{q: x_{s,q}^{(k)}=0, x_{c,s,q}^{(k)}=1} L_q \phi_{s,q}(X^{(k)}) \quad (4.22)$$

保留或删除已有服务不计入式 (4.22)。该量用于判断是否接受候选配置，不代表真实容器迁移或端到端部署的完整耗时。记周期请求数的移动平均为 n_k^{ema} ，预计请求数取 $n_k^{est} = \max\{n_k^{ema}, 1\}$ ，无请求时保留平滑估计。将至少在一台服务器上具有正历史需求的服务组成集合 Q_k^+ ，以 $Cov_k(X)$ 表示其中已有边缘副本的服务种类数。将候选布局的覆盖数量与加载节省分别减去当前布局的对应值，所得增量记为 ΔCov_k 和 ΔJ_k 。更新规则为

$$X^{(k+1)} = \begin{cases} X_c^{(k)}, & \Delta Cov_k > 0, \\ X_c^{(k)}, & \Delta Cov_k = 0 \text{ and } n_k^{est} \Delta J_k > \eta_{hys} C_{upd}, \\ X^{(k)}, & \text{otherwise} \end{cases} \quad (4.23)$$

其中， η_{hys} 为更新代价门槛系数。式 (4.23) 优先接受覆盖增加的候选配置；覆盖数量相同时，需满足加载节省的代价门槛。未满足接受条件时，下一周期沿用当前配置。配置年龄和周期统计按算法 4.1 更新，新配置及执行质量权重在下一周期启用。

算法 4.1 DCC 服务缓存协调

输入：历史信息 Γ_k 、当前配置 $X^{(k)}$ 、容量 K 、镜像大小 L 、链路参数 ℓ ，以及跨周期保留的平滑统计与配置年龄。

输出：下一周期配置 $X^{(k+1)}$ 及执行质量权重 ω_k 。

1. 按式 (4.17)–(4.19) 更新需求、执行反馈及执行质量权重。
2. 令 $X_g \leftarrow X^{(k)}$ ；在候选配置中清空满足 $h_s^{(k)} \geq H_{\min}$ 的服务器缓存条目。
3. 当存在评分为正的可行添加时，重复步骤 4~5。
4. 选择使式 (4.20) 的添加评分最大的 (s, q) ，令 $X_g[s, q] \leftarrow 1$ 。
5. 更新剩余容量、服务副本数和候选集合。
6. 令 $X_c^{(k)} \leftarrow X_g$ 。
7. 当存在能够增加需求服务覆盖的可行替换时，重复步骤 8~9。
8. 选择使式 (4.21) 的替换评分最大的 (s, r, q) 。
9. 令 $X_c^{(k)}[s, r] \leftarrow 0$ ， $X_c^{(k)}[s, q] \leftarrow 1$ 。
10. 按式 (4.22) 计算新增服务的累计加载代价。
11. 按式 (4.23) 接受候选配置或保留当前配置。
12. 将配置变化的服务器年龄重置为零，其余年龄加一并以 $H_{\min}$ 为上限；清零周期请求计数，保留历史平滑统计，返回 $X^{(k+1)}$ 和 ω_k 。

可行操作仅作用于允许调整的服务器，并满足容量约束和上述覆盖保护条件。添加评分并列时优先保留当前配置中的条目，再按 (s, q) 升序；替换评分并列时按 (s, r, q) 升序。

4.3.2 任务卸载决策过程建模

快时间尺度的卸载过程以一次任务分配为决策步。环境按 DAG 依赖关系和既定拓扑次序提供当前可分配任务，策略仅选择执行服务器。对于应用 m 的当前任务 i ，以深度 Q 学习^[38]构建卸载策略，其输入状态为

$$z_{m,i} = \left[\bar{F}_{m,i}^{pre}, \bar{c}_{m,i}, \bar{d}_{m,i}^{in}, \text{vec}(X^{(k)}), p_{m,i}, \omega_{k-1}, e_{m,i}^{cur}, e_{m,i}^{suc}, \rho_{m,i} \right] \quad (4.24)$$

其中， $\bar{F}_{m,i}^{pre}$ 、 $\bar{c}_{m,i}$ 和 $\bar{d}_{m,i}^{in}$ 分别为缩放后的前驱最晚完成时刻、计算量与初始输入量； $\text{vec}(X^{(k)})$ 为展开后的缓存矩阵； $p_{m,i}$ 标记直接前驱所在服务器。 $e_{m,i}^{cur}$ 与 $e_{m,i}^{suc}$ 分别标记当前任务和直接后继所需服务，对应位置取一，其余取零。 $\rho_{m,i}$ 为任务深

度与应用内最大深度之比，出口深度取一，其他任务深度取直接后继最大深度加一。上述拓扑与服务需求均来自应用接纳时已知的 DAG。

前驱时间特征按应用开始分配时保存的参考尺度 T_m^{ref} 缩放：

$$\bar{F}_{m,i}^{pre} = \begin{cases} \frac{\max_{j \in Pred_{m,i}} F_{m,j}}{T_m^{ref}}, & Pred_{m,i} \neq \emptyset, \\ 0, & Pred_{m,i} = \emptyset \end{cases} \quad (4.25)$$

在第 4.4.1 节的实验设置下，各应用的到达时刻均为零，式 (4.25) 中的完成时刻与相对应用到达时刻的完成时间在数值上相同。参考尺度按用户分别保存，初值为 1 s，在当前应用分配期间保持不变。训练时，在该用户应用的全部真实任务完成分配后，以原尺度的 0.9 倍与本次应用完成时间的 0.1 倍之和更新，供其下一应用使用。计算量与初始输入量的缩放方式为

$$\bar{c}_{m,i} = \frac{c_{m,i}}{c_{\max}}, \quad \bar{d}_{m,i}^{in} = \frac{d_{m,i}^{in}}{D_{\max}} \quad (4.26)$$

其中， $c_{\max}$ 与 $D_{\max}$ 的单位分别为 CPU 周期数和 bit。以上三项连续特征不作上限截断，缩放值超过一时保留原值；冻结评估沿用训练结束时的参考尺度。策略动作仍为服务器编号 $s_{m,i} \in \{1, \dots, S\}$ ，未缓存所需服务的可达服务器也可选择，其服务加载代价按式 (4.12) 计入。

环境完成本次位置分配后，依据式 (4.8) 计算当前任务完成时刻，并推进至下一可分配任务。为衡量分配对整个应用的影响，定义已分配任务的最晚相对完成时间：

$$\Phi_{m,0} = 0, \quad \Phi_{m,t} = \max_{i \in \Omega_{m,t}} (F_{m,i} - A_m), \quad t = 1, \dots, N_m \quad (4.27)$$

以其单步增量构造奖励：

$$r_{m,t} = -\frac{\Phi_{m,t} - \Phi_{m,t-1}}{\zeta}, \quad \zeta > 0 \quad (4.28)$$

其中, ζ 为固定奖励尺度。若新任务未推迟已分配任务的最晚完成时间, 本步奖励为零; 否则按增加的时间给予惩罚。当折扣因子 $\gamma = 1$ 时, 对完整应用轨迹求和有

$$\sum_{t=1}^{N_m} r_{m,t} = -\frac{1}{\zeta} \sum_{t=1}^{N_m} (\Phi_{m,t} - \Phi_{m,t-1}) = -\frac{T_m}{\zeta} \quad (4.29)$$

由于各阶段时延非负, 沿 DAG 递推后, 最晚完成时刻可由至少一个出口任务达到, 故 $\Phi_{m,N_m} = T_m$ 。式 (4.29) 给出了无折扣累计奖励与应用完成时间的负比例关系, 说明奖励定义与应用级目标一致。

4.3.3 任务—服务器匹配评分网络

卸载网络采用 Pairwise PD3QN, 构造任务—服务器成对表示, 并通过共享匹配评分结构评价各执行位置。网络首先构造同维度的任务嵌入与服务器嵌入:

$$\begin{aligned} e_{m,i}^T &= f_T \left(\left[\bar{F}_{m,i}^{pre}, \bar{c}_{m,i}, \bar{d}_{m,i}^{in}, e_{m,i}^{cur}, e_{m,i}^{suc}, \rho_{m,i} \right] \right), \\ e_{m,i,s}^S &= f_S \left(\left[x_s^{(k)}, p_{m,i,s}, \omega_{s,k-1}, x_{s,q_{m,i}}^{(k)} \right] \right) \end{aligned} \quad (4.30)$$

其中, f_T 与 f_S 均为含两个全连接层的前馈编码器, 层间采用层归一化与激活函数, 输出维度为 d_h 。两类编码器参数彼此独立, 同一策略网络内的候选服务器共享 f_S 。 $x_s^{(k)}$ 为服务器缓存行向量, $p_{m,i,s}$ 表示是否有直接前驱位于该服务器, $x_{s,q_{m,i}}^{(k)}$ 标记当前服务是否已缓存。前驱位置与缓存命中信息随当前任务变化。

为结合全部候选服务器的信息, 以任务嵌入为查询, 对服务器嵌入进行缩放点积注意力汇总^[48]。省略应用下标 m , 记可训练投影矩阵为 W_Q 、 W_K 、 W_V , 则

$$\begin{aligned}\alpha_{i,s} &= \text{softmax}_s \left(\frac{\langle W_Q e_i^T, W_K e_{i,s}^S \rangle}{\sqrt{d_h}} \right) \\ o_i &= \sum_{s=1}^S \alpha_{i,s} W_V e_{i,s}^S\end{aligned}\tag{4.31}$$

其中, $\alpha_{i,s}$ 为归一化注意力权重, o_i 为服务器信息汇总向量。将任务嵌入与汇总向量拼接, 经全连接层、归一化和激活得到共享表示 $h_i = f_C([e_i^T; o_i])$, 融合模块将 $2d_h$ 维输入映射至 d_h 维。

网络采用 Dueling 结构^[40], 分别估计状态价值与候选服务器优势。价值分支输入 h_i ; 优势分支将其与服务器嵌入 $e_{i,s}^S$ 相加, 经归一化和激活得到联合表示 $h_{i,s}^P$, 再输出该服务器的优势值。动作价值为

$$Q_\theta(z_{m,i}, s) = V_\theta(h_i) + A_\theta(h_{i,s}^P) - \frac{1}{S} \sum_{s'=1}^S A_\theta(h_{i,s'}^P)\tag{4.32}$$

其中, θ 为在线网络参数。将各服务器优势减去均值, 再与状态价值相加, 得到可供比较的动作价值。评估时选择价值最大的服务器:

$$s_{m,i}^* = \underset{s \in \{1, \dots, S\}}{\operatorname{argmax}} Q_\theta(z_{m,i}, s)\tag{4.33}$$

各候选服务器共享编码器与优势评分结构, 因此策略能够以相同计算规则比较任务与不同服务器的匹配程度。该结构不为每个服务器单独维护一套特征编码网络; 不同入口服务器之间的策略参数仍分别训练和保存。

4.3.4 缓存更新、卸载决策与训练流程

每个入口服务器分别维护在线网络、目标网络和经验池, 由接入该服务器的应用共享。训练期间, 缓存配置与历史执行质量权重在周期内固定; 到达更新边界时, 由 DCC 根据上一周期统计执行更新判定, 确定下一周期配置。逐任务分配同时积累请求计数与执行反馈, 为下一周期更新提供依据。

算法 4.2 TCOP 双时间尺度训练流程

1. 为各入口服务器初始化在线网络、目标网络与经验池，并初始化缓存配置、历史统计、执行质量权重和配置年龄。
 2. 依次接收训练 DAG；若到达缓存更新边界，基于上一周期统计运行算法 4.1 并启用其输出，随后初始化当前应用轨迹。
 3. 由环境给出当前可分配任务，按式 (4.24) 构造状态。
 4. 按训练探索规则选择服务器，完成任务分配，计算式 (4.28) 的奖励并获得后继状态。
 5. 保存转移及 DCC 所需统计；若仍有未分配任务，返回步骤 3。
 6. 应用轨迹结束后构造 3-step 样本，存入所属入口服务器经验池，并更新该用户下一应用使用的参考时间尺度。
 7. 按预设计划采样经验，执行式 (4.35) 的参数更新和目标网络同步，继续处理训练应用。
 8. 训练结束后保存各入口策略及对应缓存配置，用于冻结评估。
-

训练探索兼顾缓存命中与全候选服务器选择。若存在已缓存当前服务的服务器，则以概率 $\beta = 0.1$ 在这些服务器中等概率选择；其余情况下在全部候选服务器上执行 $\varepsilon - greedy$ 策略。任务分配后记录状态、动作、增量奖励和后继状态，待当前应用全部真实任务完成分配，再构造 3-step 回放样本；轨迹末端不足三步时使用剩余步数。

记 $n_t = \min \{3, N_m - t + 1\}$ 为从分配步 t 开始聚合的步数。采用无折扣回报，并结合 Double-Q 估计^[39]构造目标值：

$$y_t = \sum_{j=0}^{n_t-1} r_{t+j} + (1 - \delta_{t+n_t}) Q_{\theta^-} (z_{t+n_t}, s_{t+n_t}^*) \quad (4.34)$$

其中， z_{t+n_t} 为累计 n_t 步后的状态， δ_{t+n_t} 为该状态的终止标记。全部真实任务完成分配时取一，否则取零。对于非终止样本，在线网络选择 $s_{t+n_t}^*$ ，参数为 θ^- 的目标网络评价其后续回报；终止样本只保留实际累计奖励。由入口服务器经验池均匀采样 N_B 条样本，最小化 Huber 损失：

$$\mathcal{L}(\theta) = \frac{1}{N_B} \sum_{b=1}^{N_B} \text{Huber}(y_b - Q_\theta(z_b, s_b)) \quad (4.35)$$

Huber 损失对较大误差采用线性惩罚。在线网络按预设间隔使用 Adam^[49]更新，并以梯度裁剪限制梯度幅度；目标网络按既定计划同步在线参数。缓存更新、逐任务交互与训练的执行顺序见算法 4.2。

冻结评估关闭探索，即令 $\beta = \varepsilon = 0$ ，按式 (4.33) 选择服务器。具体评估协议见第 4.4.1 节。

4.3.5 复杂度分析

DCC 的计算开销主要来自贪心构造与覆盖修复。设一次加载节省指标的求值开销为 C_J 。贪心构造至多添加 B 个副本，每轮评估至多 SQ 个候选，计算量上界为 $O(BSQC_J)$ 。覆盖修复每轮至少增加一种需求服务覆盖，至多进行 $\min\{B, Q\}$ 轮，每轮检查至多 BQ 个替换组合，其上界为 $O(BQ \min\{B, Q\} C_J)$ 。按服务组织缓存位置时，一次指标求值的开销为 $C_J = O(S(Q + B))$ 。

匹配评分网络对当前任务编码一次，并依次编码、汇总和评价 S 个候选服务器。嵌入维度为 d_h 时，单次前向计算量上界为

$$O(Sd_h(Q + d_h)) \quad (4.36)$$

当服务种类数 Q 与嵌入维度 d_h 固定时，单次卸载推理的计算量随候选服务器数线性增长。训练批量大小及每个决策步的平均更新次数固定时，摊销训练计算量与前向计算同阶；冻结评估仅执行前向推理。缓存协调按周期执行，卸载评分逐任务执行。上述复杂度不包含服务实际加载和通信传输耗时，实验性能将在下一节评价。

4.4 TCOP 实验结果与分析

本节在异构缓存约束的 DAG 仿真环境中评价 TCOP，考察应用完成时间及其时延组成，并结合 workflow 类型、计算与缓存资源规模以及缓存与卸载策略的组合，分析方法的性能和各模块的作用。

4.4.1 实验环境及参数设置

实验运行于配备 Apple M3 处理器和 16 GB 内存的计算机，软件环境为 Python 3.12.8、PyTorch 2.7.0、NumPy 1.26.4、SciPy 1.15.1 和 SimPy 4.1.2。上述计算机用于运行仿真与训练，仿真中的边缘服务器按各自的计算、通信和缓存参数建模。主环境参照 DAOC 的网络规模与区域设置^[33]，在 $1\text{ km} \times 1\text{ km}$ 区域内设置 20 个用户、10 台边缘服务器和 10 种服务。

缓存容量采用异构配置，4 台服务器不设缓存，4 台各设 1 个槽位，2 台各设 2 个槽位，总缓存预算为 8 个槽位。容量值经随机排列后分配给各服务器，其分配独立于服务器位置、计算能力与后台工作量，同一随机种子下各方法使用相同的容量配置。零缓存服务器仍可执行任务，所需服务按第 4.2 节的模型从其他边缘服务器或云端加载。

实验采用 Pegasus 和 WorkflowSim 的公开科学 workflow 构造 CyberShake、Epigenomics、Inspiral、Montage 和 Sipht 五类 DAG 应用^[50, 51]。主环境中，每个应用包含 24~30 个真实任务及其依赖关系，并添加一个零开销虚拟源节点；程序与服务的对应关系在各组实验中保持一致。各独立实验场景中，应用到达时刻统一设为零。应用完成时间按式 (4.9) 计算，不计最终结果回传用户的下行时延。

上述五类 workflow 中，虚拟源节点到入口任务的边数据量均设为零，非入口任务不另设初始输入，因此所有真实任务的初始输入量均为零。真实任务之间的依赖边保留数据量，并按式 (4.6) 计入依赖数据传输时延。

服务器后台工作量、CPU 频率、服务镜像大小、通信参数以及任务间依赖数据量和计算量上限参考 DAOC 及其公开实现^[33]。其中，后台工作量和 CPU 频率

采用离散均匀抽样，服务镜像大小采用连续均匀抽样，云端单位服务获取时间等概率取值。主环境参数见表 4.2。

表 4.2 网络与仿真参数

参数	取值或范围
用户数	20
边缘服务器数	10
服务种类数	10
仿真区域	1 km × 1 km
服务器间数据率	约 31.5-40 Gbit/s
云端单位服务获取时间	0.5 或 1.0 ns/bit
服务器后台工作量	0.2-99.8 Mcycles
服务器 CPU 频率	0.2-29.8 Gcycles/s
服务镜像大小	[10, 1000) Mbit
任务初始输入量	0 bit
任务间依赖数据量上限	1.5 Mbit
任务计算量	10-200 Mcycles
无线带宽	15 kHz
各服务器缓存槽位数	[0, 0, 0, 0, 1, 1, 1, 1, 2, 2]
总缓存槽位数	8

各学习方法均从随机初始化开始训练，并统一训练 26,000 回合。超参数在开发随机种子上调优后固定，TCOP 的主要超参数见表 4.3。训练阶段，DCC 每 5 个应用周期执行一次缓存更新判定，仅在候选配置满足第 4.3.1 节的接受条件时更换配置；卸载网络每 5 个决策步执行一次梯度更新。

训练结束后，各方法统一采用第 26,000 回合的最终模型及其对应的服务缓存配置进行冻结评估，不依据测试结果选择模型检查点。配对测试时关闭探索与参数更新，缓存配置不再调整，网络参数与训练结束时的参考尺度保持固定。训练期间的验证轨迹与冻结测试结果分别报告。

表 4.3 TCOP 主要超参数

超参数	取值
状态维数（主环境）	144
嵌入维数	64
折扣因子	1.0
学习率	5×10^{-4} （固定）
优化器	Adam
批大小	64
经验池容量	10,000
梯度更新间隔	5 步
多步回报步数	3
最小探索率	0.01
DCC 缓存更新判定间隔	5 个应用周期
训练轮数	26,000 回合

除另有说明外，各组实验在 10 个随机种子下独立重复，第 4.4.6 节的计算与缓存资源共同扩展实验使用 3 个随机种子。主环境比较中，每个种子包含 100 个配对测试场景，各方法使用相同的任务实例与系统配置。实验分别报告应用完成时间的均值与第 95 百分位数（P95）。图中的柱高或折线标记表示跨种子均值，误差线表示以随机种子为统计单位的 95% Student-t 置信区间，散点表示各随机种子的结果；不能仅凭置信区间是否重叠判断方法间差异的显著性。

4.4.2 对比算法

在相同仿真环境下，将 TCOP 与八种方法进行比较，涵盖启发式卸载、学习型卸载及不同缓存组合。各方法采用第 4.4.1 节的配对测试场景，学习型方法的训练轮数与冻结评估方式保持一致。

学习型方法分别搭配 DAOC-Cache 或 DCC。DAOC-Cache 由各服务器依据本地服务需求与加载代价独立更新缓存^[33]；DCC 则通过中央协调确定跨服务器的服务缓存配置。各对比方法如下。

-
- (1) **Random**: 采用独立流行度缓存, 随机选择任务的执行服务器。
 - (2) **Nearest**: 采用与 **Random** 相同的缓存机制, 选择距入口最近的服务器执行任务。
 - (3) **SA-Nearest**: 结合距离与服务缓存信息, 优先选择已缓存所需服务的最近服务器。
 - (4) **DQN-DAOC-Cache**: 采用基础 DQN^[38] 作为卸载器, 并搭配 DAOC-Cache 进行服务缓存更新。
 - (5) **DAOC^[33]**: 采用分布式 DQN 卸载算法, 并搭配 DAOC-Cache。
 - (6) **DAOC + DCC**: 保留 DAOC 的分布式 DQN 卸载器, 将缓存机制替换为 DCC, 用于考察该卸载器下缓存协调的作用。
 - (7) **SAC + DAOC-Cache**: 使用离散动作版本的软演员—评论家算法 (SAC)^[37] 作为卸载器, 并搭配 DAOC-Cache。
 - (8) **SAC + DCC**: 将离散 SAC 卸载器与 DCC 组合, 用于与 TCOP 比较同一缓存协调机制下的不同卸载策略。

实现中, DAOC 使用两个隐藏层, 每层 64 个神经元, 激活函数为 $\tanh$; 动作引导系数初值为 0.9, 下限为 0.1, 衰减系数为 0.995, 最小探索率为 0.01^[33]。离散 SAC 采用 3-step 回报, 初始熵系数为 0.02, 目标熵为 $0.98 \ln |\mathcal{A}|$, 软更新系数为 0.005, 其中 $|\mathcal{A}|$ 为动作数。

组合消融 (第 4.4.8 节) 将 DAOC、平坦 DDQN (Flat DDQN)、离散 SAC 和 Pairwise PD3QN 分别与上述两种缓存机制组合, 比较缓存协调及卸载器的作用。TCOP 对应 DCC 与 Pairwise PD3QN 的完整组合。Flat DDQN 用于比较平坦输出与匹配评分策略的差异, 不同于总体比较中的基础 DQN。

4.4.3 训练过程与验证表现分析

图 4.2 展示各方法在训练期间的验证表现, 并列出发启式方法的性能作为参照。横轴为训练回合数, 纵轴为平均 DAG 应用完成时间, 数值越低表示验证表现越好。

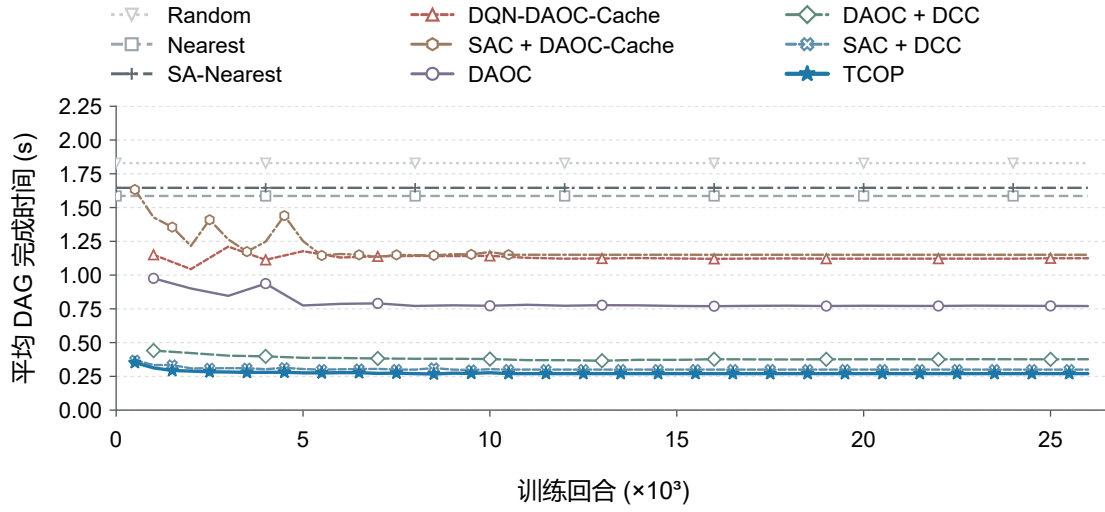


图 4.2 不同方法的训练验证表现

注：学习方法曲线为训练过程中实际记录的验证结果，启发式方法作为性能参照。

TCOP 的验证均值由第 500 回合的 0.3518 s 降至第 5,000 回合的 0.2761 s，第 10,500 回合进一步降至 0.2703 s。从图中可以看出，主要改善集中在训练前期，后续验证均值总体趋于稳定。

第 10,500 回合，SAC + DCC 的平均应用完成时间为 0.3002 s，TCOP 为 0.2703 s，相较之下降低约 9.9%。在同样采用 DCC 缓存协调机制的条件下，TCOP 在该验证点的平均完成时间较低。

训练验证曲线反映策略随训练回合变化的表现；后续总体性能按第 4.4.1 节的冻结评估协议报告。

4.4.4 总体性能分析

图 4.3 展示主环境下各方法的平均与 P95 应用完成时间，局部图放大了 TCOP 与 SAC + DCC 的结果。测试场景及统计口径与第 4.4.1 节一致。

TCOP 的平均应用完成时间为 0.2668 s，较对照方法中均值最低的 SAC + DCC (0.3001 s) 降低 11.09%，较 DAOC (0.7736 s) 降低 65.51%。在所用 10 个随机种子下，TCOP 的均值均低于这两种方法。其 P95 为 0.4336 s，也是本组比较中的最低观测值。

三种启发式方法的平均完成时间为 1.5859–1.8290 s，均高于本组学习型方法。其中，Nearest 为 1.5859 s，低于 Random；SA-Nearest 为 1.6462 s，高于 Nearest。这

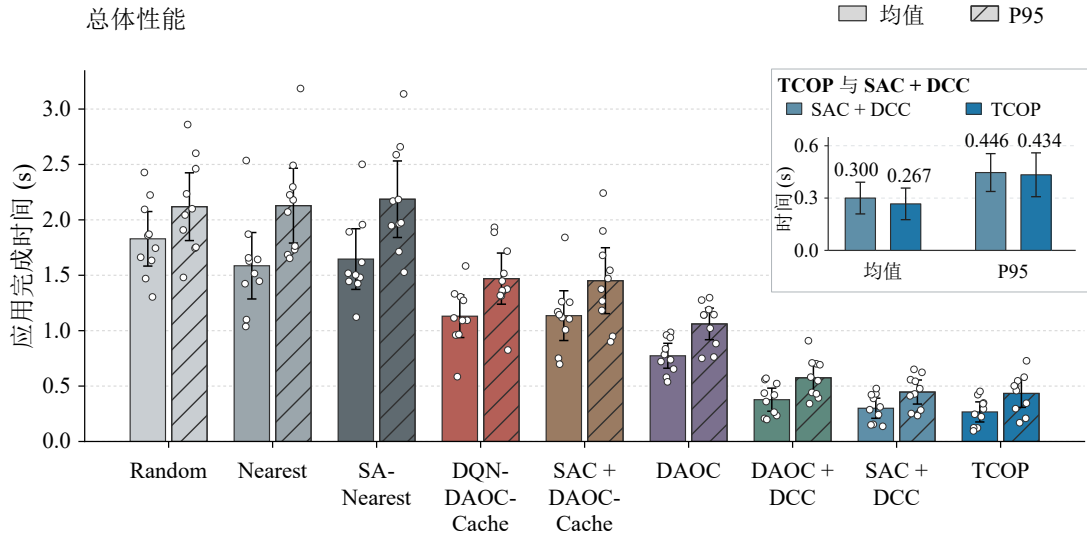


图 4.3 主环境下不同方法的平均与 P95 应用完成时间

注：误差线表示以随机种子为统计单位的 95% Student-t 置信区间，散点表示各随机种子的结果。

表明，在本实验条件下，服务器距离或缓存命中情况不足以单独作为卸载位置的选择依据。

采用 DAOC-Cache 时，DQN-DAOC-Cache 与 SAC + DAOC-Cache 的平均完成时间均约为 1.13 s，DAOC 为 0.7736 s。将缓存机制替换为 DCC 后，DAOC 与离散 SAC 的均值分别降至 0.3773 s 和 0.3001 s，两种卸载器均从缓存协调中受益。在同样采用 DCC 的条件下，TCOP 的平均完成时间进一步降低。第 4.4.7 节和第 4.4.8 节分别通过时延分解与交叉组合比较分析这些差异。

4.4.5 不同工作流类型下的性能分析

本节分别采用 CyberShake、Epigenomics、Inspirall、Montage 和 Sipht 五类工作流，比较各方法的平均 DAG 应用完成时间，结果如图 4.4 所示。各方法在每类工作流下使用相同配置，统计口径与第 4.4.1 节一致。

TCOP 在五类工作流下均取得最低的平均应用完成时间，较 SAC + DCC 降低 0.89%-22.34%，较 DAOC 降低 57.86%-71.90%。因此，在本组工作流比较中，TCOP 的平均性能优势并不局限于某一类工作流。

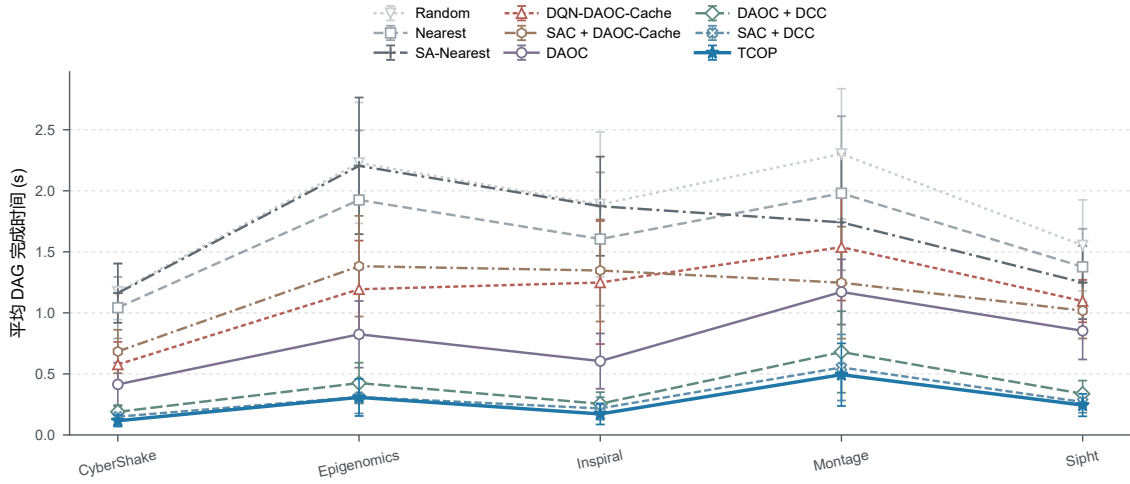


图 4.4 不同 workflow 类型下各方法的平均应用完成时间

注：折线标记表示跨种子均值，误差线表示以随机种子为统计单位的 95% Student-t 置信区间。

不同 workflow 下的改善幅度存在差异。相对 SAC + DCC，TCOP 在 CyberShake 和 Inspiral 上的平均完成时间分别降低 22.34% 和 20.92%，而在 Epigenomics 上仅降低 0.89%，两种方法的均值较为接近。

上述比较反映不同 workflow 整体条件下的性能差异。下一节进一步改变服务器数量与总缓存槽位，考察计算与缓存资源共同扩展时的调度表现。

4.4.6 计算与缓存资源共同扩展分析

本节将边缘服务器数量依次设为 5、10、15 和 20 台，总缓存槽位数同步设为 4、8、12 和 16，比较计算与缓存资源共同扩展时的平均 DAG 应用完成时间。实验采用 3 个随机种子独立重复，结果如图 4.5 所示，以下作描述性比较。

随着服务器数量由 5 台增至 20 台、总缓存槽位由 4 个增至 16 个，TCOP 的平均应用完成时间由 1.0549 s 降至 0.1809 s。由 15 台扩展至 20 台时，其平均完成时间进一步下降的幅度减小。

相对 SAC + DCC，TCOP 在 5 台服务器、4 个缓存槽位时的均值高 0.33%，未表现出均值优势；在 10、15 和 20 台服务器时，均值分别降低 5.92%、16.76% 和 21.25%。在本组比较中，资源较多时 TCOP 的相对均值优势更大。

该实验同时改变计算资源规模与缓存预算，所观察到的变化反映二者共同扩展的效果，不能单独归因于服务器数量或缓存容量。结合上一节的工作流比较，

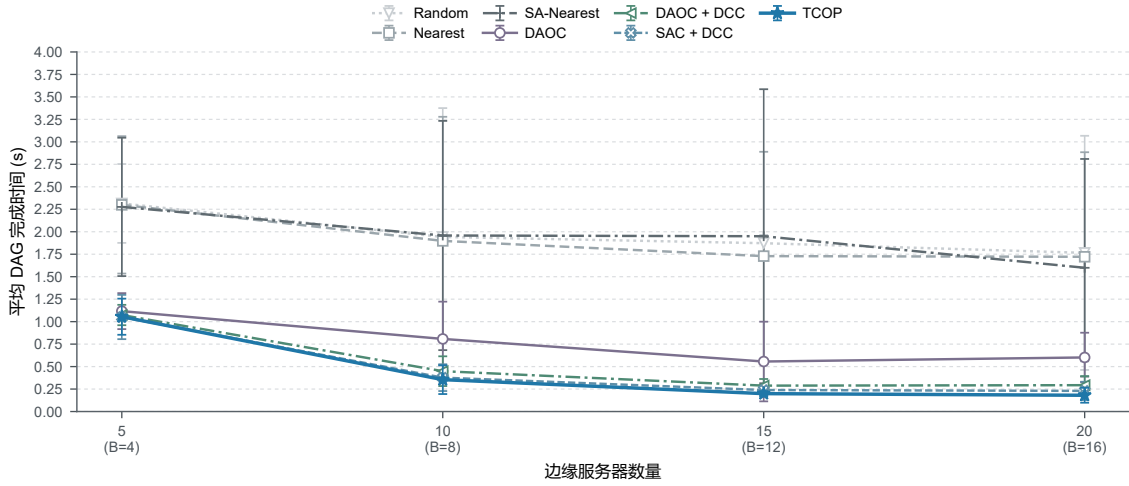


图 4.5 计算与缓存资源共同扩展时的平均应用完成时间

注：横轴括号内标注总缓存槽位数；折线标记表示跨种子均值，误差线表示以随机种子为统计单位的 95% Student-t 置信区间。

TCOP 的性能改善幅度具有配置依赖性。下一节从缓存效果与时延组成分析调度性能的来源。

4.4.7 缓存效果与时延分解分析

本节在第 4.4.1 节的主环境下，结合服务缓存指标与关键路径时延分量分析应用完成时间的变化。实验采用 10 个随机种子独立重复，统计口径与总体性能比较一致。

服务覆盖率是集群中至少具有一个缓存副本的服务种类数占全部服务种类数的比例；缓存命中率是任务直接使用执行服务器本地服务副本的请求比例；远程服务加载率是需要从其他边缘服务器或云端获取服务的请求比例。后两者在该口径下互为补量，结果如图 4.6 所示。

将 DAOC-Cache 替换为 DCC 后，DAOC 与离散 SAC 的服务覆盖率分别由 56% 和 46% 提高至 80%，达到主环境 8 个槽位对 10 种服务的覆盖上限。跨服务器协调减少了重复副本，使有限槽位能够容纳更多服务种类。

相同覆盖率下，不同卸载器的缓存命中率仍有差异。TCOP 的命中率为 58.89%，高于 DAOC + DCC 的 30.62% 和 SAC + DCC 的 42.12%，对应远程服务

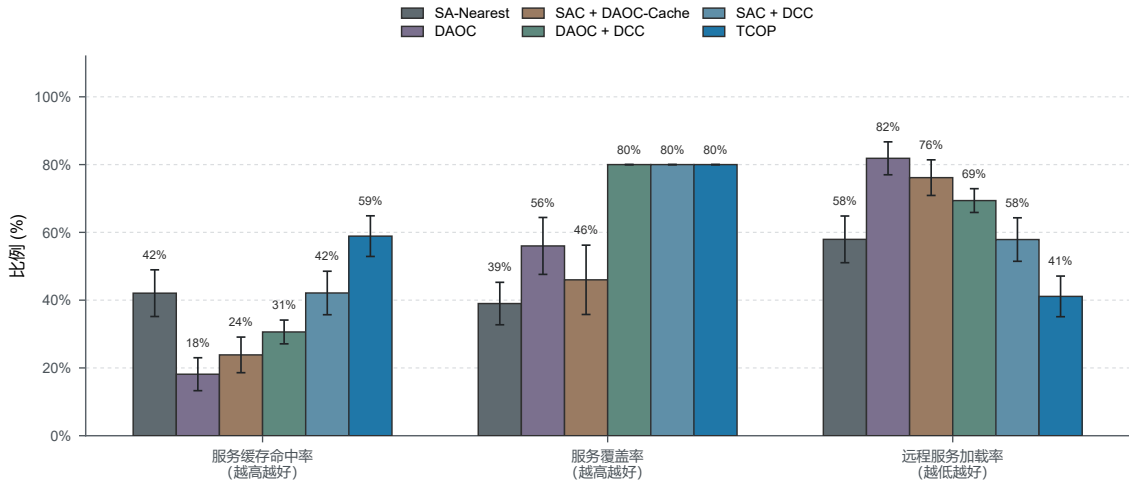


图 4.6 不同方法的服务覆盖与缓存利用

注：柱高为跨种子均值，误差线为以随机种子为统计单位的 95% Student-t 置信区间；图中数值标签经过四舍五入。

加载率为 41.11%。因此，扩大服务覆盖与提高本地副本利用是两个相关但不同的环节，任务执行位置仍需结合已有副本布局确定。

图 4.7 按关键路径比较应用完成时间及主要时延分量。

将 DAOC-Cache 替换为 DCC 后，DAOC 与离散 SAC 的服务加载时延分别由 0.7012 s 降至 0.3083 s、由 1.0424 s 降至 0.2364 s，对应的平均应用完成时间分别降低 51.23% 和 73.58%。不同卸载器下均观察到加载开销下降，与 DCC 利用跨服务器副本信息协调缓存位置的设计一致。

TCOP 的服务加载时延进一步降至 0.2014 s，较 SAC + DCC 减少 35.0 ms，但后台等待时延增加约 1.8 ms。两者的任务处理时延基本相当，最终 TCOP 的平均应用完成时间降低 11.09%。因此，本组性能改善主要来自服务获取，并非各时延分量同时下降。

4.4.8 缓存策略与卸载策略的组合消融分析

为比较缓存机制与卸载器组合下的调度效果，本节将 DAOC 卸载器、平坦 DDQN (Flat DDQN)、离散 SAC 和 Pairwise PD3QN 分别与 DAOC-Cache、DCC 组合，形成八组方法。实验在主环境下采用 10 个随机种子独立重复，结果如图 4.8 所示。

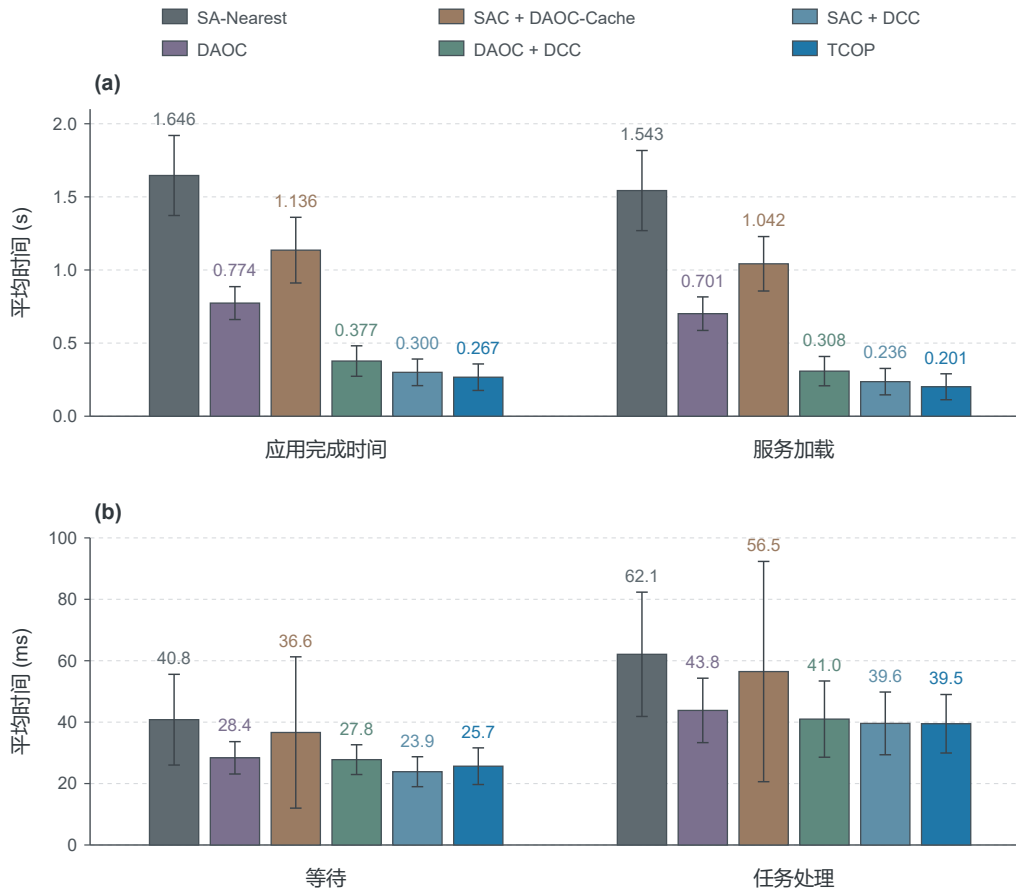


图 4.7 不同方法的应用完成时间与主要关键路径时延分量

注：(a) 单位为 s，(b) 单位为 ms；后台等待仅计外生工作量。本组工作流的初始输入量设为零，故初始输入传输为 0 ms；依赖数据传输为 0.168-0.257 ms。误差线含义同图 4.6。

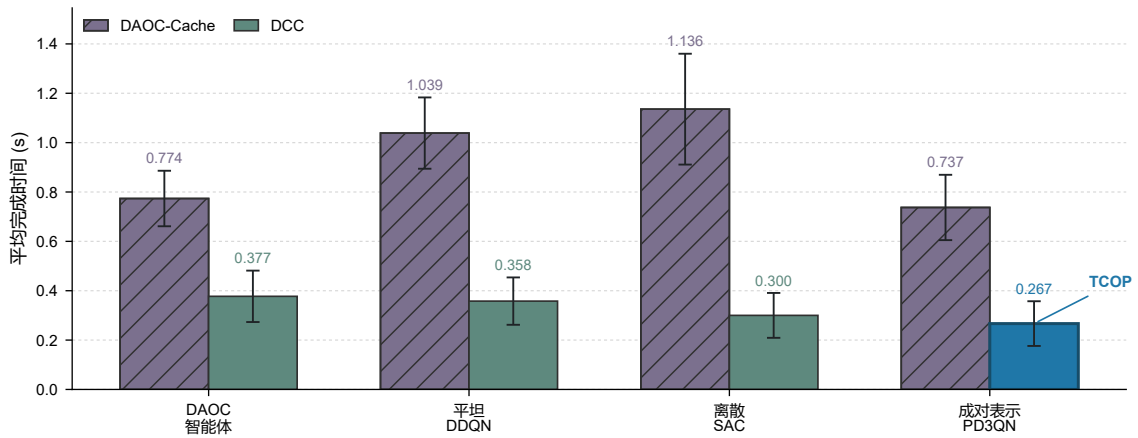


图 4.8 不同缓存机制与卸载策略组合下的平均应用完成时间

注：柱高为跨种子均值，误差线为以随机种子为统计单位的 95% Student-t 置信区间；图中数值标签经过四舍五入。

在卸载器类型相同的条件下,将 DAOC-Cache 替换为 DCC 后,四种卸载器的平均应用完成时间均下降,降幅为 51.23%-73.58%。DCC 在与不同卸载器组合时均获得较低的完成时间均值,其效果并不限于与 Pairwise PD3QN 搭配。

在 DAOC-Cache 和 DCC 两种缓存机制下,Pairwise PD3QN 对应组合的平均应用完成时间分别比 Flat DDQN 对应组合降低 29.01% 和 25.46%。Flat DDQN 与总体性能比较中的基础 DQN 是不同的控制方法。

各组合使用各自最终模型及对应缓存布局进行冻结评估,比较的是缓存机制与卸载器的整体组合性能,尚未分离任务-服务器匹配评分结构的独立增益。TCOP 在本组八种组合中取得最低平均应用完成时间。

4.5 本章小结

本章面向计算能力与缓存容量异构的边缘系统,提出双时间尺度的服务协调与 DAG 任务卸载方法 TCOP。该方法协调低频服务缓存更新与逐任务执行位置选择,以降低具有服务需求和任务依赖的应用的平均完成时间。

仿真实验表明,在主环境下,TCOP 的平均应用完成时间为 0.2668 s,较 SAC + DCC 和 DAOC 分别降低 11.09% 和 65.51%。缓存指标与关键路径时延分解结果表明,性能改善主要体现在服务加载开销减少,并伴随服务覆盖扩大和本地副本利用提高。组合实验表明,不同缓存机制与卸载器的搭配会影响整体性能。不同工作流与资源配置下的改善幅度存在差异,在最小资源配置下,TCOP 未优于 SAC + DCC。

本章采用给定缓存容量与外生后台工作量,尚未显式刻画并发用户任务之间的排队竞争,真实服务迁移及边缘系统中的运行效果仍需验证。

第五章 总结与展望

5.1 全文工作总结

本文围绕异构边缘环境下的低时延任务调度问题，分别研究模型预部署条件下的多 DNN 在线推理调度，以及缓存容量受限条件下的 DAG 任务卸载。前者关注动态负载和执行干扰对请求时延的影响，后者关注任务依赖与服务布局对应用完成时间的影响。主要工作总结如下。

(1) 针对多 DNN 在线推理中的执行状态变化，提出历史增强与风险感知调度方法 HERA，将近期已完成批次的反馈与请求风险用于在线调度。基于实测硬件性能标定的仿真实验表明，在 Random-switch 动态负载下，HERA 相对 FCFS 的总体 SLA 违约率和 P99 时延分别降低 8.79% 和 20.12%。消融结果显示，历史编码有助于降低动态负载下的违约率和执行能耗，风险门控主要抑制尾部时延增长。静态与动态两类主实验中的平均单请求执行能耗均低于设定阈值。

(2) 针对缓存受限的 DAG 应用，提出双时间尺度服务协调与任务卸载方法 TCOP，协调服务布局与任务执行位置。在主实验配置的冻结评估中，TCOP 的平均应用完成时间为 0.2668 s，较 SAC + DCC 降低 11.09%。缓存指标与时延分解显示，主要时延改善体现在服务加载开销减少，并伴随服务覆盖扩大和本地副本利用提高。组合实验反映了缓存机制与卸载器搭配对整体性能的影响。

两项研究分别从运行时执行决策和服务资源协调入手：模型可直接执行时，重点在于利用运行反馈判断请求风险；服务尚需获取时，还应协调缓存布局与任务位置。上述认识分别基于两类任务模型及其评价结果。

5.2 未来工作展望

结合现有研究的模型假设与实验范围，后续可从以下方面继续开展研究。

(1) 联合刻画任务排队与服务部署。本文分别讨论预部署模型下的独立请求和缓存受限的依赖任务，尚未在同一模型中处理两类约束。未来可面向持续到达的依赖任务，将并发用户任务的排队竞争、服务加载与迁移代价纳入统一时间模型，研究缓存调整、执行位置和批处理决策之间的影响。在此基础上，评估联合决策相对独立优化的收益及其适用条件。

(2) 提高策略对运行环境变化的适应能力。现有风险门控采用固定阈值，缓存协调采用预设更新周期。后续可依据近期执行预测误差与违约反馈在线调整风险阈值，并根据服务需求变化调整缓存更新频率，在不同负载与资源配置下评估调整效果及额外开销。还可考察模型集合或节点配置改变后的策略迁移效果。

(3) 扩展推理质量与资源消耗的权衡。本文多 DNN 调度保持原始模型结构，不涉及剪枝、量化或提前退出，因而未将推理精度作为决策变量。未来可在明确任务精度要求的前提下，引入不同模型版本或推理模式，联合选择执行设备、批大小与推理配置，分析精度、时延和能耗之间的关系，尤其关注资源紧张时不同请求的服务质量变化。

(4) 开展真实边缘原型验证。现有结果主要来自仿真，硬件标定能够提供执行性能依据，但仍需验证完整调度流程的实际开销。后续可在异构边缘设备上实现请求接入、状态采集、策略推理和服务缓存管理，测量服务迁移、网络竞争与决策计算对端到端时延的影响，并在一致负载与资源预算下进行对照实验，检验所提方法在实际运行条件下的效果。

参考文献

- [1] MAO Y, YOU C, ZHANG J, et al. A survey on mobile edge computing: The communication perspective[J]. IEEE Communications Surveys & Tutorials, 2017, 19(4): 2322-2358. DOI: 10.1109/COMST.2017.2745201.
- [2] 丁男, 方玺淇, 郝云涛, 等. 面向移动机器人端边协同推理的混合数据流调度算法研究 [J]. 电子学报, 2025, 53(12): 4575-4591. DOI: 10.12263/DZXB.20250674.
- [3] ZHANG Z, ZHAO Y, LI H, et al. BCEdge: SLO-aware DNN inference services with adaptive batch-concurrent scheduling on edge devices[J]. IEEE Transactions on Network and Service Management, 2024, 21(4): 4131-4145. DOI: 10.1109/TN SM.2024.3409701.
- [4] DEAN J, BARROSO L A. The tail at scale[J]. Communications of the ACM, 2013, 56(2): 74-80. DOI: 10.1145/2408776.2408794.
- [5] ZHAO G, XU H, ZHAO Y, et al. Offloading tasks with dependency and service caching in mobile edge computing[J]. IEEE Transactions on Parallel and Distributed Systems, 2021, 32(11): 2777-2792. DOI: 10.1109/TPDS.2021.3076687.
- [6] 李云, 南子煜, 姚枝秀, 等. 面向 DAG 任务的分布式智能计算卸载和服务缓存联合优化 [J]. 中山大学学报 (自然科学版中英文), 2025, 64(1): 71-82. DOI: 10.13471/j.cnki.acta.snus.ZR20240181.
- [7] ISLAM A, DEBNATH A, GHOSE M, et al. A survey on task offloading in multi-access edge computing[J]. Journal of Systems Architecture, 2021, 118: 102225. DOI: 10.1016/j.sysarc.2021.102225.
- [8] 谢满德, 黄竹芳, 孙浩. 云边端协同下多用户细粒度任务卸载调度策略 [J]. 电信科学, 2024, 40(4): 107-121. DOI: 10.11959/j.issn.1000-0801.2024086.
- [9] 李松, 李顺, 王博文, 等. 数字孪生边缘网络端到端时延优化的任务卸载与资源分配方法 [J]. 电子与信息学报, 2025, 47(3): 633-644. DOI: 10.11999/JEIT240344.

- [10] SUN G, YUAN M, SUN Z, et al. Online collaborative resource allocation and task offloading for multi-access edge computing[J]. IEEE Transactions on Mobile Computing, 2025, 24(11): 11430-11448. DOI: 10.1109/TMC.2025.3580365.
- [11] TANG M, WONG V W S. Deep reinforcement learning for task offloading in mobile edge computing systems[J]. IEEE Transactions on Mobile Computing, 2022, 21(6): 1985-1997. DOI: 10.1109/TMC.2020.3036871.
- [12] 卢先领, 李德康. 面向大规模多接入边缘计算场景的任务卸载算法 [J]. 电子与信息学报, 2025, 47(1): 116-127. DOI: 10.11999/JEIT240624.
- [13] 代钰, 景宗明, 杨雷, 等. 部分可观测环境中基于图强化的任务卸载与资源分配方法 [J]. 东北大学学报 (自然科学版), 2025, 46(1): 9-17. DOI: 10.12068/j.issn.1005-3026.2025.20230216.
- [14] WANG Z, GOUDARZI M, BUYYA R. TF-DDRL: A Transformer-enhanced distributed DRL technique for scheduling IoT applications in edge and cloud computing environments[J]. IEEE Transactions on Services Computing, 2025, 18(2): 1039-1053. DOI: 10.1109/TSC.2025.3528346.
- [15] 张智涵, 张天魁, 党梅梅, 等. 边缘智能计算的协同推理关键技术研究 [J/OL]. 电信科学, 2026[2026-09-23]. DOI: 10.11959/j.issn.1000-0801.DXKX260099.
- [16] ZHANG Z, ZHAO Y, LI H, et al. DVFO: Learning-based DVFS for energy-efficient edge-cloud collaborative inference[J]. IEEE Transactions on Mobile Computing, 2024, 23(10): 9042-9059. DOI: 10.1109/TMC.2024.3357218.
- [17] ZHAO Z, LING N, GUAN N, et al. Miriam: Exploiting elastic kernels for real-time multi-DNN inference on edge GPU[C]//Proceedings of the 21st ACM Conference on Embedded Networked Sensor Systems. New York: ACM, 2023: 97-110. DOI: 10.1145/3625687.3625789.
- [18] GUJARATI A, KARIMI R, ALZAYAT S, et al. Serving DNNs like Clockwork: Performance predictability from the bottom up[C]//14th USENIX Symposium on Operating Systems Design and Implementation. USENIX Association, 2020: 443-462. <https://www.usenix.org/conference/osdi20/presentation/gujarati>.

- [19] ZHANG G, ZHANG Y, HUANG X, et al. Coinf: QoS-aware DRL-based inference task scheduling framework with batching processing[J]. ACM Transactions on Embedded Computing Systems, 2026, 25(1): Article 3, 1-20. DOI: 10.1145/3777373.
- [20] CHEN J, XU F, GU Y, et al. HarmonyBatch: Batching multi-SLO DNN inference with heterogeneous serverless functions[C]//2024 IEEE/ACM 32nd International Symposium on Quality of Service. IEEE, 2024: 1-10. DOI: 10.1109/IWQoS61813.2024.10682915.
- [21] NGUYEN T T, LIEBE L, TAU N Q, et al. OctopInf: Workload-aware inference serving for edge video analytics[C]//2025 IEEE International Conference on Pervasive Computing and Communications. IEEE, 2025: 128-137. DOI: 10.1109/PerCom64205.2025.00032.
- [22] 魏学超, 周哲, 徐盈辉, 等. PetS: 针对参数高效 Transformer 模型的可扩展推理服务系统 [J]. 计算机研究与发展, 2025, 62(7): 1816-1832. DOI: 10.7544/isn1000-1239.202440206.
- [23] 吴舒霞, 张俊杰, 陈德珑, 等. 面向边缘实时视频分析的资源高效持续学习框架 [J]. 系统仿真学报, 2026, 38(2): 294-306. DOI: 10.16182/j.issn1004731x.joss.25-0590.
- [24] XU J, CHEN L, ZHOU P. Joint service caching and task offloading for mobile edge computing in dense networks[C]//IEEE INFOCOM 2018. IEEE, 2018: 207-215. DOI: 10.1109/INFOCOM.2018.8485977.
- [25] 徐建, 苏圣超, 汪义旺. 车联网环境下基于深度强化学习的多 DAG 应用部分卸载 [J]. 工程科学学报, 2025, 47(12): 2498-2509. DOI: 10.13374/j.issn2095-9389.2024.09.05.004.
- [26] LIANG Y, ZHANG S, WU J. Cost-efficient delay-bounded dependent task offloading with service caching at edges[J]. IEEE Transactions on Computers, 2025, 74(11): 3568-3581. DOI: 10.1109/TC.2025.3598749.

- [27] 梁吉申, 张冬雪, 余刚, 等. 面向时空差异化需求的任务卸载和服务缓存联合优化 [J]. 陆军工程大学学报, 2025, 4(1): 10-19. DOI: 10.12018/j.issn.2097-0730.20240830002.
- [28] 唐朝刚, 李召, 肖硕, 等. 一种面向车载边缘计算基于服务缓存的任务协同卸载算法 [J]. 计算机学报, 2025, 48(4): 864-876. DOI: 10.11897/SP.J.1016.2025.00864.
- [29] 郑家瑜, 麦著学, 陈哲毅. 数字孪生云边网络下服务缓存与计算卸载优化 [J]. 系统仿真学报, 2025, 37(11): 2741-2753. DOI: 10.16182/j.issn1004731x.joss.24-0574.
- [30] 左琳立, 夏士超, 李云, 等. 车联网的服务缓存和任务迁移机制 [J]. 电子与信息学报, 2025, 47(8): 2563-2572. DOI: 10.11999/JEIT241097.
- [31] YU S, CHEN X, ZHOU Z, et al. When deep reinforcement learning meets federated learning: Intelligent multitimescale resource management for multiaccess edge computing in 5G ultradense network[J]. IEEE Internet of Things Journal, 2021, 8(4): 2238-2251. DOI: 10.1109/JIOT.2020.3026589.
- [32] CHEN G, LIN W, SHI F, et al. PAWSSP: A two-stage parallelism-aware algorithm for joint workflow scheduling and service placement in edge computing[J]. IEEE Transactions on Services Computing, 2026, 19(1): 572-588. DOI: 10.1109/TSC.2025.3643326.
- [33] GOLZARI OSKOU M R, SANSÒ B. Distributed dependency-aware task offloading and service caching in cloudlet-based edge computing networks[J]. IEEE Transactions on Services Computing, 2026, 19(2): 1120-1133. DOI: 10.1109/TSC.2026.3664339.
- [34] SUTTON R S, PRECUP D, SINGH S. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning[J]. Artificial Intelligence, 1999, 112(1-2): 181-211. DOI: 10.1016/S0004-3702(99)00052-1.

- [35] CHO K, VAN MERRIËNBOER B, GULCEHRE C, et al. Learning phrase representations using RNN encoder-decoder for statistical machine translation[C]//Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics, 2014: 1724-1734. DOI: 10.3115/v1/D14-1179.
- [36] HAARNOJA T, ZHOU A, ABBEEL P, et al. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor[C]//Proceedings of the 35th International Conference on Machine Learning. PMLR, 2018, 80: 1861-1870. <https://proceedings.mlr.press/v80/haarnoja18b.html>.
- [37] CHRISTODOULOU P. Soft actor-critic for discrete action settings[EB/OL]. (2019-10-18)[2026-09-23]. arXiv:1910.07207. DOI: 10.48550/arXiv.1910.07207.
- [38] MNIH V, KAVUKCUOGLU K, SILVER D, et al. Human-level control through deep reinforcement learning[J]. Nature, 2015, 518(7540): 529-533. DOI: 10.1038/nature14236.
- [39] VAN HASSELT H, GUEZ A, SILVER D. Deep reinforcement learning with double Q-learning[C]//Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence. AAAI Press, 2016: 2094-2100. DOI: 10.1609/aaai.v30i1.10295.
- [40] WANG Z, SCHAUL T, HESSEL M, et al. Dueling network architectures for deep reinforcement learning[C]//Proceedings of the 33rd International Conference on Machine Learning. PMLR, 2016, 48: 1995-2003. <https://proceedings.mlr.press/v48/wangf16.html>.
- [41] HOWARD A, SANDLER M, CHU G, et al. Searching for MobileNetV3[C]//Proceedings of the IEEE/CVF International Conference on Computer Vision. IEEE, 2019: 1314-1324. DOI: 10.1109/ICCV.2019.00140.
- [42] TAN M, LE Q V. EfficientNet: Rethinking model scaling for convolutional neural networks[C]//Proceedings of the 36th International Conference on Machine Learning. PMLR, 2019, 97: 6105-6114. <https://proceedings.mlr.press/v97/tan19a.html>.

- [43] HE K, ZHANG X, REN S, et al. Deep residual learning for image recognition[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. IEEE, 2016: 770-778. DOI: 10.1109/CVPR.2016.90.
- [44] DENG J, DONG W, SOCHER R, et al. ImageNet: A large-scale hierarchical image database[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. IEEE, 2009: 248-255. DOI: 10.1109/CVPR.2009.5206848.
- [45] OLSTON C, FIEDEL N, GOROVVOY K, et al. TensorFlow-Serving: Flexible, high-performance ML serving[C]//Workshop on ML Systems at NIPS. 2017. <https://arxiv.org/abs/1712.06139>.
- [46] NVIDIA. Batchers: Triton Inference Server User Guide[EB/OL]. [2026-09-23]. https://docs.nvidia.com/deeplearning/triton-inference-server/archives/triton-inference-server-2580/user-guide/docs/user_guide/batcher.html.
- [47] CUI W, WEI M, CHEN Q, et al. Ebird: Elastic batch for improving responsiveness and throughput of deep learning services[C]//Proceedings of the IEEE International Conference on Computer Design. IEEE, 2019: 497-505. DOI: 10.1109/ICCD46524.2019.00075.
- [48] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need[C]//Advances in Neural Information Processing Systems 30. 2017: 5998-6008.
- [49] KINGMA D P, BA J. Adam: A method for stochastic optimization[C]//International Conference on Learning Representations. 2015.
- [50] DEELMAN E, VAHI K, JUVE G, et al. Pegasus, a workflow management system for science automation[J]. Future Generation Computer Systems, 2015, 46: 17-35. DOI: 10.1016/j.future.2014.10.008.
- [51] CHEN W, DEELMAN E. WorkflowSim: A toolkit for simulating scientific workflows in distributed environments[C]//2012 IEEE 8th International Conference on E-Science. IEEE, 2012: 1-8. DOI: 10.1109/eScience.2012.6404430.